**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**
**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN**



**BÙI VĂN DŨNG**

**LUẬN VĂN THẠC SĨ**

# PHÁT HIỆN XÂM NHẬP (IDS) DỰA TRÊN APACHE SPARK CHO BẢO MẬT MẠNG IoT

**THẠC SĨ NGÀNH CÔNG NGHỆ THÔNG TIN**

**TP. HỒ CHÍ MINH, 2026**

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÙI VĂN DŨNG – 220201014

LUẬN VĂN THẠC SĨ

# PHÁT HIỆN XÂM NHẬP (IDS) DỰA TRÊN APACHE SPARK CHO BẢO MẬT MẠNG IoT

THẠC SĨ NGÀNH CÔNG NGHỆ THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN
TS. NGUYỄN VĂN DŨ
TS. ĐỖ TRÍ NHỰT

TP. HỒ CHÍ MINH, 2026

# LỜI CẢM ƠN

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành đến quý thầy cô của Trường Đại học Công nghệ Thông tin, Đại học Quốc gia Thành phố Hồ Chí Minh đã tận tình chỉ dạy, giúp tôi tiếp thu được nhiều kiến thức quý báu trong suốt thời gian học tập tại trường, cũng như tạo điều kiện thuận lợi cho tôi thực hiện luận văn này. Kính chúc thầy cô luôn dồi dào sức khỏe và gặt hái được nhiều thành công trong tương lai.

Đặc biệt, tôi xin chân thành cảm ơn hai thầy TS. Nguyễn Văn Dũ và TS. Đỗ Trí Nhựt đã tận tình giúp đỡ, hướng dẫn, cung cấp cho tôi những kiến thức, kỹ năng cần thiết để thực hiện nghiên cứu và hoàn thành đề tài này.

Tôi cũng xin gửi lời cảm ơn đến ThS. Nguyễn Vũ Thái đã đồng hành, hỗ trợ tôi trong quá trình thực nghiệm và công bố bài báo khoa học liên quan đến đề tài.

Mặc dù đã cố gắng trong quá trình nghiên cứu, nhưng do kiến thức còn hạn chế nên luận văn vẫn còn nhiều thiếu sót. Vì vậy, tôi rất mong nhận được sự đóng góp ý kiến của Quý Thầy Cô để đề tài của tôi được hoàn thiện hơn.

Tôi xin chân thành cảm ơn!

**Bùi Văn Dũng**

# TÓM TẮT

Sự phát triển nhanh chóng của Internet of Things (IoT) đã mang lại nhiều tiện ích trong các lĩnh vực như nhà thông minh, công nghiệp, y tế và giao thông. Tuy nhiên, đi kèm là nguy cơ ngày càng gia tăng về các mối đe dọa an ninh mạng, đặc biệt khi thiết bị IoT thường hạn chế tài nguyên tính toán và bảo mật kém. Trong bối cảnh đó, Intrusion Detection Systems (IDS) là một hướng phòng thủ nhằm phát hiện và ngăn chặn các hành vi tấn công bất thường vào hệ thống mạng IoT.

Luận văn này tập trung nghiên cứu, thiết kế và triển khai một hệ thống IDS dựa trên nền tảng xử lý dữ liệu lớn Apache Spark. Spark có khả năng xử lý dữ liệu phân tán, tốc độ cao và hỗ trợ hiệu quả các thuật toán học máy qua thư viện MLlib. Hệ thống phát hiện hành vi xâm nhập trong dữ liệu mạng IoT bằng các mô hình học máy truyền thống, học tổ hợp và học sâu MLP (cùng một autoencoder đóng vai trò cổng lọc bất thường ở tầng biên).

Luận văn phân tích đặc điểm dữ liệu mạng IoT, xử lý dữ liệu đầu vào từ tập dữ liệu công khai CICIDS2017, huấn luyện và đánh giá mô hình theo các chỉ số độ chính xác (Accuracy), precision, recall và F1-score, đồng thời ghi nhận thời gian huấn luyện và kích thước mô hình. Quá trình đánh giá áp dụng quy trình loại trừ rò rỉ dữ liệu (loại đặc trưng rò rỉ nhãn, loại bỏ trùng lặp ở mức đặc trưng) nhằm bảo đảm kết quả phản ánh đúng hiệu năng mô hình. Bên cạnh đó, hệ thống được kiểm thử hiệu năng trên cụm thiết bị biên nhằm khảo sát khả năng mở rộng và tính khả thi triển khai thực tiễn.

Dưới cấu hình đánh giá loại trừ rò rỉ dữ liệu, lựa chọn đặc trưng SHAP Top-30 giữ được hiệu năng gần tương đương cấu hình đầy đủ trong khi giảm khoảng 60% số đặc trưng: Random Forest đạt F1 0,9955 và XGBoost (XGB) đạt 0,9970 trên CICIDS2017. Các khác biệt giữa phương pháp được kiểm chứng bằng kiểm định hoán vị ghép cặp và khoảng tin cậy hiệu chỉnh Nadeau–Bengio. Học tổ hợp (Hybrid Bagging 0,9966) không vượt được mô hình đơn tốt nhất, trong khi chi phí huấn luyện và dung lượng mô hình cao hơn nhiều lần. Trên cụm hai thiết bị biên Jetson Orin Nano Super, kiến trúc tách pipeline (cổng lọc autoencoder trên một bo, bộ phân loại PySpark trên bo còn lại) duy trì được 22,4 luồng/giây, gấp khoảng 9 lần một nút (2,5 luồng/giây), với độ trễ đầu cuối p95 khoảng 7 giây (đã gồm thời gian chờ hàng đợi) và năng lượng giảm từ 3,04 xuống 0,60 J mỗi dự đoán. Cái giá là recall: phát lại trọn 10.000 luồng, hệ thống không báo nhầm luồng benign nào nhưng chỉ phát hiện 69,5% luồng tấn công (F1 0,820), và mọi luồng bị sót đều do cổng lọc bỏ qua. Cùng mô hình chạy qua ONNX Runtime nhanh hơn PySpark khoảng 7.200 lần, cho thấy

nút thắt nằm ở engine suy luận chứ không ở mô hình. Khía cạnh xử lý phân tán được kiểm chứng ở mức minh chứng khái niệm (proof-of-concept), chưa khẳng định quy mô dữ liệu lớn thực thụ. Kết quả cho thấy một quy trình phát hiện xâm nhập thời gian thực khả thi trên thiết bị biên, đồng thời là cơ sở để kết hợp học máy với nền tảng dữ liệu lớn cho bảo mật IoT.

# MỤC LỤC

**LỜI CẢM ƠN** i

**TÓM TẮT** ii

**MỤC LỤC** iv

**DANH MỤC CÁC BẢNG** vii

**DANH MỤC CÁC HÌNH VẼ VÀ ĐỒ THỊ** viii

**DANH MỤC TỪ VIẾT TẮT** ix

**CHƯƠNG 1. MỞ ĐẦU** 1

1.1 Lý do chọn đề tài . . . 1

1.2 Các nghiên cứu liên quan . . . 1

1.3 Mục tiêu, đối tượng và phạm vi nghiên cứu . . . 4

1.3.1 Mục tiêu nghiên cứu . . . 4

1.3.2 Đối tượng nghiên cứu . . . 5

1.3.3 Phạm vi nghiên cứu . . . 5

1.3.4 Câu hỏi nghiên cứu . . . 6

1.4 Phương pháp nghiên cứu . . . 7

1.5 Các đóng góp chính của đề tài . . . 7

1.6 Công trình khoa học đã công bố . . . 8

1.7 Cấu trúc của luận văn . . . 9

**CHƯƠNG 2. CƠ SỞ LÝ THUYẾT** 11

2.1 Tổng quan về IoT . . . 11

2.2 Hệ thống phát hiện xâm nhập (IDS) . . . 11

2.3 Apache Spark và xử lý dữ liệu lớn . . . 12

2.3.1 Dữ liệu lớn . . . 12

2.3.2 Các thành phần của Apache Spark . . . 14

2.3.3 Kiến trúc của Apache Spark . . . 15

2.4 Học máy trong phát hiện xâm nhập . . . 16

2.4.1 Một số thuật toán học máy . . . 17

2.4.2 Một số thuật toán học tổ hợp . . . . . 21

**CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN 24**

3.1 Thu thập dữ liệu và tiền xử lý . . . . . 24

3.2 Lựa chọn đặc trưng và giảm chiều dữ liệu . . . . . 27

3.3 Huấn luyện mô hình . . . . . 31

3.3.1 Huấn luyện các mô hình cơ sở . . . . . 31

3.3.2 Các phương pháp Ensemble Learning . . . . . 32

3.4 Đánh giá mô hình . . . . . 36

3.5 Tối ưu hóa siêu tham số và huấn luyện mô hình . . . . . 38

3.6 Triển khai biên . . . . . 40

**CHƯƠNG 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN 41**

4.1 Môi trường thực nghiệm . . . . . 41

4.2 Kết quả thực nghiệm . . . . . 42

4.2.1 Kịch bản 1: Đánh giá hiệu năng cơ sở (Baseline) . . . . . 45

4.2.2 Kịch bản 2: So sánh chiến lược giảm chiều dữ liệu . . . . . 45

4.2.3 Kịch bản 3: Tối ưu hóa siêu tham số . . . . . 48

4.2.4 Phân tích khả năng giải thích với SHapley Additive exPlanations (SHAP) . . . . . 48

4.2.5 Phân tích rò rỉ dữ liệu (data leakage) . . . . . 49

4.2.6 Phân tích hiệu năng hệ thống . . . . . 50

4.2.7 Kiểm định ý nghĩa thống kê . . . . . 52

4.2.8 Kiểm tra độ ổn định trong phân phối . . . . . 53

4.2.9 Tổng quát hóa liên bộ dữ liệu . . . . . 53

4.2.10 Đánh giá đa lớp theo loại tấn công . . . . . 59

4.3 Thực nghiệm và đánh giá hiệu năng mô hình trên Jetson Orin Nano Super . 65

4.3.1 Các công cụ sử dụng trong hệ thống . . . . . 65

4.3.2 Mô hình hệ thống đề xuất . . . . . 67

4.3.3 Lựa chọn mô hình cho thiết bị biên . . . . . 70

4.3.4 Kết quả huấn luyện các mô hình ứng viên trên cụm Spark . . . . . 71

4.3.5 Kết quả đo hiệu năng (benchmark) trên Jetson Orin Nano Super . . 75

4.3.6 Phân tích và lựa chọn mô hình . . . . . 76

4.3.7 So sánh engine suy luận: chi phí của việc đồng nhất stack . . . . . 78

4.3.8 Kết quả ba chế độ triển khai phân tán . . . . . 80
4.4 Thảo luận và so sánh tổng hợp . . . . . 83

**CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 85**
5.1 Những đóng góp chính của luận văn . . . . . 85
5.2 Hạn chế của đề tài . . . . . 87
5.3 Hướng phát triển trong tương lai . . . . . 88

**PHỤ LỤC A. CÔNG TRÌNH KHOA HỌC ĐÃ CÔNG BỐ 90**
A.1 Bài báo tại Hội nghị FAIR 2026 . . . . . 90
A.2 Bài báo tại Hội nghị SOICT 2026 . . . . . 95

**TÀI LIỆU THAM KHẢO 96**

# DANH MỤC CÁC BẢNG


3.1 Mô tả bộ dữ liệu CICIDS2017 . . . . . 25
3.2 Mô tả bộ dữ liệu CSE-CIC-IDS2018 (dùng cho thí nghiệm liên bộ dữ liệu) . 26
3.3 Tám bộ phân loại cơ sở được sử dụng trong pipeline . . . . . 31
3.4 Các phương pháp học tổ hợp (ensemble) sử dụng trong luận văn . . . . . 32
3.5 Các tiêu chí đánh giá mô hình . . . . . 36
3.6 Ma trận nhầm lẫn của mô hình phát hiện xâm nhập . . . . . 37
3.7 Thông số phần cứng thiết bị biên . . . . . 40

4.1 Tham số cố định của các mô hình học máy . . . . . 43
4.2 Hiệu năng phân loại kịch bản Baseline . . . . . 45
4.3 So sánh hiệu năng tốt nhất của các phương pháp giảm chiều . . . . . 46
4.4 So sánh lựa chọn đặc trưng: RF Importance vs SHAP (Top-30) . . . . . 47
4.5 Kết quả hiệu năng phân loại kịch bản Tối ưu hóa siêu tham số . . . . . 48
4.6 Phân tích loại bỏ (ablation) rò rỉ đặc trưng cổng đích . . . . . 50
4.7 Ảnh hưởng của `destination_port` theo từng thuật toán . . . . . 51
4.8 So sánh thời gian xử lý và kích thước mô hình kịch bản Baseline . . . . . 51
4.9 Độ ổn định qua nhiều lần lấy mẫu và kiểm định ghép cặp . . . . . 53
4.10 Kiểm tra độ ổn định trong phân phối trên mẫu con tập kiểm tra . . . . . 53
4.11 Tổng quát hóa liên bộ dữ liệu CICIDS2017 và CSE-CIC-IDS2018 . . . . . 54
4.12 Thích ứng miền không giám sát trên tác vụ liên bộ dữ liệu . . . . . 58
4.13 Ngân sách nhãn miền đích: 1% đủ để phục hồi hiệu năng . . . . . 59
4.14 Hiệu năng theo từng lớp tấn công . . . . . 60
4.15 Điểm vận hành của cổng lọc bất thường theo ngưỡng quantile . . . . . 70
4.16 Các mô hình ứng viên cho triển khai biên (giai đoạn chọn mô hình) . . . . . 73
4.17 So sánh hiệu năng ba mô hình ứng viên trên Jetson Orin Nano Super . . . . 75
4.18 Chi phí suy luận một nút của cùng một mô hình: PySpark, NumPy và ONNX 78
4.19 Chi phí suy luận theo kích thước lô: PySpark và ONNX Runtime . . . . . 79
4.20 Kiểm tra chịu tải của từng chế độ triển khai . . . . . 81
4.21 Kết quả từng chế độ triển khai ở mức tải cao nhất còn theo kịp . . . . . 81
4.22 Pipeline split khi phát lại CICIDS2017 và CSE-CIC-IDS2018 . . . . . 83

# DANH MỤC CÁC HÌNH VẼ VÀ ĐỒ THỊ


2.1 Mô hình 5V của dữ liệu lớn . . . . . 13
2.2 Các thành phần của Apache Spark . . . . . 14
2.3 Sơ đồ kiến trúc hệ thống Apache Spark . . . . . 15

3.1 Quy trình tổng thể huấn luyện, đánh giá và triển khai mô hình đề xuất . . . 24
3.2 Sơ đồ phân chia dữ liệu CICIDS2017 . . . . . 29

4.1 Cụm huấn luyện phân tán Apache Spark Standalone . . . . . 44
4.2 Top 30 đặc trưng quan trọng nhất theo SHAP (mô hình XGBoost) . . . . . 62
4.3 Biểu đồ SHAP Summary (beeswarm) . . . . . 63
4.4 Mức thổi phồng F1 do `destination_port` theo từng thuật toán . . . . 63
4.5 Ma trận nhầm lẫn đa lớp theo loại tấn công, Random Forest dùng toàn bộ đặc trưng (chuẩn hóa theo hàng) . . . . . 64
4.6 Kiến trúc hệ thống IDS phân tán trên cụm 2 Jetson Orin Nano Super . . . . 67
4.7 Giao diện terminal của pipeline IDS trên hai nút Jetson . . . . . 72
4.8 Dashboard Grafana giám sát hiệu năng IDS theo thời gian thực . . . . . 73
4.9 Dashboard Grafana chi tiết các cuộc tấn công được phát hiện . . . . . 73
4.10 Dashboard Grafana tại thời điểm tải đỉnh của chế độ tách pipeline . . . . . 74
4.11 Email cảnh báo tấn công gửi qua Mailtrap . . . . . 74
4.12 Thông báo cảnh báo tấn công gửi qua Slack . . . . . 75
4.13 So sánh hiệu năng ba mô hình trên Jetson Orin Nano Super . . . . . 76
4.14 Biểu đồ radar đánh đổi giữa các mô hình trên thiết bị biên . . . . . 77
4.15 Chi phí suy luận theo kích thước lô: PySpark và ONNX Runtime . . . . . 79
4.16 Thông lượng duy trì và năng lượng mỗi dự đoán của các cấu hình triển khai 80

A.1 Thư chấp nhận đăng bài của Hội nghị FAIR 2026 . . . . . 91
A.2 Giấy chứng nhận trình bày bài báo tại Hội nghị FAIR 2026 . . . . . 91
A.3 Thư chấp nhận đăng bài của SOICT 2026 . . . . . 95

# DANH MỤC TỪ VIẾT TẮT

| | |
|---|---|
| **AC** | accuracy |
| **API** | Application Programming Interface |
| **BD** | Big Data |
| **CNN** | Convolutional Neural Network |
| **CPU** | Central Processing Unit |
| **CSV** | Comma-Separated Values |
| **CV** | Cross-Validation |
| **DDoS** | Distributed Denial of Service |
| **DoS** | Denial of Service |
| **DT** | Decision Trees |
| **EL** | Ensemble Learning |
| **FN** | False Negative |
| **FP** | False Positive |
| **FPR** | False Positive Rate |
| **FS** | F1-score |
| **FTP** | File Transfer Protocol |
| **GBT** | Gradient Boosted Trees |
| **GPU** | Graphics Processing Unit |
| **GS** | Grid Search |
| **HTTP** | Hypertext Transfer Protocol |
| **IDS** | Intrusion Detection Systems |
| **IoT** | Internet of Things |
| **IP** | Internet Protocol |
| **JVM** | Java Virtual Machine |
| **kNN** | k-Nearest Neighbors |
| **LGBM** | LightGBM |
| **LR** | Logistic Regression |
| **LSTM** | Long Short-Term Memory |
| **ML** | Machine Learning |
| **MLP** | Multilayer Perceptron |

| | |
|---|---|
| **NB** | Naive Bayes |
| **ONNX** | Open Neural Network Exchange |
| **PCA** | Principal Component Analysis |
| **PR** | precision |
| **RAM** | Random Access Memory |
| **RE** | recall |
| **RF** | Random Forest |
| **RFI** | Random Forest Importance |
| **RNN** | Recurrent Neural Network |
| **SCP** | Secure Copy Protocol |
| **SHAP** | SHapley Additive exPlanations |
| **SQL** | Structured Query Language |
| **SSH** | Secure Shell |
| **SVM** | Support Vector Machine |
| **TN** | True Negative |
| **TP** | True Positive |
| **XGB** | XGBoost |
| **XSS** | Cross-Site Scripting |

# CHƯƠNG 1. MỞ ĐẦU

## 1.1 Lý do chọn đề tài

IoT đang phát triển nhanh và được ứng dụng rộng rãi trong nhiều lĩnh vực như nhà thông minh, y tế, giao thông, công nghiệp thông minh... Sự bùng nổ về số lượng thiết bị IoT dẫn đến sự gia tăng đáng kể về lưu lượng và loại hình dữ liệu truyền trong mạng. Tuy nhiên, các thiết bị IoT thường có khả năng xử lý hạn chế và thiếu các cơ chế bảo mật mạnh, khiến chúng trở thành mục tiêu dễ bị tấn công bởi tin tặc.

Để đối phó với các mối đe dọa an ninh mạng, các hệ thống phát hiện xâm nhập (IDS) đang được nghiên cứu và phát triển rộng rãi. Đặc biệt, trong những năm gần đây, việc kết hợp học máy (Machine Learning (ML)) với các nền tảng xử lý dữ liệu lớn giúp hệ thống nâng cao độ chính xác phát hiện và thích nghi tốt hơn với môi trường mạng thay đổi liên tục. Cần lưu ý rằng bộ phân loại chính trong luận văn là mô hình học có giám sát trên CICIDS2017 nên chỉ phát hiện được các họ tấn công đã biết. Khả năng phát hiện hành vi bất thường chưa từng thấy (nếu có) đến từ cổng lọc bất thường (anomaly gate) học không giám sát, chứ không phải từ bộ phân loại. Tuy nhiên, phần lớn các nghiên cứu hiện nay tập trung vào việc xây dựng mô hình học máy trên tập dữ liệu tĩnh, dung lượng nhỏ và huấn luyện ngoại tuyến (offline).

Trong khi đó, mạng IoT hiện đại sinh ra lượng dữ liệu rất lớn, liên tục theo thời gian thực đòi hỏi các hệ thống IDS có khả năng xử lý dữ liệu lớn (Big Data (BD)) hiệu quả, có thể hoạt động trong môi trường phân tán và thời gian thực.

Apache Spark là một nền tảng xử lý dữ liệu lớn, với khả năng xử lý song song, phân tán và dễ dàng tích hợp với thư viện học máy Spark MLlib. Đây là một giải pháp tiềm năng để xây dựng hệ thống IDS hiện đại đáp ứng yêu cầu về quy mô và thời gian thực trong môi trường IoT.

## 1.2 Các nghiên cứu liên quan

Trong những năm gần đây, các hệ thống phát hiện xâm nhập (Intrusion Detection System, IDS) dựa trên học máy và học sâu đã thu hút nhiều sự quan tâm của cộng đồng nghiên cứu nhằm nâng cao độ chính xác và khả năng thích ứng trước các hình thức tấn công mạng ngày càng tinh vi [1, 2, 3]. Nhiều công trình tập trung khai thác các kiến trúc học sâu như autoencoder, mạng tích chập (Convolutional Neural Network (CNN)) và mạng nơ-ron hồi

quy (Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM)) để tự động trích xuất đặc trưng và phát hiện bất thường trong lưu lượng mạng. Tuy nhiên, phần lớn các nghiên cứu hiện nay vẫn được triển khai trong môi trường đơn máy, xử lý dữ liệu tĩnh với quy mô tương đối nhỏ, và chưa tích hợp các nền tảng xử lý dữ liệu lớn như Apache Spark hay Hadoop nhằm đáp ứng yêu cầu về khả năng mở rộng và xử lý thời gian thực trong các hệ thống mạng hiện đại.

Một số nghiên cứu đã đề xuất các mô hình IDS học sâu dựa trên các kiến trúc như autoencoder, CNN và LSTM nhằm nâng cao khả năng học đặc trưng và phát hiện xâm nhập mạng. Các mô hình này được đánh giá trên các tập dữ liệu chuẩn như NSL-KDD, CIC-DDoS2019 và CICIDS2017, cho thấy hiệu quả cao trong việc phát hiện bất thường. Tuy nhiên, quá trình huấn luyện và đánh giá vẫn chủ yếu được thực hiện trong môi trường cục bộ và theo cơ chế xử lý theo lô (batch) [4, 5]. Tương tự, các mô hình lai dựa trên LSTM (chẳng hạn CNN-LSTM, hay LSTM được tối ưu hóa siêu tham số) cho bài toán phát hiện bất thường trong mạng cũng đạt được kết quả khả quan, nhưng vẫn triển khai trên nền tảng đơn nút (single-node) và chưa khai thác được khả năng mở rộng của các hệ thống Big Data [6, 7]. Gần đây, một số nghiên cứu đã triển khai IDS học máy trực tiếp trên phần cứng biên NVIDIA Jetson [8, 9], hay trên môi trường biên mô phỏng ràng buộc tài nguyên [10]. Xu hướng này tiếp tục được mở rộng trong các công trình mới nhất: mô hình sinh (generative) được chạy trực tiếp trên Jetson Orin Nano để vừa phân loại lưu lượng vừa sinh dữ liệu huấn luyện tổng hợp [11], các mô hình rút gọn được thiết kế riêng cho lưu lượng điều khiển công nghiệp [12]. Ngay cả một công trình tự giới thiệu là tiết kiệm năng lượng cũng chưa đo năng lượng và độ trễ suy luận trên thiết bị thật [13]. Tuy vậy, điểm chung của các nghiên cứu này là vẫn dừng ở quy mô *đơn nút*: bộ phát hiện được đo trên một bo mạch duy nhất, nên câu hỏi phân bổ tải suy luận trên nhiều nút biên và lợi ích thực tế của việc bổ sung nút thứ hai vẫn còn bỏ ngỏ. Ở hướng ngược lại, việc ghép Kafka với Spark để phát hiện xâm nhập phân tán đã được nghiên cứu [14], nhưng công trình đó chỉ báo cáo độ chính xác và thời gian thực thi, không mô tả phần cứng chạy thí nghiệm, không công bố mức tiêu thụ tài nguyên (CPU, bộ nhớ, năng lượng) và không đánh giá trên thiết bị biên.

Bên cạnh các phương pháp học sâu, các kỹ thuật học máy truyền thống như Support Vector Machine (Support Vector Machine (SVM)), k-Nearest Neighbors (k-Nearest Neighbors (kNN)) và Decision Tree vẫn được sử dụng rộng rãi trong các hệ thống IDS. Một số nghiên cứu đã tiến hành so sánh hiệu năng của các mô hình này trên các bộ dữ liệu phổ biến như NSL-KDD và UNSW-NB15, qua đó chỉ ra rằng các phương pháp học máy truyền

thống có tiềm năng ứng dụng thực tiễn cao. Tuy nhiên, các mô hình này chủ yếu hoạt động ở chế độ offline, xử lý dữ liệu theo từng đợt và chưa đáp ứng được yêu cầu về xử lý dữ liệu liên tục trong môi trường mạng động [15]. Ngoài ra, một số công trình so sánh nhiều mô hình học máy và học sâu cho IDS, ghi nhận các mô hình dựa trên cây như Random Forest cho kết quả tốt trên các bộ dữ liệu lớn, nhưng toàn bộ thực nghiệm vẫn được thực hiện trên một máy đơn [16].

Ngoài các kiến trúc nêu trên, nhiều mô hình học sâu khác như CNN, RNN và các mô hình lai CNN-LSTM cũng đã được nghiên cứu và đánh giá cho bài toán phát hiện xâm nhập mạng. Các nghiên cứu này cho thấy khả năng cải thiện đáng kể độ chính xác so với các phương pháp truyền thống khi thử nghiệm trên các bộ dữ liệu như CICIDS2018. Tuy nhiên, phần lớn các mô hình vẫn thiếu khả năng triển khai trong môi trường thời gian thực và chưa tận dụng được sức mạnh của các nền tảng xử lý dữ liệu lớn [17].

Trong bối cảnh Internet of Vehicles (IoV), một số nghiên cứu đã đề xuất các hệ thống IDS dựa trên học máy và học sâu nhằm bảo vệ các mạng phương tiện thông minh. Các phương pháp này bao gồm việc sử dụng các mô hình dựa trên cây (tree-based), các hệ thống phát hiện xâm nhập đa tầng, cũng như các mô hình ensemble để cải thiện độ chính xác phát hiện tấn công. Mặc dù đạt được kết quả khả quan trên các bộ dữ liệu thử nghiệm, các hệ thống này vẫn chủ yếu được huấn luyện và đánh giá trong môi trường ngoại tuyến, xử lý dữ liệu theo batch và chưa tích hợp các nền tảng xử lý dữ liệu lớn hoặc cơ chế xử lý dữ liệu theo luồng [18, 19, 20].

Bên cạnh đó, một số hướng tiếp cận gần đây đã nghiên cứu việc áp dụng học bán giám sát, học liên kết (federated learning) và học tăng dần (incremental/active learning) cho bài toán IDS nhằm nâng cao khả năng thích ứng của mô hình và bảo vệ quyền riêng tư dữ liệu. Các phương pháp này cho phép phân tán quá trình huấn luyện trên nhiều nút hoặc thiết bị IoT khác nhau. Một khảo sát hệ thống gần đây đã tổng hợp toàn cảnh hướng nghiên cứu IDS dựa trên học liên kết [21]. Tuy nhiên, cần lưu ý rằng việc phân tán ở đây diễn ra trong pha *huấn luyện*, còn suy luận vẫn thực hiện cục bộ trên từng nút. Ngoài ra, dữ liệu vẫn chủ yếu được xử lý theo từng đợt, thiếu cơ chế xử lý luồng tập trung và chưa khai thác đầy đủ khả năng mở rộng cũng như song song hóa của các nền tảng Big Data [22, 23, 24, 25, 26].

Từ tổng quan các công trình nghiên cứu nêu trên, có thể nhận thấy rằng phần lớn các hệ thống IDS hiện tại vẫn hoạt động trong môi trường ngoại tuyến, chưa hỗ trợ xử lý dữ liệu liên tục theo thời gian thực và chưa tận dụng được khả năng mở rộng của các nền tảng xử lý dữ liệu lớn. Do đó, luận văn này hướng đến việc tích hợp các mô hình học máy dựa trên

nền tảng Apache Spark nhằm xây dựng một hệ thống phát hiện xâm nhập có khả năng xử lý dữ liệu lớn, hỗ trợ thời gian thực và áp dụng hiệu quả trong môi trường mạng IoT hiện đại.

## 1.3 Mục tiêu, đối tượng và phạm vi nghiên cứu

### 1.3.1 Mục tiêu nghiên cứu

Mục tiêu của đề tài "PHÁT HIỆN XÂM NHẬP (IDS) DỰA TRÊN APACHE SPARK CHO BẢO MẬT MẠNG IoT" là xây dựng một hệ thống phát hiện xâm nhập có khả năng phân tích và phát hiện các hành vi bất thường hoặc tấn công mạng trong môi trường IoT bằng cách kết hợp các mô hình học máy với nền tảng xử lý dữ liệu lớn Apache Spark nhằm đạt được:

- Phát hiện hiệu quả các dạng tấn công mạng phổ biến (như DoS, probing, brute-force) trên các tập dữ liệu hiện đại như CICIDS2017.
- Thiết kế và triển khai pipeline xử lý dữ liệu phân tán và song song sử dụng Apache Spark, bao gồm các bước: tiền xử lý dữ liệu, trích xuất đặc trưng, huấn luyện mô hình và phát hiện thời gian thực.
- Khảo sát hiệu năng hệ thống, hướng tới khả năng mở rộng, độ trễ thấp và tính linh hoạt, đồng thời kiểm chứng khả năng xử lý phân tán ở mức minh chứng khái niệm (proof-of-concept) trên cụm thiết bị biên, làm tiền đề cho việc triển khai trên các hệ thống lớn hoặc mạng IoT nhiều thiết bị.
- So sánh và đánh giá các mô hình học máy phổ biến như Decision Tree, Support Vector Machine, Logistic Regression, Naive Bayes (NB), Random Forest, Gradient Boosted Trees (GBT), XGB, Multilayer Perceptron (MLP) và các thuật toán học tổ hợp (ensemble) trong môi trường xử lý phân tán, từ đó chọn ra mô hình phù hợp nhất.
- Áp dụng quy trình đánh giá chặt chẽ, loại trừ rò rỉ dữ liệu (loại đặc trưng rò rỉ nhãn, loại bỏ trùng lặp ở mức đặc trưng, kiểm định ý nghĩa thống kê) nhằm bảo đảm kết quả phản ánh đúng hiệu năng mô hình.
- Đề xuất giải pháp thực tiễn giúp phát hiện tấn công nhanh chóng và chính xác trong các hệ thống mạng hiện đại có yêu cầu xử lý dữ liệu lớn và liên tục.

### 1.3.2 Đối tượng nghiên cứu

Đề tài này tập trung nghiên cứu và triển khai phương pháp phát hiện xâm nhập trong lĩnh vực bảo mật mạng IoT trên nền tảng xử lý dữ liệu lớn Apache Spark. Với sự phát triển nhanh chóng của các thiết bị IoT, vấn đề bảo đảm an toàn và bảo mật cho hệ thống ngày càng trở nên cấp thiết, đặc biệt là trong việc phát hiện các hành vi bất thường hoặc tấn công mạng tiềm ẩn.

Apache Spark, với khả năng xử lý dữ liệu phân tán và tốc độ tính toán cao, cho phép xử lý khối lượng lớn dữ liệu sinh ra từ các thiết bị IoT trong thời gian thực. Trong khuôn khổ nghiên cứu này, luận văn sử dụng các tính năng của Spark như xử lý luồng dữ liệu (streaming), học máy phân tán (MLlib) và phân tích dữ liệu quy mô lớn để xây dựng mô hình phát hiện xâm nhập hiệu quả. Kết quả là một giải pháp kỹ thuật có thể áp dụng cho bài toán bảo mật IoT trong thực tiễn.

### 1.3.3 Phạm vi nghiên cứu

Đề tài tập trung vào việc nghiên cứu và áp dụng các kỹ thuật phân tích dữ liệu lớn nhằm phát hiện xâm nhập trong hệ thống mạng IoT với mục đích là sử dụng các khả năng của nền tảng Apache Spark để xử lý và phân tích dữ liệu hiệu quả. Cụ thể, phạm vi nghiên cứu bao gồm:

- **Dữ liệu đầu vào**: Dữ liệu được sử dụng là tập dữ liệu công khai CICIDS2017. Bộ CSE-CIC-IDS2018 được dùng thêm cho thí nghiệm đánh giá khác bộ dữ liệu.
- **Công nghệ sử dụng**: Sử dụng Apache Spark làm nền tảng xử lý dữ liệu lớn, đặc biệt tập trung vào các thành phần như Spark Core, Spark SQL, Spark Streaming và Spark MLlib để hỗ trợ tiền xử lý dữ liệu, huấn luyện mô hình học máy và phát hiện xâm nhập theo thời gian thực.
- **Phương pháp phát hiện xâm nhập**: Áp dụng các thuật toán học máy được triển khai và tối ưu hóa trong môi trường Spark để bảo đảm khả năng mở rộng và hiệu năng cao.
- **Môi trường triển khai**: Thử nghiệm hệ thống trên cụm hai thiết bị biên NVIDIA Jetson Orin Nano Super, với lưu lượng phát lại từ bộ dữ liệu công khai CICIDS2017, nhằm đánh giá khả năng phát hiện xâm nhập và hiệu năng vận hành.

**Lưu ý về phạm vi dữ liệu.** CICIDS2017 là bộ dữ liệu lưu lượng mạng doanh nghiệp mô phỏng, không chứa lưu lượng đặc thù IoT (ví dụ các giao thức MQTT, CoAP) hay thiết bị IoT thực. Tính chất "IoT" của đề tài do đó nằm ở *kịch bản triển khai*: toàn bộ pipeline suy luận được thiết kế, tối ưu và đo đạc trên thiết bị biên tài nguyên hạn chế (NVIDIA Jetson Orin Nano Super), vốn là lớp thiết bị làm gateway/biên trong kiến trúc IoT ba tầng, chứ không nằm ở bản chất lưu lượng của dữ liệu huấn luyện. CICIDS2017 được chọn vì là bộ chuẩn công khai, được cộng đồng sử dụng rộng rãi, có nhãn chi tiết và bao phủ các họ tấn công (DoS/DDoS, brute-force, web attack, botnet) vốn cũng xuất hiện phổ biến trong các mạng IoT. Các nguồn lỗi của bộ dữ liệu này cũng đã được nghiên cứu kỹ, cho phép áp dụng quy trình loại trừ rò rỉ một cách có căn cứ. Việc kiểm chứng trên các bộ dữ liệu IoT chuyên biệt (Bot-IoT, TON_IoT, CIC-IoT-2023) được xác định là hướng phát triển tiếp theo (Chương 5).

**Mô hình mối đe dọa (threat model).** Luận văn giả định kẻ tấn công nằm bên trong hoặc ở biên mạng IoT và tìm cách né tránh phát hiện. Phạm vi bảo vệ giới hạn ở việc phát hiện các tấn công đã biết theo kiểu CICIDS2017, dưới dạng phân loại nhị phân Benign/Attack. Các kịch bản đối kháng chủ động (adversarial/evasion) nhằm cố ý đánh lừa mô hình nằm ngoài phạm vi đánh giá của luận văn.

### 1.3.4 Câu hỏi nghiên cứu

Luận văn được định hướng bởi ba câu hỏi nghiên cứu:

- **RQ1**: Việc giảm chiều/lựa chọn đặc trưng (RF Importance, SHapley Additive exPlanations (SHAP), Principal Component Analysis (PCA)) có giữ được hiệu năng phát hiện gần với mức dùng toàn bộ đặc trưng hay không, khi số đặc trưng giảm hơn 60%?
- **RQ2**: Các mô hình học tổ hợp (Ensemble Learning (EL)) có mang lại lợi thế đáng kể so với mô hình đơn tốt nhất hay không, khi khác biệt được kiểm chứng bằng kiểm định thống kê thay vì so sánh một lần đo?
- **RQ3**: Một pipeline IDS dựa trên Spark có khả thi về thông lượng, độ trễ và năng lượng khi triển khai phân tán trên cụm thiết bị biên tài nguyên hạn chế (Jetson Orin Nano Super) hay không?

## 1.4 Phương pháp nghiên cứu

- **Thu thập, phân tích, xử lý dữ liệu**: Sử dụng bộ dữ liệu CICIDS2017, từ đó tiến hành phân tích, xử lý và trực quan hóa dữ liệu.
- **Nghiên cứu về phương pháp tối ưu hóa siêu tham số, các phương pháp giảm chiều dữ liệu**: Nghiên cứu các phương pháp giảm chiều dữ liệu như PCA, SHAP và Random Forest Importance nhằm giảm nhiễu và tăng độ chính xác, đồng thời tối ưu hóa siêu tham số thông qua tìm kiếm lưới (Grid Search) và kiểm định chéo (cross-validation) giúp nâng cao hiệu năng mô hình. Các bước này được triển khai trên nền tảng Apache Spark nhằm bảo đảm khả năng xử lý dữ liệu lớn hiệu quả và mở rộng.
- **Nghiên cứu và đề xuất mô hình học máy, học sâu (MLP và một autoencoder làm cổng lọc bất thường), học tổ hợp phát hiện xâm nhập sử dụng Spark Machine Learning**: Sử dụng nhiều thuật toán khác nhau như Decision Tree, Support Vector Machine, Logistic Regression, NB, Random Forest, GBT, XGB, MLP và các thuật toán học tổ hợp (ensemble) để xây dựng mô hình nhằm so sánh, đánh giá mô hình có hiệu năng tốt nhất.
- **Thực nghiệm và đánh giá hiệu năng mô hình trên cụm Jetson Orin Nano Super**: Xây dựng một hệ thống phát hiện xâm nhập mạng IDS *phân tán* triển khai trên cụm hai thiết bị biên Jetson Orin Nano Super (cổng lọc bất thường trên một nút, bộ phân loại PySpark trên nút còn lại) nhằm đánh giá khả năng hoạt động trong môi trường tài nguyên hạn chế. Hệ thống được thiết kế theo kiến trúc xử lý dữ liệu thời gian thực, tận dụng Apache Kafka cho cơ chế truyền tải thông điệp, Spark Streaming cho xử lý luồng dữ liệu, PostgreSQL để quản lý dữ liệu, InfluxDB để lưu trữ chỉ số hiệu năng theo thời gian, Grafana để trực quan hóa và Mailtrap phục vụ kiểm thử cơ chế cảnh báo qua email.

## 1.5 Các đóng góp chính của đề tài

- **Đề xuất giải pháp ứng dụng Apache Spark trong phát hiện xâm nhập cho hệ thống mạng IoT**: Đề tài đã xây dựng một mô hình phát hiện xâm nhập dựa trên nền tảng xử lý dữ liệu lớn Apache Spark, tận dụng khả năng xử lý phân tán và thời gian thực của Spark, hướng tới đáp ứng yêu cầu về hiệu năng và tính mở rộng trong môi trường IoT có khối lượng dữ liệu lớn và biến đổi liên tục. Khả năng xử lý phân tán

được kiểm chứng ở mức minh chứng khái niệm (proof-of-concept) trên cụm thiết bị biên.

- **Xây dựng quy trình xử lý và phát hiện xâm nhập khép kín trên dữ liệu IoT**: Từ bước tiền xử lý dữ liệu, trích xuất đặc trưng, huấn luyện mô hình đến đánh giá kết quả, đề tài đã thiết kế và hiện thực một quy trình phát hiện xâm nhập hoàn chỉnh có thể áp dụng cho các hệ thống thực tế.

- **Ứng dụng và đánh giá các thuật toán học máy trong môi trường phân tán**: Đề tài triển khai và so sánh hiệu quả của một số thuật toán học máy và mô hình học sâu MLP trên nền tảng Apache Spark MLlib (ngoài ra một autoencoder được dùng làm cổng lọc bất thường ở tầng biên), qua đó làm rõ ưu điểm và hạn chế của từng mô hình trong việc xử lý dữ liệu IoT.

- **Đánh giá hiệu năng hệ thống trong môi trường xử lý phân tán**: Ngoài độ chính xác của mô hình, đề tài còn phân tích hiệu năng xử lý (thời gian thực thi, khả năng mở rộng) của hệ thống nhằm khảo sát tính khả thi của giải pháp. Việc kiểm chứng được thực hiện ở mức minh chứng khái niệm (proof-of-concept) trên cụm thiết bị biên với bộ dữ liệu CICIDS2017, làm cơ sở cho việc mở rộng quy mô về sau.

- **Quy trình đánh giá loại trừ rò rỉ dữ liệu và có kiểm định thống kê**: Đề tài nhận diện và loại bỏ các nguồn rò rỉ dữ liệu thường gặp trên CICIDS2017 (đặc trưng `destination_port` rò rỉ nhãn, trùng lặp luồng ở mức đặc trưng), đồng thời áp dụng kiểm định ý nghĩa thống kê, giúp kết quả phản ánh đúng hiệu năng mô hình thay vì các con số bị thổi phồng.

- **Đánh giá tổng quát hóa giữa hai bộ dữ liệu**: Đề tài huấn luyện trên CICIDS2017 rồi kiểm tra trên CSE-CIC-IDS2018 (và ngược lại), cho thấy hiệu năng gần hoàn hảo trên cùng bộ dữ liệu không chuyển sang bộ khác, và đo lượng nhãn của mạng mới cần có để khôi phục hiệu năng.

## 1.6 Công trình khoa học đã công bố

Kết quả của luận văn được trình bày trong hai bài báo khoa học (viết chung với ThS. Nguyễn Vũ Thái, TS. Đỗ Trí Nhựt và TS. Nguyễn Văn Dũ). Bài thứ nhất, về phần đánh giá ngoại tuyến, đã được trình bày tại Hội nghị FAIR 2026. Bài thứ hai, về phần triển khai trên cụm thiết bị biên, đã được chấp nhận đăng tại SOICT 2026:

1. Thai Nguyen Vu, **Bui Van Dung**, Tri Nhut Do, Van Du Nguyen, "A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline," National Conference on Fundamental and Applied Information Technology Research (FAIR), 2026.

2. Thai Nguyen Vu, **Bui Van Dung**, Tri Nhut Do, Van Du Nguyen, "A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark," International Symposium on Information and Communication Technology (SOICT 2026), Communications in Computer and Information Science (CCIS), Springer (**chỉ mục Scopus**). Đã được chấp nhận đăng và trình bày báo cáo (oral), TP. Hồ Chí Minh, 04–05/12/2026.

## 1.7 Cấu trúc của luận văn

Luận văn với đề tài "PHÁT HIỆN XÂM NHẬP (IDS) DỰA TRÊN APACHE SPARK CHO BẢO MẬT MẠNG IoT" được trình bày bao gồm 5 chương. Nội dung tóm tắt từng chương được trình bày như sau:

- **Chương 1: Mở đầu**: Trình bày tổng quan về bối cảnh và lý do chọn đề tài, mục tiêu nghiên cứu, phạm vi nghiên cứu, câu hỏi nghiên cứu và phương pháp tiếp cận. Ngoài ra, chương này nêu ý nghĩa khoa học, thực tiễn và bố cục của luận văn.

- **Chương 2: Cơ sở lý thuyết**: Trình bày các khái niệm cơ bản và nền tảng lý thuyết phục vụ cho đề tài, bao gồm tổng quan về mạng IoT, các lỗ hổng và mối đe dọa bảo mật, phương pháp phát hiện xâm nhập (IDS), và kiến thức liên quan đến Apache Spark. Chương này cũng phân tích các thuật toán học máy thường dùng cho phát hiện xâm nhập.

- **Chương 3: Phương pháp thực hiện**: Trình bày chi tiết các bước xây dựng hệ thống phát hiện xâm nhập, bao gồm thu thập và tiền xử lý dữ liệu, thiết kế kiến trúc hệ thống sử dụng Apache Spark, lựa chọn và triển khai các thuật toán học máy, cùng quy trình huấn luyện và đánh giá mô hình.

- **Chương 4: Thực nghiệm, đánh giá và thảo luận**: Trình bày kết quả thực nghiệm trên CICIDS2017 và CSE-CIC-IDS2018 cùng kết quả triển khai trên cụm thiết bị biên, đánh giá hiệu năng của các mô hình đã triển khai theo các tiêu chí như độ chính xác, Recall, F1-score, kích thước của mô hình và thời gian xử lý. Đồng thời, chương

này phân tích, so sánh và thảo luận kết quả nhằm làm rõ hiệu quả của phương pháp đề xuất.

- **Chương 5: Kết luận và hướng phát triển**: Tổng kết những kết quả chính mà đề tài đạt được, trình bày những đóng góp chính, đồng thời chỉ ra những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai như cải thiện phát hiện các lớp tấn công hiếm, kiểm chứng trên bộ dữ liệu IoT chuyên biệt và tối ưu suy luận biên bằng ONNX.

# CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này trình bày các nền tảng lý thuyết liên quan đến đề tài, bao gồm tổng quan về Internet vạn vật (IoT), các mối đe dọa an ninh trong hệ thống IoT, và vai trò của hệ thống phát hiện xâm nhập (IDS). Bên cạnh đó, trong chương này, luận văn cũng giới thiệu kiến trúc và khả năng của Apache Spark trong xử lý dữ liệu lớn, các kỹ thuật học máy ứng dụng trong phát hiện xâm nhập và cách Spark MLlib hỗ trợ xây dựng mô hình học máy phân tán. (Các nghiên cứu liên quan và khoảng trống nghiên cứu đã được khảo sát ở Chương 1.)

## 2.1 Tổng quan về IoT

Internet of Things (IoT) là mạng lưới các thiết bị vật lý được tích hợp cảm biến, phần mềm và khả năng kết nối mạng, cho phép thu thập và trao đổi dữ liệu qua Internet [27]. IoT bao gồm các thiết bị như cảm biến môi trường, thiết bị gia dụng thông minh và phương tiện giao thông tạo ra khối lượng dữ liệu lớn với đặc điểm đa dạng, tốc độ cao và yêu cầu xử lý thời gian thực. Theo báo cáo Cisco Annual Internet Report (2018–2023), tổng số thiết bị nối mạng toàn cầu được dự báo đạt khoảng 29,3 tỷ vào năm 2023, trong đó khoảng một nửa là kết nối máy-với-máy (M2M/IoT) [28]. Kiến trúc IoT thường được chia thành ba tầng:

- **Tầng cảm biến (Perception Layer)**: Thu thập dữ liệu từ cảm biến và thiết bị.
- **Tầng mạng (Network Layer)**: Truyền tải dữ liệu qua các giao thức như MQTT, CoAP hoặc HTTP.
- **Tầng ứng dụng (Application Layer)**: Phân tích dữ liệu để cung cấp dịch vụ [29].

Tuy nhiên, các thiết bị IoT thường có tài nguyên hạn chế (bộ nhớ, năng lượng, khả năng tính toán), dẫn đến thách thức trong việc triển khai các giải pháp bảo mật truyền thống. Các mối đe dọa như tấn công từ chối dịch vụ (DDoS), tấn công giả mạo và khai thác lỗ hổng giao thức đòi hỏi các giải pháp bảo mật chuyên biệt như IDS [30].

## 2.2 Hệ thống phát hiện xâm nhập (IDS)

Hệ thống phát hiện xâm nhập (IDS) giám sát và phân tích hoạt động mạng hoặc hệ thống để phát hiện các hành vi bất thường hoặc đáng ngờ, nhằm bảo vệ hệ thống khỏi các cuộc tấn công mạng [31]. Mục tiêu là duy trì tính bảo mật, toàn vẹn và sẵn sàng của hệ thống IoT. IDS được phân loại như sau:

**Dựa trên vị trí triển khai:**

- **Network-based IDS (NIDS)**: Giám sát lưu lượng mạng để phát hiện các mẫu tấn công.
- **Host-based IDS (HIDS)**: Tập trung vào hoạt động của một thiết bị cụ thể.

**Dựa trên phương pháp phát hiện:**

- **Signature-based IDS**: Sử dụng các mẫu tấn công đã biết, hiệu quả với các mối đe dọa quen thuộc nhưng hạn chế với các tấn công mới.
- **Anomaly-based IDS**: Phát hiện hành vi bất thường dựa trên mô hình hoạt động bình thường, phù hợp với các cuộc tấn công chưa biết [32, 33]. Các bộ phát hiện dựa trên học sâu như autoencoder [34] hay mạng niềm tin sâu (Deep Belief Network) [35] là một hướng phổ biến của nhóm này.
- **Hybrid IDS**: Kết hợp cả hai phương pháp để tăng độ chính xác [31].

Trong IoT, IDS phải xử lý khối lượng dữ liệu lớn và đáp ứng yêu cầu thời gian thực, đồng thời phù hợp với tài nguyên hạn chế của thiết bị. Luận văn tiếp cận các thách thức này bằng cách kết hợp học máy với một nền tảng dữ liệu lớn là Apache Spark.

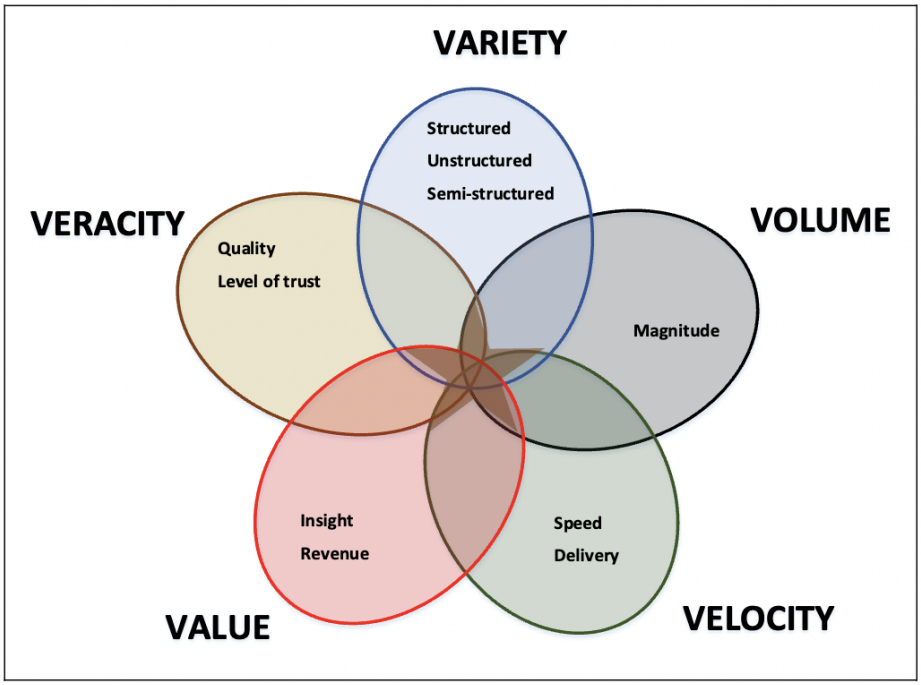
## 2.3 Apache Spark và xử lý dữ liệu lớn

### 2.3.1 Dữ liệu lớn

Dữ liệu lớn (*Big Data*) là một thuật ngữ được sử dụng để mô tả tập hợp dữ liệu có quy mô lớn, phức tạp và đa dạng đến mức không thể xử lý hiệu quả bằng các công cụ và phương pháp truyền thống, bao gồm dữ liệu có cấu trúc, dữ liệu không có cấu trúc và dữ liệu bán cấu trúc.

Ngày nay, kích thước của các tập dữ liệu có thể được coi là dữ liệu lớn dao động từ **terabyte**, **petabyte** đến **exabyte**. Khái niệm về dữ liệu lớn sẽ phụ thuộc vào ngành công nghiệp, cách dữ liệu được sử dụng, lượng dữ liệu lịch sử liên quan và nhiều đặc điểm khác.

Dữ liệu lớn thường được nhận diện thông qua năm đặc điểm cốt lõi: **Khối lượng** (*Volume*), **Tốc độ** (*Velocity*), **Tính đa dạng** (*Variety*), **Tính xác thực** (*Veracity*) và **Giá trị** (*Value*) (Hình 2.1). Năm đặc điểm này là cơ sở để phân biệt dữ liệu lớn với các loại dữ liệu thông thường.




Hình 2.1. Mô hình 5V của dữ liệu lớn (Nguồn: [36])

- **Khối lượng (Volume)**: Đề cập đến quy mô dữ liệu cực lớn.
- **Tốc độ (Velocity)**: Tốc độ dữ liệu được tạo ra, truyền tải và xử lý.
- **Tính đa dạng (Variety)**: Nguồn và định dạng dữ liệu đa dạng như văn bản, hình ảnh, video, v.v.
- **Tính xác thực (Veracity)**: Mức độ đáng tin cậy và chất lượng của dữ liệu.
- **Giá trị (Value)**: Tiềm năng khai thác thông tin hữu ích từ dữ liệu.

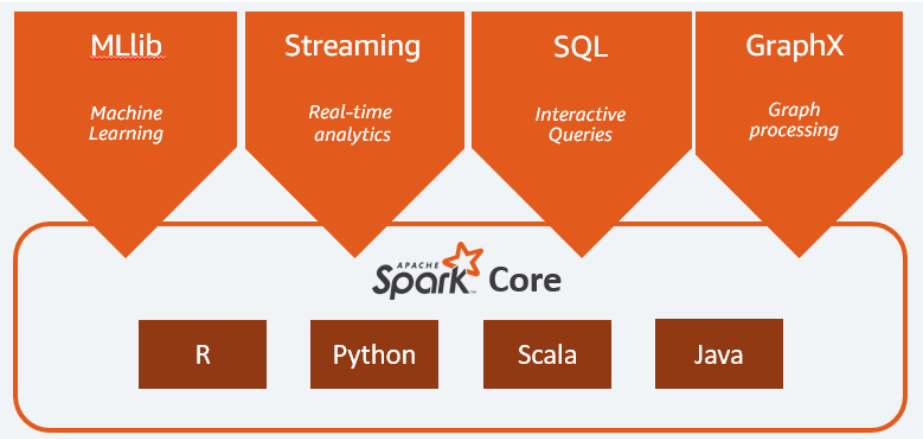
Dữ liệu sinh ra từ các thiết bị IoT được xem là một trong những nguồn tạo ra dữ liệu lớn (BD) quan trọng và phát triển nhanh chóng hiện nay. Với hàng tỷ thiết bị cảm biến, máy móc, thiết bị đeo tay và phương tiện kết nối Internet, lượng dữ liệu được tạo ra mỗi ngày rất lớn, đa dạng về định dạng và tốc độ phát sinh liên tục theo thời gian thực. Dữ liệu IoT mang đầy đủ các đặc điểm của dữ liệu lớn như: khối lượng (volume) lớn, tốc độ (velocity) cao, tính đa dạng (variety) về loại dữ liệu, mức độ xác thực (veracity) không đồng đều và tiềm năng mang lại giá trị (value) lớn nếu được khai thác hiệu quả. Dữ liệu IoT do đó vừa là một phần của dữ liệu lớn, vừa là một trong những nguồn thúc đẩy nhu cầu phát triển các nền tảng xử lý dữ liệu lớn hiện đại.

### 2.3.2 Các thành phần của Apache Spark

Trong những năm gần đây, các nền tảng xử lý dữ liệu lớn như Apache Spark đã thu hút sự quan tâm rộng rãi từ cộng đồng nghiên cứu và công nghiệp. Apache Spark là một hệ thống xử lý dữ liệu phân tán quy mô lớn, có khả năng thực thi nhanh các tác vụ trên các cụm máy tính. So với Hadoop MapReduce, Spark có thể đạt tốc độ xử lý nhanh hơn khoảng 100 lần trong một số trường hợp nhờ vào khả năng xử lý trong bộ nhớ (*in-memory processing*) [37, 38].

Bên cạnh đó, Spark cung cấp giao diện lập trình linh hoạt thông qua các API hỗ trợ nhiều ngôn ngữ phổ biến như Java, Scala, Python và R, giúp việc phát triển và triển khai ứng dụng trở nên thuận tiện hơn.

Apache Spark bao gồm nhiều thành phần chính tạo thành một hệ sinh thái hoàn chỉnh, hỗ trợ đa dạng các tác vụ phân tích dữ liệu như xử lý dữ liệu theo lô (batch), truy vấn SQL, phân tích luồng dữ liệu thời gian thực, học máy và xử lý đồ thị (Hình 2.2).




Hình 2.2. Các thành phần của Apache Spark (Nguồn: [38])

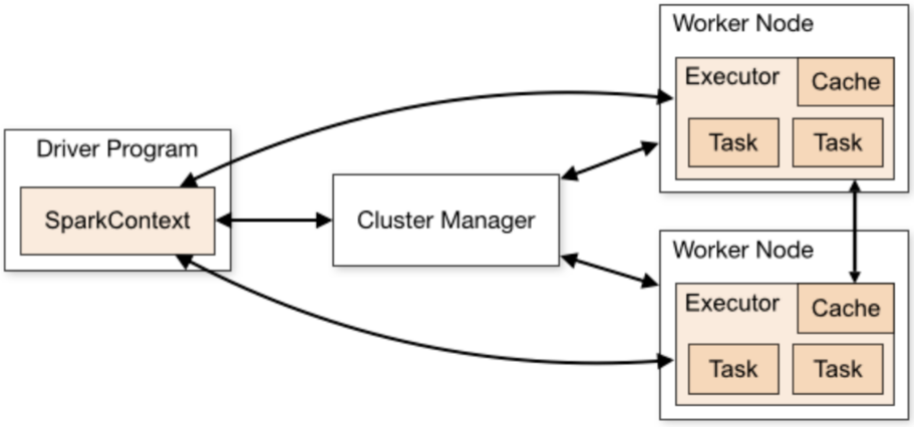
Apache Spark bao gồm nhiều thành phần cốt lõi, mỗi thành phần đảm nhận một chức năng cụ thể trong quá trình xử lý dữ liệu lớn:

- **Spark Core**: Thành phần trung tâm của Spark, chịu trách nhiệm quản lý bộ nhớ, lập lịch, xử lý lỗi, phân phối tác vụ và giao tiếp với các hệ thống lưu trữ. Nó cung cấp các API cấp cao cho Java, Scala, Python và R, giúp đơn giản hóa việc xử lý phân tán.
- **MLlib**: Thư viện học máy tích hợp sẵn trong Spark [39], hỗ trợ huấn luyện và triển khai các mô hình học máy trên quy mô lớn. MLlib cung cấp các thuật toán như phân loại, hồi quy, phân cụm, khai thác mẫu và lọc cộng tác.

- **Spark Streaming**: Công cụ xử lý dữ liệu thời gian thực bằng cách chia luồng dữ liệu thành các lô nhỏ để xử lý tương tự như dữ liệu theo lô. Công cụ này cho phép sử dụng lại mã nguồn và hỗ trợ nhiều nguồn dữ liệu như Kafka, Flume, Twitter, HDFS.
- **Spark SQL**: Thành phần truy vấn dữ liệu phân tán, hỗ trợ ngôn ngữ SQL tiêu chuẩn và HiveQL với hiệu năng cao nhờ tối ưu hóa Catalyst và định dạng cột. Nó hỗ trợ nhiều nguồn dữ liệu như JSON, Hive, Parquet, JDBC, MongoDB, Cassandra.
- **GraphX**: Thư viện xử lý đồ thị phân tán, hỗ trợ biểu diễn, chuyển đổi và phân tích dữ liệu đồ thị. GraphX đi kèm với các thuật toán đồ thị phổ biến như PageRank, Connected Components và nhiều thuật toán phân tích khác.

### 2.3.3 Kiến trúc của Apache Spark

Kiến trúc của Apache Spark được xây dựng theo mô hình master–worker trên nền tảng xử lý phân tán. Thành phần trung tâm là **Spark Driver**, chịu trách nhiệm điều phối quá trình thực thi ứng dụng, bao gồm việc lên lịch, phân phối tác vụ và giám sát tiến trình. Driver xây dựng một đồ thị phi chu trình có hướng (*DAG*, Directed Acyclic Graph) để xác định luồng thực thi công việc. Tổng quan kiến trúc master–worker được minh họa ở Hình 2.3.




Hình 2.3. Sơ đồ kiến trúc hệ thống Apache Spark (Nguồn: [37])

Các tác vụ được phân phối tới các **Worker Nodes**, nơi chứa các **Executor**, tức các tiến trình thực thi thực tế. Mỗi Executor chịu trách nhiệm thực thi các tác vụ (task) và lưu trữ dữ liệu trong bộ nhớ hoặc ổ đĩa nếu cần. Spark sử dụng **Cluster Manager** (như YARN, Mesos hoặc Standalone) để quản lý tài nguyên trên cụm. Dữ liệu được lưu trữ trên các hệ thống như HDFS, S3, hoặc các cơ sở dữ liệu quan hệ.

## 2.4 Học máy trong phát hiện xâm nhập

Học máy (ML) đã trở thành một công cụ thiết yếu trong việc phát hiện các mẫu tấn công và hành vi bất thường trong lưu lượng mạng IoT, nhờ khả năng tự động học và thích ứng từ dữ liệu lịch sử [40]. Các kỹ thuật học máy phổ biến bao gồm:

- **Học có giám sát (Supervised Learning)**: Sử dụng dữ liệu đã được gán nhãn để huấn luyện mô hình phân loại. Một số thuật toán điển hình gồm Random Forest (RF), SVM và mạng nơ-ron nhân tạo (Neural Networks).
- **Học không giám sát (Unsupervised Learning)**: Áp dụng cho các tình huống không có nhãn, tập trung vào việc phát hiện các mẫu bất thường. K-Means là một thuật toán phổ biến trong nhóm này.
- **Học tăng cường (Reinforcement Learning)**: Mô hình học dựa trên phản hồi từ môi trường nhằm tối ưu hóa hành vi hoặc quyết định của hệ thống theo thời gian. Hướng này không thuộc phạm vi thực nghiệm của luận văn và chỉ được nêu cho đủ ba nhóm phương pháp.

Trong bối cảnh mạng IoT, học máy được ứng dụng rộng rãi để phân tích lưu lượng dữ liệu và nhận diện các loại tấn công như DDoS, botnet hay giả mạo gói tin. Chẳng hạn, Koroniotis và cộng sự (2019) xây dựng bộ dữ liệu Bot-IoT và báo cáo các mô hình học máy, học sâu đạt độ chính xác gần tuyệt đối khi phát hiện các tấn công DoS/DDoS trong môi trường IoT [41]. Tuy nhiên, các kết quả gần như tuyệt đối (xấp xỉ 100%) trên các tập dữ liệu công khai cần được xem xét thận trọng, vì chúng thường là dấu hiệu của rò rỉ dữ liệu (data leakage) hơn là hiệu năng thực sự của mô hình. *Rò rỉ dữ liệu* là tình huống trong đó thông tin lẽ ra không sẵn có ở thời điểm triển khai thực tế lại lọt vào quá trình huấn luyện hoặc đánh giá, khiến điểm số bị thổi phồng và không phản ánh đúng khả năng tổng quát hóa của mô hình. Luận văn này nhận diện và xử lý vấn đề đó một cách tường minh ở Chương 4.

Tuy nhiên, việc ứng dụng học máy trong hệ thống IoT vẫn còn gặp nhiều thách thức vì yêu cầu xử lý theo thời gian thực và hạn chế về tài nguyên tính toán. Việc tích hợp học máy với các nền tảng xử lý dữ liệu lớn như Apache Spark cho phép xử lý khối lượng dữ liệu lớn với tốc độ cao và triển khai các thuật toán học máy trong môi trường phân tán.

### 2.4.1 Một số thuật toán học máy

- **Cây quyết định (Decision Trees (DT))** là một trong những thuật toán học máy có giám sát phổ biến, được áp dụng rộng rãi trong các bài toán phân loại. Cấu trúc cơ bản của một cây quyết định bao gồm ba thành phần chính: **nút gốc**, **nút quyết định** và **nút lá**. Cây quyết định có ưu điểm là dễ hiểu, dễ diễn giải, có thể trực quan hóa và không yêu cầu giả định về phân phối dữ liệu. Tuy nhiên, DT cũng dễ bị quá khớp (*overfitting*) nếu không chọn được các tham số hợp lý hoặc xử lý trên dữ liệu nhiễu [42].

- **Rừng ngẫu nhiên (RF)** là một thuật toán học máy được sử dụng phổ biến trong cả hai bài toán phân loại và hồi quy. Đây là một kỹ thuật học tổ hợp (*ensemble learning*) dựa trên việc xây dựng và tổng hợp nhiều cây quyết định độc lập nhằm nâng cao độ chính xác và độ ổn định của mô hình. Thuật toán hoạt động bằng cách tạo ra nhiều cây quyết định trên các tập con khác nhau được chọn ngẫu nhiên từ tập dữ liệu huấn luyện. Mỗi cây được xây dựng dựa trên một mẫu bootstrap (lấy mẫu có hoàn lại), đồng thời tại mỗi nút chia, một tập con ngẫu nhiên của các đặc trưng được chọn để tìm điểm chia tốt nhất, thay vì sử dụng toàn bộ đặc trưng như trong cây quyết định truyền thống [43].

  Kết quả cuối cùng được xác định bằng cách:

  - ◇ Trong bài toán phân loại: lấy kết quả theo nguyên tắc đa số phiếu (*majority voting*) từ các cây.
  - ◇ Trong bài toán hồi quy: lấy giá trị trung bình từ các dự đoán của từng cây.

  Việc kết hợp nhiều cây giúp RF giảm thiểu hiện tượng quá khớp (*overfitting*) vốn là một nhược điểm thường gặp của các cây quyết định đơn lẻ. Khi số lượng cây trong rừng tăng lên, hiệu năng dự đoán của mô hình có xu hướng ổn định và cải thiện hơn.

- **Hồi quy logistic (Logistic Regression (LR))** là một phương pháp thống kê được giới thiệu bởi David Cox vào cuối thập niên 1950. Đây là một kỹ thuật học máy có giám sát thường được sử dụng để giải quyết các bài toán phân loại nhị phân, trong đó biến đầu ra (phản hồi) là một biến định tính, nhận giá trị `0` hoặc `1`. Mô hình hồi quy logistic cho phép ước tính xác suất xảy ra của một sự kiện cụ thể dựa trên một hoặc nhiều biến đầu vào (độc lập). Một trong những điểm mạnh của phương pháp này là

khả năng xác định mức độ ảnh hưởng của từng yếu tố đến xác suất xuất hiện của kết quả, giúp các nhà phân tích hiểu rõ hơn mối quan hệ giữa biến đầu vào và đầu ra [44].

Về mặt toán học, hàm hồi quy logistic được biểu diễn dưới dạng tuyến tính như sau:

$$z_i = \beta_0 + \beta_1 x_{i,1} + \beta_2 x_{i,2} + \cdots + \beta_n x_{i,n} \tag{2.1}$$

Trong đó:

- $x_{i,j}$ là giá trị của biến độc lập thứ $j$ tại quan sát thứ $i$
- $\beta_0$ là hệ số chặn (bias), $\beta_j$ là hệ số hồi quy ứng với biến $x_j$
- $z_i$ là đầu ra tuyến tính trước khi đưa qua hàm kích hoạt (sigmoid)

Sau đó, xác suất xảy ra của sự kiện được tính thông qua hàm logistic (sigmoid):

$$P(Y_i = 1 \mid \mathbf{x}_i) = \frac{1}{1 + e^{-z_i}} \tag{2.2}$$

Trong quá trình huấn luyện, mô hình tìm các hệ số $\beta_j$ tối ưu sao cho hàm mất mát (cross-entropy loss) được tối thiểu hóa.

- **GBT** là một phương pháp *boosting* (kết hợp tuần tự các mô hình học yếu, cần phân biệt với "học tăng cường" – reinforcement learning) sử dụng các cây quyết định như mô hình học yếu (weak learners). Ý tưởng chính của boosting là xây dựng một mô hình mạnh bằng cách kết hợp nhiều mô hình học yếu được huấn luyện theo trình tự tuần tự. Ở mỗi bước, thuật toán cố gắng giảm thiểu lỗi của mô hình trước bằng cách học từ phần dư (residual) hoặc gradient của hàm mất mát [45]. Quá trình huấn luyện GBT gồm các bước:
  - Bắt đầu với một mô hình dự đoán ban đầu (thường là giá trị trung bình với bài toán hồi quy).
  - Tính toán sai số còn lại giữa nhãn thực và dự đoán.
  - Huấn luyện một cây quyết định để dự đoán phần dư.
  - Cập nhật mô hình tổng bằng cách cộng mô hình hiện tại với mô hình mới nhân với một hệ số học $\eta$ (learning rate).

Giả sử $F_0(x)$ là mô hình khởi tạo ban đầu, mô hình tổng tại bước $m$ được cập nhật theo công thức:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x) \tag{2.3}$$

Trong đó:

- $\diamond$ $h_m(x)$ là cây quyết định thứ $m$ học theo gradient của hàm mất mát.
- $\diamond$ $\eta$ là hệ số học (learning rate), thường $\eta \in (0, 1)$.

- **LightGBM (LGBM)** là một framework triển khai thuật toán GBT, được phát triển nhằm tối ưu hóa tốc độ huấn luyện và khả năng mở rộng trên các tập dữ liệu lớn [46], với cùng sơ đồ cập nhật boosting theo gradient đã trình bày ở mục GBT phía trên. Khác với GBT truyền thống, LGBM sử dụng cơ chế phát triển cây theo hướng leaf-wise thay vì level-wise, đồng thời áp dụng thuật toán histogram-based splitting và kỹ thuật Gradient-based One-Side Sampling (GOSS) nhằm giảm chi phí tính toán và tăng tốc độ huấn luyện. Tuy nhiên, LGBM *không* được đưa vào bộ thuật toán so sánh thực nghiệm vì lý do tương thích phần cứng trình bày ở Chương 3. XGBoost và GBT đại diện cho họ gradient boosting.

- **NB** là một phương pháp phân loại xác suất dựa trên định lý Bayes với giả định độc lập có điều kiện giữa các đặc trưng [47]. Mặc dù giả định này thường không hoàn toàn đúng trong thực tế, mô hình vẫn cho hiệu quả tốt trong nhiều bài toán phân loại dữ liệu nhiều chiều.

  Theo định lý Bayes:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)} \tag{2.4}$$

  Với giả định độc lập giữa các đặc trưng:

$$P(x|y) = \prod_{i=1}^{n} P(x_i|y) \tag{2.5}$$

  Mô hình lựa chọn lớp có xác suất hậu nghiệm lớn nhất:

$$\hat{y} = \arg\max_{y} P(y) \prod_{i=1}^{n} P(x_i|y) \tag{2.6}$$

Ưu điểm của NB là đơn giản, tốc độ huấn luyện nhanh và phù hợp với dữ liệu kích thước lớn. Tuy nhiên, hạn chế chính nằm ở giả định độc lập giữa các đặc trưng.

- **Support Vector Machine (SVM)** là một thuật toán học có giám sát được đề xuất bởi Cortes và Vapnik [48], nhằm tìm siêu phẳng tối ưu phân tách dữ liệu với khoảng cách biên (margin) lớn nhất.

  Trong trường hợp dữ liệu tuyến tính, bài toán tối ưu được biểu diễn như sau:

$$\min_{w,b} \frac{1}{2}||w||^2 \tag{2.7}$$

  với điều kiện:

$$y_i(w \cdot x_i + b) \geq 1, \quad \forall i = 1, \ldots, N \tag{2.8}$$

  Trong trường hợp dữ liệu không tuyến tính, SVM sử dụng kỹ thuật *kernel trick* để ánh xạ dữ liệu sang không gian nhiều chiều hơn, cho phép phân tách tuyến tính trong không gian mới. Nhờ cơ chế tối ưu hóa chặt chẽ, SVM thường đạt hiệu năng cao trong các bài toán phân loại với dữ liệu nhiều chiều.

- **Multilayer Perceptron – MLP** là một mô hình học sâu cơ bản gồm nhiều lớp ẩn, có khả năng mô hình hóa các mối quan hệ phi tuyến giữa các đặc trưng [49].

  Mỗi nơ-ron trong mạng thực hiện phép biến đổi tuyến tính:

$$z = w^T x + b \tag{2.9}$$

  sau đó áp dụng hàm kích hoạt phi tuyến:

$$a = \sigma(z) \tag{2.10}$$

  Trong đó $\sigma(\cdot)$ có thể là các hàm kích hoạt như sigmoid, tanh hoặc ReLU. Quá trình huấn luyện sử dụng thuật toán lan truyền ngược (*backpropagation*) nhằm cập nhật trọng số thông qua tối thiểu hóa hàm mất mát bằng gradient descent. MLP có khả năng biểu diễn mạnh nhưng yêu cầu dữ liệu huấn luyện đủ lớn và quá trình điều chỉnh siêu tham số cẩn thận để tránh hiện tượng quá khớp.

- **XGB** là một phiên bản cải tiến của thuật toán GBT truyền thống, được đề xuất bởi Chen và Guestrin vào năm 2016. XGB được thiết kế để tối ưu hóa tốc độ huấn luyện, hiệu năng mô hình, và khả năng xử lý dữ liệu quy mô lớn trong các hệ thống tính toán phân tán [50]. XGB sử dụng các chiến lược sau để cải thiện hiệu năng:

  ◇ Khai thác gradient và Hessian để tối ưu hóa hiệu quả (second-order optimization).

  ◇ Cắt tỉa cây (pruning) theo độ phức tạp thay vì độ sâu cố định.

  ◇ Tính song song trong việc tìm điểm chia tối ưu (split finding).

  ◇ Hỗ trợ xử lý thiếu giá trị (missing values) tự động.

  Mô hình XGB đã được ứng dụng thành công trong nhiều bài toán học máy thực tế như dự báo tài chính, chẩn đoán y tế, phát hiện gian lận và phân tích hành vi khách hàng, nhờ độ chính xác cao và hiệu quả tính toán tốt.

### 2.4.2 Một số thuật toán học tổ hợp

**Học tổ hợp (Ensemble Learning, EL)** là một kỹ thuật trong học máy nhằm kết hợp nhiều mô hình riêng lẻ để nâng cao độ chính xác và độ ổn định trong dự đoán. Thay vì chỉ dựa vào một mô hình đơn lẻ, phương pháp EL kết hợp dự đoán của nhiều mô hình (ví dụ DT, RF, LR) để xây dựng một hệ thống dự đoán chính xác hơn. Nguyên lý của EL là: mỗi mô hình riêng lẻ có thể chỉ đạt độ chính xác thấp hoặc dự đoán sai ở một số trường hợp, nhưng sự kết hợp hợp lý của chúng có thể giảm thiểu sai số, tăng độ chính xác nhờ tận dụng điểm mạnh và khắc phục điểm yếu của từng mô hình thành phần [51].

Kỹ thuật EL được sử dụng rộng rãi trong nhiều lĩnh vực yêu cầu độ chính xác cao như: phát hiện gian lận, phân tích dữ liệu lớn, nhận diện hình ảnh và xử lý ngôn ngữ tự nhiên. Trong lĩnh vực bảo mật mạng IoT, EL cũng được ứng dụng để cải thiện khả năng phát hiện bất thường hoặc xâm nhập dựa trên nhiều mô hình học máy được huấn luyện song song hoặc tuần tự.

Một số phương pháp phổ biến của EL bao gồm:

- **Voting**: Với bài toán phân loại, mỗi mô hình con bỏ phiếu cho một lớp và lớp có số phiếu cao nhất sẽ là đầu ra cuối cùng (đa số phiếu hoặc trung bình xác suất). Các bước của voting được mô tả chi tiết ở Thuật toán 2.1.

- **Boosting**: Huấn luyện các mô hình tuần tự, trong đó mỗi mô hình mới tập trung vào các mẫu mà mô hình trước đã dự đoán sai (ví dụ: GBT, AdaBoost, XGB). Các bước của boosting được mô tả chi tiết ở Thuật toán 2.2.

- **Bagging (Bootstrap Aggregating)**: Huấn luyện nhiều mô hình trên các tập con ngẫu nhiên (lấy mẫu có hoàn lại) của tập dữ liệu gốc, sau đó tổng hợp kết quả dự đoán (ví dụ: Random Forest). Các bước của bagging được mô tả chi tiết ở Thuật toán 2.3.

EL đặc biệt hữu ích trong các hệ thống phát hiện xâm nhập (IDS) vì cần phân tích lượng lớn dữ liệu phức tạp, và việc kết hợp nhiều mô hình có thể giúp tăng khả năng phát hiện các gói tin bất thường mà một mô hình đơn lẻ có thể bỏ sót.

**Thuật toán 2.1.** Voting: Kết hợp dự đoán từ nhiều mô hình phân loại

**Đầu vào:** Tập huấn luyện $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$
Tập mô hình phân loại cơ sở $M_1, M_2, \ldots, M_K$
Kiểu voting: `HARD` hoặc `SOFT`
**Đầu ra:** Nhãn dự đoán cuối cùng $y_{\text{vote}}$ cho điểm dữ liệu $x$

**Khởi tạo:** danh sách $S = \{\}$ ▷ Lưu dự đoán hoặc xác suất
**if** voting = `HARD` **then**
  **for** $j = 1$ to $K$ **do**
    $y_j \leftarrow M_j(x)$ ▷ Dự đoán nhãn của mô hình thứ $j$
    Thêm $y_j$ vào $S$
  **end for**
  $y_{\text{vote}} \leftarrow$ nhãn xuất hiện nhiều nhất trong $S$
**else if** voting = `SOFT` **then**
  Khởi tạo xác suất tổng $P_{\text{sum}}[c] = 0, \forall c \in \{1, \ldots, C\}$
  **for** $j = 1$ to $K$ **do**
    Lấy xác suất $P_j(y = c \mid x)$ từ mô hình $M_j$
    **for** mỗi lớp $c = 1$ to $C$ **do**
      $P_{\text{sum}}[c] \leftarrow P_{\text{sum}}[c] + P_j(y = c \mid x)$
    **end for**
  **end for**
  $y_{\text{vote}} \leftarrow \arg\max_c P_{\text{sum}}[c]$
**end if**
**return** $y_{\text{vote}}$

**Thuật toán 2.2.** Boosting: Kết hợp tuần tự các mô hình học yếu

**Đầu vào:** Tập dữ liệu huấn luyện $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$
Số lượng vòng lặp (mô hình yếu): $T$
**Đầu ra:** Mô hình boosting $F_T(x)$

**Khởi tạo:** $F_0(x) \leftarrow 0$ ▷ Mô hình khởi đầu có thể là 0 hoặc dự đoán trung bình
**for** $t = 1$ to $T$ **do**
    Tính giả-phần-dư (pseudo-residuals) theo gradient âm của hàm mất mát: $r_i^{(t)} = -\left[\dfrac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{t-1}}$ ▷ Với mất mát bình phương (hồi quy), rút gọn thành $y_i - F_{t-1}(x_i)$; với phân loại dùng log-loss
    Huấn luyện mô hình yếu $h_t(x)$ trên các giả-phần-dư $r_i^{(t)}$
    Cập nhật mô hình: $F_t(x) = F_{t-1}(x) + \eta \cdot h_t(x)$
**end for**
**return** $F_T(x)$

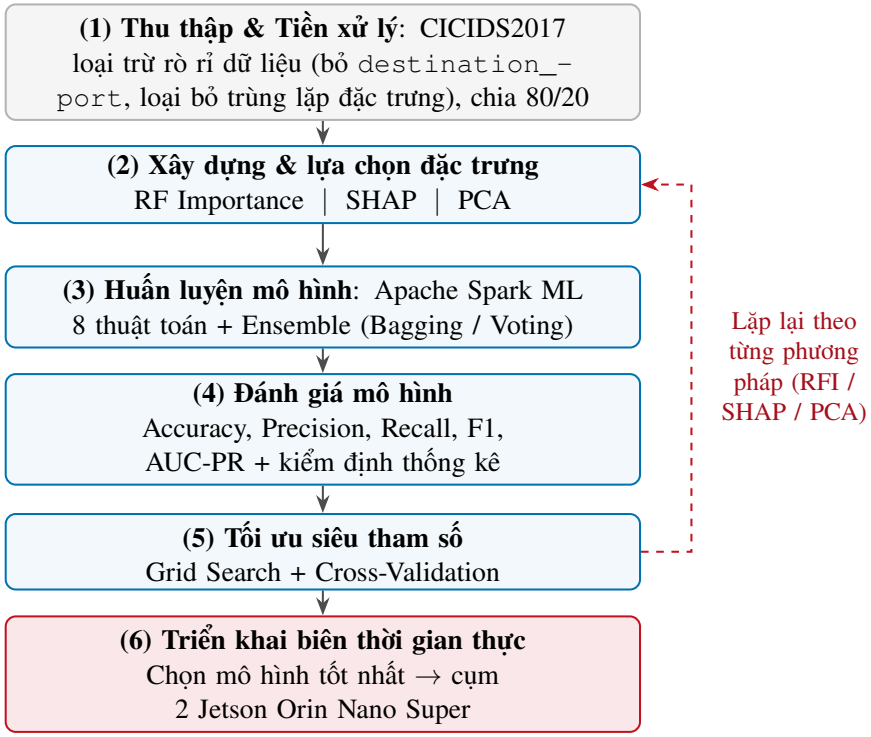
**Thuật toán 2.3.** Bagging: Bootstrap Aggregating

**Đầu vào:** Tập huấn luyện $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$, với $x_i \in \mathbb{R}^d$, $y_i \in \{0, 1\}$ (tổng quát: $C$ lớp)
Số lượng mô hình cơ sở: $K$
**Đầu ra:** Dự đoán cuối cùng $y_{\text{bag}}$ cho một điểm dữ liệu $x$

**Khởi tạo:** Danh sách mô hình $M_1, M_2, \ldots, M_K$
**for** $k = 1$ to $K$ **do**
    Tạo mẫu bootstrap $\mathcal{D}^{(k)}$ từ $\mathcal{D}$ (lấy mẫu ngẫu nhiên có hoàn lại)
    Huấn luyện mô hình $M_k \leftarrow \text{Train}(\mathcal{D}^{(k)})$
**end for**
**Khởi tạo:** Danh sách dự đoán $Y = \{\}$
**for** $k = 1$ to $K$ **do**
    Thực hiện dự đoán $y_k \leftarrow M_k(x)$
    Thêm $y_k$ vào $Y$
**end for**
Tính $y_{\text{bag}} \leftarrow \text{Voting}(Y)$ ▷ Hard voting hoặc trung bình xác suất nếu dùng soft
**return** $y_{\text{bag}}$

# CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN

Trong chương này, luận văn sẽ trình bày phương pháp đề xuất để xây dựng mô hình phát hiện xâm nhập dựa trên Apache Spark cho bảo mật mạng IoT. Quá trình thực hiện bao gồm 6 bước chính, được thể hiện qua Hình 3.1: (1) Thu thập dữ liệu và tiền xử lý; (2) Xây dựng đặc trưng và lựa chọn đặc trưng; (3) Huấn luyện mô hình; (4) Đánh giá mô hình; (5) Tối ưu siêu tham số và (6) Triển khai biên thời gian thực. Các bước (1)–(5) được lặp lại tương ứng với từng phương pháp lựa chọn đặc trưng (Random Forest Importance (RFI), SHAP, PCA), sau đó lựa chọn mô hình tốt nhất để triển khai trên thiết bị biên.




Hình 3.1. Quy trình tổng thể huấn luyện, đánh giá và triển khai mô hình đề xuất

## 3.1 Thu thập dữ liệu và tiền xử lý

Trong nghiên cứu này, tập dữ liệu được sử dụng là CICIDS2017 được phát hành bởi Viện An ninh mạng Canada (CIC). Tập dữ liệu này được xây dựng nhằm mô phỏng lưu lượng mạng thực tế kết hợp với nhiều loại tấn công mạng hiện đại, phục vụ cho việc huấn luyện và đánh giá các mô hình phát hiện xâm nhập. Tập dữ liệu CICIDS2017 tập trung vào các kịch bản tấn công đơn lẻ trong môi trường kiểm soát. Các bộ dữ liệu phát hiện xâm nhập mạng công khai, bao gồm CICIDS2017, có những giới hạn đã được khảo sát một cách hệ thống (lỗi gán nhãn, tạo tác khi sinh lưu lượng, và khả năng tổng quát hóa hạn chế) [52].

Đây là lý do luận văn áp dụng quy trình loại trừ rò rỉ ở bước tiền xử lý và bổ sung đánh giá liên bộ dữ liệu ở Chương 4. Thông tin chi tiết về tập dữ liệu được trình bày trong Bảng 3.1.

Bảng 3.1. Mô tả bộ dữ liệu CICIDS2017

| **Thuộc tính** | **Giá trị** |
|---|---|
| Tổ chức phát hành | Canadian Institute for Cybersecurity (CIC) |
| Thời gian phát hành | 2017 |
| Giao thức mạng | HTTP, HTTPS, FTP, SSH, DNS, ... |
| Số lượng bản ghi (CSV) | Hơn 2,8 triệu bản ghi |
| Thời gian thu thập | 03/07/2017 – 07/07/2017 |
| Loại tấn công mô phỏng | DoS, DDoS, PortScan, Brute-force, Infiltration, Botnet, ... |
| Số loại nhãn | 15 (1 nhãn bình thường + 14 loại tấn công) |
| Định dạng dữ liệu | CSV |
| Mục đích | Phát hiện xâm nhập mạng trong môi trường thực tế giả lập |
| Nguồn dữ liệu | [53] |

Bên cạnh CICIDS2017 (bộ dữ liệu chính cho toàn bộ các thí nghiệm), luận văn sử dụng thêm **CSE-CIC-IDS2018** cho thí nghiệm *tổng quát hóa liên bộ dữ liệu* (Mục 4.2.9): huấn luyện trên một bộ, kiểm tra trên bộ kia và ngược lại. Đây là bộ kế nhiệm của CICIDS2017, do CIC phối hợp với Cơ quan An ninh Truyền thông Canada (CSE) thu thập trên hạ tầng AWS quy mô lớn hơn nhiều. Đặc trưng luồng được trích xuất bằng CICFlowMeter phiên bản mới hơn nên tên cột được ánh xạ về danh mục tên cột chuẩn của CICIDS2017 qua bảng bí danh tường minh trong mã nguồn. Thông tin chi tiết ở Bảng 3.2.

Bảng 3.2. Mô tả bộ dữ liệu CSE-CIC-IDS2018 (dùng cho thí nghiệm liên bộ dữ liệu)

| **Thuộc tính** | **Giá trị** |
|---|---|
| Tổ chức phát hành | CIC phối hợp CSE (Communications Security Establishment) |
| Thời gian phát hành | 2018 |
| Hạ tầng thu thập | Testbed trên AWS (∼450 máy nạn nhân, 50 máy tấn công) |
| Số lượng bản ghi (CSV) | ∼16,2 triệu luồng (10 tệp CSV theo ngày) |
| Thời gian thu thập | 14/02/2018 – 02/03/2018 |
| Loại tấn công mô phỏng | DDoS (HOIC, LOIC-HTTP/UDP), DoS (Hulk, GoldenEye, Slowloris, SlowHTTPTest), Bot, Brute-force (SSH, FTP, Web, XSS), SQL Injection, Infiltration |
| Số loại nhãn | 15 (1 nhãn bình thường + 14 loại tấn công) |
| Phân bố nhị phân (sau làm sạch) | ∼13,35 triệu Benign / ∼2,35 triệu Attack |
| Phần dùng trong luận văn | Mẫu phân tầng theo nhãn 20% (`seed`=42), còn ∼2,39 triệu luồng sau khử trùng lặp mức đặc trưng |
| Định dạng dữ liệu | CSV (CICFlowMeter v3, tên cột ánh xạ về chuẩn CICIDS2017) |
| Nguồn dữ liệu | [54] |

**Tiền xử lý dữ liệu** Việc tiền xử lý dữ liệu nhằm loại bỏ nhiễu, giá trị ngoại lệ và giá trị bị thiếu được thực hiện trước khi đưa dữ liệu vào huấn luyện mô hình. Các kỹ thuật được áp dụng gồm chuẩn hóa tên cột, căn chỉnh lược đồ giữa các tệp, thay thế giá trị vô cùng, loại bỏ bản ghi trùng lặp/thiếu, và gán nhãn nhị phân (BENIGN $= 0$, Attack $= 1$).

Bên cạnh đó, luận văn áp dụng hai bước *loại trừ rò rỉ dữ liệu* ngay trong giai đoạn tiền xử lý, trước khi phân chia tập, để kết quả đánh giá phản ánh đúng hiệu năng mô hình. Rò rỉ dữ liệu thuộc nhóm cạm bẫy "data snooping", một trong hai cạm bẫy phổ biến nhất của học máy trong an ninh (có mặt ít nhất một phần ở 73% số bài được khảo sát) [55].

**Bước một: loại đặc trưng rò rỉ nhãn.** Cột `destination_port` (và `source_port` nếu có) bị loại. Trong CICIDS2017, mỗi loại tấn công được sinh nhằm vào những cổng dịch vụ cố định, nên cổng đích gần như mã hóa trực tiếp nhãn. Mô hình khi đó "ghi nhớ" ánh xạ cổng–nhãn thay vì học hành vi luồng. Cột `protocol` cũng bị loại với lý do tương tự: đây là biến phân loại có ít mức giá trị, tương quan mạnh với cổng dịch vụ, nên mang cùng nguy cơ trở thành "lối tắt" định danh dịch vụ thay vì hành vi.

**Bước hai: loại bỏ trùng lặp ở mức đặc trưng.** Các luồng được gom nhóm theo toàn bộ vector đặc trưng. Nhóm có nhãn nhất quán được giữ lại đúng một đại diện. Nhóm trùng đặc trưng nhưng mâu thuẫn nhãn (cùng vector đặc trưng, nhãn vừa Benign vừa Attack) bị loại toàn bộ, vì không thể xác định nhãn đúng. Hiện tượng này phù hợp với các lỗi gán nhãn đã được ghi nhận trên CICIDS2017, trong đó một số luồng tấn công bị gán nhãn sai [56].

Có 270 nhóm như vậy, gồm 44.260 hàng. Cách xử lý này tất định: nếu giữ lại một hàng tùy ý thì kết quả phụ thuộc cách phân mảnh dữ liệu và không tái lập được giữa các lần chạy. Bước này tránh việc cùng một mẫu xuất hiện ở cả tập huấn luyện lẫn tập kiểm tra khi phân chia ngẫu nhiên.

Bước loại bỏ trùng lặp làm thay đổi phân bố lớp: các luồng benign trùng lặp nhiều hơn nên bị loại nhiều hơn, khiến tỷ lệ benign/attack sau xử lý khác tỷ lệ ban đầu. Vì vậy phân bố lớp trước và sau khi loại trùng lặp được báo cáo tường minh. Toàn bộ quá trình tiền xử lý CICIDS2017 được trình bày ở Thuật toán 3.1.

**Phân chia tập dữ liệu**

Tập dữ liệu CICIDS2017 được chia ngẫu nhiên (random split) theo tỷ lệ 80/20 cho huấn luyện và kiểm tra, với seed cố định (=42) để bảo đảm tái lập. Các chỉ số hiệu năng được **công bố trên tập kiểm tra**: tập này được giữ lại đến cuối, không tham gia huấn luyện, xếp hạng đặc trưng hay tinh chỉnh siêu tham số. Kết quả trên tập huấn luyện chỉ dùng để chẩn đoán hiện tượng quá khớp (overfitting) hoặc thiếu khớp (underfitting).

Ba biện pháp bảo đảm thông tin của tập kiểm tra không lọt vào quá trình huấn luyện:

- **Biến đổi đặc trưng**: `StandardScaler`, PCA và việc xếp hạng đặc trưng (RF Importance, SHAP) đều được fit chỉ trên tập huấn luyện $D_{\text{train}}$ rồi transform lên tập kiểm tra.

- **Chọn mô hình**: việc chọn Top-3 mô hình thành phần cho các ensemble, chọn mô hình đại diện mỗi phương pháp và chọn cặp phương pháp đưa vào kiểm định thống kê (Chương 4) đều thực hiện trên một tập kiểm định (validation) tách từ tập huấn luyện theo tỷ lệ 80/20 (`seed`=42). Các mô hình cơ sở được huấn luyện trên phần còn lại.

- **Chọn ngưỡng**: ngưỡng của cổng lọc bất thường (Chương 4) chỉ được chọn trên 10% lưu lượng benign tách từ tập huấn luyện (`seed`=42).

Toàn bộ quy trình phân chia được minh họa ở Hình 3.2.

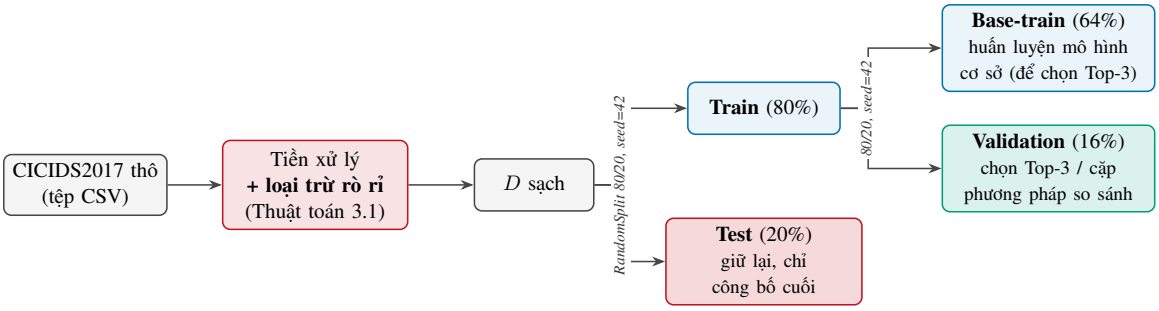
## 3.2 Lựa chọn đặc trưng và giảm chiều dữ liệu

Trong quá trình tiền xử lý dữ liệu cho bài toán phát hiện xâm nhập mạng, lựa chọn đặc trưng bao gồm **chọn đặc trưng** và **trích xuất đặc trưng** là một bước quan trọng nhằm giảm chiều dữ liệu, loại bỏ những đặc trưng không cần thiết, và cải thiện hiệu năng của mô hình học máy [57]. Dữ liệu số được gom vào vector đặc trưng bằng `VectorAssembler`

**Thuật toán 3.1.** Tiền xử lý dữ liệu CICIDS2017

**Đầu vào:** Tập $N$ tệp CSV từ bộ dữ liệu CICIDS2017: $\{F_1, F_2, \ldots, F_N\}$
Tỷ lệ phân chia huấn luyện/kiểm tra $\alpha = 0{,}8$
**Đầu ra:** Tập huấn luyện $D_{\text{train}}$ và tập kiểm tra $D_{\text{test}}$ ở định dạng Parquet

**Bước 1: Gộp dữ liệu từ nhiều tệp CSV**
$D \leftarrow \emptyset$
**for** $i = 1$ to $N$ **do**
  Đọc tệp $F_i$ thành DataFrame $df_i$ với lược đồ suy luận tự động
  CLEANCOLUMNNAMES($df_i$) ▷ Chuẩn hóa tên cột: xóa khoảng trắng, chuyển chữ thường
  HANDLEINFINITYVALUES($df_i$) ▷ Thay thế $\pm\infty$ bằng giá trị NaN
  **if** $D = \emptyset$ **then**
    $\mathcal{S} \leftarrow \text{Schema}(df_i)$ ▷ Lưu lược đồ chuẩn từ tệp đầu tiên
    $D \leftarrow df_i$
  **else**
    ALIGNSCHEMA($df_i, \mathcal{S}$) ▷ Căn chỉnh lược đồ về chuẩn $\mathcal{S}$
    $D \leftarrow D \cup df_i$ ▷ Gộp theo tên cột (unionByName)
  **end if**
**end for**
**Bước 2: Làm sạch dữ liệu**
$D \leftarrow \text{RemoveDuplicates}(D)$ ▷ Loại bỏ các bản ghi trùng lặp hoàn toàn (giống hệt trên mọi cột)
$D \leftarrow \text{RemoveNulls}(D)$ ▷ Loại bỏ các bản ghi chứa giá trị null/NaN
**Bước 3: Tạo nhãn nhị phân**
$\forall (x_i, \text{label}_i) \in D: \; y_i \leftarrow \begin{cases} 0 & \text{nếu label}_i = \text{“BENIGN”} \\ 1 & \text{ngược lại (Attack)} \end{cases}$
**→ Hai bước loại trừ rò rỉ dữ liệu (data leakage): Bước 4 và Bước 4b**
**Bước 4 [loại rò rỉ – I]: Xác định các đặc trưng số (đã loại trừ rò rỉ)**
$\mathcal{C}_{\text{exclude}} \leftarrow \{$label, label_binary, source_ip, destination_ip, flow_id, timestamp, protocol, **destination_port, source_port**$\}$ ▷ Loại bỏ đặc trưng gây rò rỉ nhãn
$\mathcal{F} \leftarrow \{c \in \text{columns}(D) \mid c \notin \mathcal{C}_{\text{exclude}} \wedge \text{dtype}(c) \in \{\text{double, float, int, bigint}\}\}$
**Bước 4b [loại rò rỉ – II]: Loại bỏ trùng lặp ở mức đặc trưng (tất định)**
$G \leftarrow \text{GroupBy}(D, \mathcal{F})$ ▷ Gom nhóm theo toàn bộ vector đặc trưng
**for** mỗi nhóm $g \in G$ **do**
  **if** $|\{y_i : (x_i, y_i) \in g\}| > 1$ **then** ▷ Nhóm mâu thuẫn nhãn (Benign lẫn Attack)
    Loại toàn bộ nhóm $g$ khỏi $D$; ghi log số nhóm/hàng xung đột
  **else**
    Giữ lại đúng một đại diện của $g$ (thứ tự tất định theo nhãn)
  **end if**
**end for**
**Bước 5: Phân chia và lưu trữ**
$D_{\text{train}}, D_{\text{test}} \leftarrow \text{RandomSplit}(D, [\alpha, 1-\alpha], \text{seed} = 42)$
Lưu $D_{\text{train}}, D_{\text{test}}$ dưới định dạng Parquet
**return** $D_{\text{train}}, D_{\text{test}}$




Hình 3.2. Sơ đồ phân chia dữ liệu CICIDS2017 (train/test 80/20, `seed`=42, trong đó train tách tiếp 80/20 thành base-train/validation)

(khoảng 78 đặc trưng số trước khi loại cổng, chính xác là 77 đặc trưng sau khi loại `destination_port`/`source_port`) và được chuẩn hóa bằng `StandardScaler` (chuẩn hóa Z-score). Ba phương pháp được sử dụng trong luận văn là *RF*, SHAP và PCA.

a) **Principal Component Analysis (PCA)**: Là phương pháp giảm chiều dữ liệu không giám sát, nhằm biến đổi tập đặc trưng ban đầu thành một tập các thành phần chính (principal components) mới không tương quan tuyến tính với nhau và giữ lại phần lớn phương sai của dữ liệu. PCA thực hiện bằng cách tính ma trận hiệp phương sai và tìm các vector riêng (eigenvectors) tương ứng với các giá trị riêng (eigenvalues). Các thành phần chính được xác định bằng cách giải bài toán:

$$\mathbf{C}\mathbf{v} = \lambda\mathbf{v} \tag{3.1}$$

Trong đó:

- $\mathbf{C}$ là ma trận hiệp phương sai của dữ liệu.
- $\mathbf{v}$ là vector riêng (principal component).
- $\lambda$ là giá trị riêng, biểu thị lượng phương sai được giải thích.

Các thành phần chính có giá trị riêng lớn nhất sẽ được lựa chọn để tạo thành không gian đặc trưng mới với số chiều thấp hơn nhưng vẫn giữ được phần lớn thông tin của dữ liệu. Đây là phương pháp được sử dụng phổ biến trong phân tích dữ liệu đa biến và học máy để giảm chiều, loại bỏ nhiễu và trực quan hóa dữ liệu [58].

b) **SHapley Additive exPlanations (SHAP)**: Là phương pháp giải thích mô hình dựa trên lý thuyết trò chơi, dùng để đo lường mức độ đóng góp của từng đặc trưng vào dự đoán của mô hình. SHAP tính giá trị Shapley cho mỗi đặc trưng bằng cách xét trung bình đóng góp biên của đặc trưng đó trên tất cả các tập con có thể:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)] \tag{3.2}$$

Trong đó:

- $F$ là tập tất cả các đặc trưng.
- $S$ là tập con của các đặc trưng không chứa $i$.
- $f(S)$ là giá trị dự đoán của mô hình khi chỉ sử dụng tập đặc trưng $S$.
- $\phi_i$ là mức độ đóng góp của đặc trưng $i$ vào kết quả dự đoán.

Các đặc trưng có giá trị SHAP lớn (theo độ lớn tuyệt đối) được xem là có ảnh hưởng quan trọng hơn đến quyết định của mô hình. Phương pháp này được ứng dụng rộng rãi trong giải thích mô hình phức tạp như XGBoost, Random Forest, và mạng nơ-ron [59].

c) **Random Forest Feature Importance**: Thuật toán RF có khả năng đánh giá tầm quan trọng của từng đặc trưng dựa trên mức độ đóng góp vào việc giảm độ bất thuần (impurity) tại các nút phân chia của cây quyết định. Độ quan trọng $I(f_j)$ của đặc trưng $f_j$ được tính như sau:

$$I(f_j) = \frac{1}{T} \sum_{t=1}^{T} \sum_{n \in N_t(f_j)} \Delta i(n) \tag{3.3}$$

Trong đó:

- $T$ là số lượng cây trong rừng.
- $N_t(f_j)$ là tập các nút trong cây $t$ sử dụng đặc trưng $f_j$ để phân chia.
- $\Delta i(n)$ là lượng giảm độ bất thuần (ví dụ: Gini hoặc Entropy) tại nút $n$.

Sau khi huấn luyện mô hình RF, các đặc trưng được xếp hạng theo độ quan trọng và 30 đặc trưng đứng đầu được giữ lại (Top-30). Với SHAP, 30 đặc trưng có giá trị SHAP trung bình tuyệt đối lớn nhất được giữ lại theo cùng cách.

## 3.3 Huấn luyện mô hình

### 3.3.1 Huấn luyện các mô hình cơ sở

Tám thuật toán phân loại có giám sát được huấn luyện độc lập trên tập đặc trưng đã qua bước xây dựng và lựa chọn đặc trưng. Bảng 3.3 liệt kê các thuật toán cùng thông tin chi tiết về biến thể và phương pháp học.

Bảng 3.3. Tám bộ phân loại cơ sở được sử dụng trong pipeline

| **Thuật toán** | **Biến thể / Ghi chú** | **Phương pháp học** |
|---|---|---|
| Decision Tree | CART | Phân chia đệ quy theo Gini / Entropy |
| Logistic Regression | Chuẩn hóa L2 | Tối ưu hàm mất mát logistic |
| Linear SVM | SVC tuyến tính | Tối đa hóa biên phân loại |
| Naive Bayes | Phân phối Gaussian | Ước lượng xác suất hậu nghiệm |
| Random Forest | Bagging | Tổ hợp cây quyết định ngẫu nhiên |
| GBT | Gradient Boosting | Tối thiểu hóa hàm mất mát theo gradient |
| XGBoost | Gradient Boosting tối ưu | Regularised Gradient Boosting |
| MLP | Mạng nơ-ron nhân tạo | Lan truyền ngược (Backpropagation) |

Đáng lưu ý, LightGBM (một thư viện gradient boosting phổ biến) *không* được đưa vào danh sách trên vì thư viện gốc (native, thông qua SynapseML) chỉ hỗ trợ kiến trúc x86_64, không chạy được trên thiết bị biên Jetson Orin Nano Super (ARM64), vốn là mục tiêu triển khai của nghiên cứu. XGBoost và GBT đã đại diện đầy đủ cho họ gradient boosting.

**Xử lý mất cân bằng lớp.** Do tỷ lệ Benign/Attack lệch nhau đáng kể, sáu mô hình MLlib (Decision Tree, Logistic Regression, Linear SVC, Random Forest, GBT, Naive Bayes) được huấn luyện với trọng số lớp (`weightCol`) đặt theo tỷ lệ nghịch của tần suất lớp benign/attack. XGBoost dùng tham số `scale_pos_weight` với vai trò tương đương. Riêng MLP là ngoại lệ không áp trọng số lớp do API `MultilayerPerceptronClassifier` của Spark MLlib không hỗ trợ `weightCol`.

Sau khi huấn luyện, tám mô hình được xếp hạng theo chiều giảm dần của F1-Score trên tập kiểm định (validation, tách từ tập huấn luyện, xem mục Phân chia tập dữ liệu) để xác định top-3 mô hình tốt nhất phục vụ bước kết hợp ensemble tiếp theo. Tập kiểm tra hoàn toàn không được sử dụng ở bước chọn mô hình này nhằm tránh rò rỉ chọn lọc (selection leakage).

### 3.3.2 Các phương pháp Ensemble Learning

Bảng 3.4. Các phương pháp học tổ hợp (ensemble) sử dụng trong luận văn

| **Phương pháp** | **Mô tả chi tiết** |
|---|---|
| Hybrid Bagging | Top-3 mô hình kết hợp theo sơ đồ $(3\text{-}2\text{-}2)$ sub-samples với *Weighted Soft Voting*: dự đoán xác suất có trọng số |
| Soft Voting | Trung bình xác suất Top-3 mô hình theo F1-Score<br>Nhãn *Attack* nếu xác suất trung bình $\geq 0{,}5$ |

#### a) Áp dụng Voting

Sau khi hoàn tất giai đoạn huấn luyện đơn mô hình, luận văn tiến hành xây dựng mô hình ensemble dựa trên kỹ thuật Voting để nâng cao độ chính xác và tính ổn định của hệ thống phân loại. Hai phương pháp tổ hợp được sử dụng được tóm tắt trong Bảng 3.4. Cụ thể, ba mô hình phân loại nhị phân có chỉ số F1-Score cao nhất ở giai đoạn trước sẽ được lựa chọn để tham gia vào quá trình ensemble.

Mỗi mô hình trong tập được chọn sẽ đưa ra dự đoán độc lập cho từng mẫu trong tập kiểm tra. Các dự đoán này sau đó được tổng hợp thông qua cơ chế bầu chọn. Trong trường hợp Soft Voting, xác suất dự đoán lớp *Attack* của các mô hình thành phần được lấy trung bình. Mẫu được gán nhãn *Attack* nếu xác suất trung bình đạt từ $0{,}5$ trở lên, ngược lại được gán nhãn *Benign*.

Việc áp dụng kỹ thuật Voting cho phép khai thác sự đa dạng giữa các mô hình học cơ sở, từ đó giảm sai số do thiên lệch của từng mô hình riêng lẻ và cải thiện khả năng tổng quát hóa trên dữ liệu chưa từng thấy. Quy trình chi tiết được trình bày trong Thuật toán 3.2.

**Thuật toán 3.2.** Soft Voting Ensemble

**Đầu vào:** Tập dữ liệu huấn luyện $D = \{(x_i, y_i)\}_{i=1}^{n}$ với $x_i \in \mathbb{R}^d$, $y_i \in \{0, 1\}$
Tập kiểm định $D_{\text{val}}$ (tách từ $D$, dùng để xếp hạng và chọn mô hình thành phần)
Tập dữ liệu kiểm tra $D_{\text{test}} = \{(x_j)\}_{j=1}^{m}$
Tập $K$ thuật toán học cơ sở khác loại $\{\mathcal{L}_1, \mathcal{L}_2, \ldots, \mathcal{L}_K\}$
**Đầu ra:** Nhãn dự đoán cuối cùng $y_{\text{vote}} \in \{0, 1\}$ cho mỗi mẫu

**Bước 1: Huấn luyện và đánh giá $K$ mô hình cơ sở**
**for** $k = 1$ to $K$ **do**
    $M_k \leftarrow \text{Train}(\mathcal{L}_k, D \setminus D_{\text{val}})$ ▷ Huấn luyện trên phần train lõi, tách khỏi $D_{\text{val}}$
    $F_k \leftarrow \text{F1-Score}(M_k, D_{\text{val}})$ ▷ Tính điểm F1 trên tập kiểm định (validation), không chạm tập kiểm tra
**end for**
**Bước 2: Chọn Top-3 mô hình có F1 cao nhất**
Sắp xếp $\{M_1, \ldots, M_K\}$ theo $F_k$ giảm dần
Chọn 3 mô hình tốt nhất: $\mathcal{S} = \{M^{(1)}, M^{(2)}, M^{(3)}\}$ với $F^{(1)} \geq F^{(2)} \geq F^{(3)}$
**Bước 3: Thu thập xác suất dự đoán từ 3 mô hình được chọn**
**for** mỗi mẫu $x_j \in D_{\text{test}}$ **do**
    **for** mỗi $M^{(s)} \in \mathcal{S}$, $s = 1, 2, 3$ **do**
        $p_s^{(j)} \leftarrow P^{(s)}(y = 1 \mid x_j)$ ▷ Xác suất lớp Attack từ mô hình $M^{(s)}$
    **end for**
**end for**
**Bước 4: Kết hợp bằng bầu chọn mềm (Soft Voting)**
**for** mỗi mẫu $x_j \in D_{\text{test}}$ **do**
    $\bar{p}_j \leftarrow \frac{1}{3} \sum_{s=1}^{3} p_s^{(j)}$ ▷ Trung bình xác suất lớp Attack
    **if** $\bar{p}_j \geq 0{,}5$ **then**
        $y_{\text{vote}}^{(j)} \leftarrow 1$ ▷ Dự đoán: Attack
    **else**
        $y_{\text{vote}}^{(j)} \leftarrow 0$ ▷ Dự đoán: Benign
    **end if**
**end for**
**return** $\{y_{\text{vote}}^{(j)}\}_{j=1}^{m}$

**b) Áp dụng Bagging**

Để áp dụng kỹ thuật học tổ hợp Bagging, luận văn đề xuất xây dựng một mô hình ensemble lai (Hybrid Bagging Ensemble) dựa trên các mô hình cơ sở có hiệu năng cao nhất (đã huấn luyện và xếp hạng ở bước trước). Cụ thể, thay vì chỉ sử dụng một mô hình đơn lẻ, quy trình bắt đầu bằng việc huấn luyện và đánh giá $K$ thuật toán học cơ sở trên tập dữ liệu huấn luyện $D$. Hiệu năng của từng mô hình được đo lường thông qua chỉ số F1-Score, từ đó lựa chọn ba mô hình có kết quả tốt nhất để đưa vào giai đoạn kết hợp.

Sau khi xác định Top-3 mô hình, phương pháp Bagging được áp dụng theo phân bổ số bản sao $(3, 2, 2)$. Đối với mỗi mô hình trong ba mô hình được chọn, nhiều tập dữ liệu bootstrap được tạo ra bằng cách lấy mẫu ngẫu nhiên có hoàn lại từ tập huấn luyện ban đầu. Trên mỗi tập bootstrap này, một bản sao của mô hình tương ứng được huấn luyện độc lập, tạo thành các *bộ học cơ sở (base learners)* trong ensemble. Mỗi base learner được gán trọng số tỷ lệ với F1-Score của mô hình gốc nhằm phản ánh mức độ tin cậy tương đối của nó trong quá trình tổng hợp. Việc phân bổ bất đối xứng $(3, 2, 2)$, tức cấp nhiều bản sao bootstrap hơn cho mô hình hạng cao, là một lựa chọn thiết kế thực nghiệm nhằm thiên ensemble về phía các bộ học mạnh hơn trong khi vẫn giữ tính đa dạng từ ba thuật toán khác loại. Đây là một heuristic, không phải kết quả của tối ưu hóa lý thuyết, và các tỷ lệ khác (chẳng hạn 3-3-3) có thể được khảo sát trong nghiên cứu tiếp theo.

Trong giai đoạn dự đoán, toàn bộ các mô hình trong ensemble đưa ra phân phối xác suất dự đoán cho mẫu đầu vào mới. Kết quả cuối cùng được xác định thông qua cơ chế Soft Voting có trọng số, trong đó xác suất của từng lớp được tính bằng tổng có trọng số của các xác suất thành phần. Nhãn dự đoán cuối cùng được chọn là lớp có xác suất tổng hợp lớn nhất.

Cách tiếp cận này cho phép giảm phương sai của các mô hình cơ sở nhờ cơ chế lấy mẫu bootstrap, đồng thời duy trì khả năng học biểu diễn của từng thuật toán riêng lẻ. Việc chọn lọc các mô hình có hiệu năng cao nhất trước khi áp dụng Bagging nhằm cân bằng giữa độ chính xác và tính ổn định của hệ thống. Chi tiết quy trình được trình bày trong Thuật toán 3.3.

**Thuật toán 3.3.** Hybrid Bagging Ensemble Learning

**Đầu vào:** Tập dữ liệu huấn luyện $D = \{(x_i, y_i)\}_{i=1}^{n}$ với $x_i \in \mathbb{R}^d$, $y_i \in \{0, 1\}$ (BENIGN $= 0$, Attack $= 1$)
Tập kiểm định $D_{\text{val}}$ (tách từ $D$, dùng để xếp hạng và chọn Top-3)
Tập $K$ thuật toán học cơ sở $\{\mathcal{L}_1, \mathcal{L}_2, \ldots, \mathcal{L}_K\}$
Phân bổ số bản sao $\mathbf{r} = (r_1, r_2, r_3) = (3, 2, 2)$
**Đầu ra:** Nhãn dự đoán cuối cùng $y_{\text{bag}} \in \{0, 1\}$

**Bước 1: Huấn luyện và đánh giá $K$ mô hình cơ sở**
**for** $k = 1$ to $K$ **do**
  $M_k \leftarrow \text{Train}(\mathcal{L}_k, D \setminus D_{\text{val}})$ ▷ Huấn luyện trên phần train lõi, tách khỏi $D_{\text{val}}$
  $F_k \leftarrow \text{F1-Score}(M_k, D_{\text{val}})$ ▷ Tính điểm F1 trên tập kiểm định (validation), không chạm tập kiểm tra
**end for**
**Bước 2: Chọn Top-3 mô hình có F1 cao nhất**
Sắp xếp $\{M_1, \ldots, M_K\}$ theo $F_k$ giảm dần
Chọn 3 mô hình tốt nhất: $M^{(1)}, M^{(2)}, M^{(3)}$ với $F^{(1)} \geq F^{(2)} \geq F^{(3)}$
**Bước 3: Tạo ensemble theo phân bố 3-2-2**
$\mathcal{E} \leftarrow \emptyset$ ▷ Tập ensemble gồm 7 mô hình
**for** $j = 1$ to $3$ **do**
  **for** $i = 1$ to $r_j$ **do**
    Tạo tập bootstrap $D_{j,i}$ bằng cách lấy ngẫu nhiên $n$ mẫu từ $D \setminus D_{\text{val}}$ **có hoàn lại**
    $h_{j,i} \leftarrow \text{Train}(\mathcal{L}^{(j)}, D_{j,i})$ ▷ Huấn luyện bản sao thứ $i$ của mô hình hạng $j$
    Gán trọng số $w_{j,i} \leftarrow F^{(j)}$ ▷ Trọng số dựa trên F1-Score
    Thêm $(h_{j,i}, w_{j,i})$ vào $\mathcal{E}$
  **end for**
**end for**
Chuẩn hóa trọng số: $w_{j,i} \leftarrow \dfrac{w_{j,i}}{\sum_{(h,w)\in\mathcal{E}} w}$
**Bước 4: Dự đoán cho mẫu mới $x$ (Soft Voting có trọng số)**
$T \leftarrow |\mathcal{E}| = 7$ ▷ Tổng số mô hình trong ensemble
**for** mỗi $(h_t, w_t) \in \mathcal{E}, t = 1, \ldots, T$ **do**
  $\mathbf{p}_t \leftarrow h_t(x)$ ▷ Lấy vector xác suất dự đoán
**end for**
Tính xác suất có trọng số: $\bar{p}(c \mid x) = \sum_{t=1}^{T} w_t \cdot P_t(c \mid x), \quad \forall c \in \{0, 1\}$
$y_{\text{bag}} \leftarrow \arg\max_{c\in\{0,1\}} \bar{p}(c \mid x)$
**return** $y_{\text{bag}}$

### 3.4 Đánh giá mô hình

Mỗi mô hình (cơ sở và ensemble) được đánh giá trên bốn chiều:

Bảng 3.5. Các tiêu chí đánh giá mô hình

| **Tiêu chí** | **Mô tả** |
|---|---|
| Độ chính xác (Accuracy) | Tỷ lệ dự đoán đúng trên toàn bộ mẫu kiểm tra |
| Precision | Tỷ lệ cảnh báo đúng trong tổng số cảnh báo được phát ra |
| Recall | Tỷ lệ phát hiện đúng trong tổng số mẫu tấn công thực tế |
| F1-Score | Trung bình điều hòa của Precision và Recall |
| Area Under the Curve – Receiver Operating Characteristic | Diện tích dưới đường cong ROC, đo hiệu năng phân biệt lớp |
| Area Under the Curve – PR | Diện tích dưới đường cong Precision-Recall |
| Chi phí tính toán | Thời gian huấn luyện, thời gian dự đoán và kích thước mô hình |
| Khả năng giải thích (Explainable Artificial Intelligence) | Biểu đồ SHAP: summary plot, bar chart và waterfall plot |

Hiệu năng của mô hình được đánh giá thông qua các chỉ số phân loại phổ biến như Accuracy, Precision, Recall và F1-Score, kết hợp với phân tích đường cong AUC (ROC và PR), chi phí tính toán (thời gian huấn luyện và suy luận), cũng như khả năng giải thích của mô hình (ví dụ: biểu đồ tổng quan SHAP). Các tiêu chí đánh giá được tổng hợp trong Bảng 3.5. Để kiểm chứng mô hình phát hiện xâm nhập, tập dữ liệu kiểm tra được sử dụng nhằm đánh giá khả năng tổng quát hóa của mô hình đã huấn luyện, trong đó kết quả dự đoán được đối chiếu với nhãn thực tế và đo lường bằng các thước đo đánh giá. Các chỉ số như accuracy (AC), precision (PR), recall (RE), F1-score (FS) được tính toán theo các công thức (3.4), (3.5), (3.6) và (3.7). Những kết quả này là cơ sở để so sánh giữa các phương pháp lựa chọn đặc trưng và lựa chọn mô hình phù hợp cho triển khai thực tế.

$$\text{Accuracy (AC)} = \frac{TP + TN}{TP + TN + FP + FN} \tag{3.4}$$

$$\text{Precision (PR)} = \frac{TP}{TP + FP} \tag{3.5}$$

$$\text{Recall (RE)} = \frac{TP}{TP + FN} \tag{3.6}$$

$$\text{F1-Score (FS)} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.7}$$

Các đại lượng TP, FP, FN, TN dùng trong các công thức trên được minh họa qua ma trận nhầm lẫn ở Bảng 3.6.

Bảng 3.6. Ma trận nhầm lẫn của mô hình phát hiện xâm nhập

| | **Dự đoán: Positive** | **Dự đoán: Negative** |
|---|---|---|
| **Thực tế: Positive** | TP (True Positive) | FN (False Negative) |
| **Thực tế: Negative** | FP (False Positive) | TN (True Negative) |

Trong đó, TP (True Positive) là số lượng lưu lượng mạng hoặc phiên kết nối thực sự là tấn công và được hệ thống IDS phát hiện đúng là xâm nhập. TN (True Negative) là những lưu lượng bình thường (không phải tấn công) và được mô hình phân loại chính xác là hợp lệ. FP (False Positive) là các lưu lượng bình thường nhưng bị hệ thống nhận diện nhầm là tấn công. Ngược lại, FN (False Negative) là những cuộc tấn công thực sự xảy ra nhưng không được hệ thống phát hiện và bị phân loại nhầm là lưu lượng bình thường.

## 3.5 Tối ưu hóa siêu tham số và huấn luyện mô hình

Trong quá trình xây dựng mô hình học máy, việc lựa chọn các siêu tham số (hyperparameters) phù hợp là một yếu tố quan trọng ảnh hưởng đến hiệu năng của mô hình. Để tối ưu hóa các siêu tham số này, phương pháp **Tìm kiếm theo lưới** (Grid Search (GS)) kết hợp với **Cross-Validation (CV)** đã được sử dụng trong môi trường phân tán của Apache Spark. Các mô hình phân loại được huấn luyện và so sánh gồm: Decision Tree, Logistic Regression, Linear SVC, Naive Bayes, Random Forest, GBT, XGBoost và MLP, sau đó xếp hạng theo F1-Score để chọn Top-3 mô hình cho các phương pháp ensemble.

- **GS**: Là phương pháp thử nghiệm tất cả các tổ hợp có thể của các siêu tham số được định nghĩa trước. Ví dụ, với một mô hình Random Forest, các tham số có thể bao gồm số lượng cây (`numTrees`), độ sâu tối đa (`maxDepth`), và chiến lược chọn tập con đặc trưng tại mỗi nút (`featureSubsetStrategy`). Spark MLlib hỗ trợ đối tượng `ParamGridBuilder` để xây dựng lưới các tham số.
- **CV**: Là phương pháp đánh giá mô hình bằng cách chia dữ liệu huấn luyện thành $k$ phần, sau đó huấn luyện và đánh giá mô hình $k$ lần, mỗi lần sử dụng một phần khác nhau làm dữ liệu kiểm tra và các phần còn lại làm dữ liệu huấn luyện. Trong Spark, đối tượng `CrossValidator` được sử dụng để thực hiện quy trình này.

Quy trình tối ưu hóa được thực hiện như sau:

1. Xây dựng pipeline gồm các bước tiền xử lý dữ liệu (ví dụ: chuẩn hóa, biến đổi đặc trưng) và mô hình học máy.
2. Sử dụng `ParamGridBuilder` để định nghĩa tập lưới các siêu tham số cần tìm kiếm.
3. Khởi tạo `CrossValidator` với pipeline, lưới các tham số, và bộ đánh giá hiệu năng (ví dụ: `BinaryClassificationEvaluator`).
4. Huấn luyện mô hình bằng cách gọi phương thức `fit()` của `CrossValidator` trên tập dữ liệu huấn luyện.
5. Chọn mô hình tốt nhất dựa trên điểm đánh giá trung bình trên các fold.

**Đoạn mã minh họa cách dùng API của Spark MLlib** (các giá trị trong lưới chỉ để minh họa, còn các mô hình được tinh chỉnh và kết quả tương ứng nằm ở Kịch bản 3, Chương 4):

```
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
from pyspark.ml import Pipeline

rf = RandomForestClassifier(labelCol="label", featuresCol="
    features")

pipeline = Pipeline(stages=[rf])

paramGrid = ParamGridBuilder() \
    .addGrid(rf.numTrees, [10, 20, 50]) \
    .addGrid(rf.maxDepth, [5, 10, 15]) \
    .build()

evaluator = BinaryClassificationEvaluator(labelCol="label",
    metricName="areaUnderPR")

cv = CrossValidator(estimator=pipeline,
                    estimatorParamMaps=paramGrid,
                    evaluator=evaluator,
                    numFolds=5)

cvModel = cv.fit(trainingData)
```

Quy trình này bảo đảm chọn được tổ hợp siêu tham số tốt nhất trong lưới đã định nghĩa, và Spark rút ngắn thời gian tìm kiếm nhờ xử lý phân tán, song song hóa trên cụm máy.

## 3.6 Triển khai biên

Mô hình tốt nhất sau đánh giá được đóng gói thành `PipelineModel` và triển khai trên một cụm *hai* thiết bị biên NVIDIA Jetson Orin Nano Super (CPU ARM 64-bit + GPU NVIDIA), tách vai trò: Jetson #1 chạy cổng lọc bất thường (autoencoder) để lọc lưu lượng bình thường, Jetson #2 chạy bộ phân loại PySpark trên các luồng nghi vấn. Mô hình sau khi huấn luyện trên cụm được chuyển sang thiết bị biên bằng Secure Copy Protocol (SCP). Quy trình suy luận thời gian thực theo chuỗi: Kafka Consumer nhận dữ liệu mạng (topic `ids-network-flow`), cổng lọc bất thường chuyển tiếp luồng nghi vấn sang `ids-suspicious-flow`, PySpark Streaming (micro-batch) xử lý và trích xuất đặc trưng, kết quả dự đoán ghi vào PostgreSQL phục vụ trực quan hóa trên Grafana và kích hoạt cảnh báo khi phát hiện bất thường. Hệ thống hỗ trợ ba chế độ: tách pipeline (pipeline split), mở rộng theo chiều ngang (horizontal scaling) và cụm Spark (Spark cluster). Thông số phần cứng chi tiết của thiết bị biên được trình bày trong Bảng 3.7.

Bảng 3.7. Thông số phần cứng thiết bị biên

| **Thuộc tính** | **Giá trị** |
|---|---|
| Nền tảng | NVIDIA Jetson Orin Nano Super Developer Kit ($\times 2$) |
| CPU | 6 nhân Arm Cortex-A78AE, 1,7 GHz |
| GPU | NVIDIA Ampere, 1024 nhân CUDA + 32 Tensor Core |
| RAM | 8 GB LPDDR5 (102 GB/s) |
| Lưu trữ | SSD NVMe 256 GB |
| Hiệu năng AI | tới $\sim$67 TOPS (khung 25 W) |
| Hệ điều hành | Ubuntu (NVIDIA JetPack / L4T) |
| Vai trò | Worker huấn luyện (Spark) & thiết bị suy luận biên |

# CHƯƠNG 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN

Chương này trình bày kết quả thực nghiệm và trả lời ba câu hỏi nghiên cứu nêu ở Mục 1.3.4. Phần đầu đánh giá các mô hình trên cụm Spark: hiệu năng cơ sở với toàn bộ đặc trưng, so sánh ba phương pháp giảm chiều (RQ1), tinh chỉnh siêu tham số, và so sánh học tổ hợp với mô hình đơn (RQ2). Tiếp theo là các phép kiểm tra độ tin cậy của những kết quả đó: ảnh hưởng của rò rỉ dữ liệu, kiểm định thống kê, đánh giá khác bộ dữ liệu và đánh giá đa lớp. Phần sau triển khai mô hình được chọn lên cụm hai Jetson Orin Nano Super và đo thông lượng, độ trễ và năng lượng (RQ3). Mục cuối tổng hợp và thảo luận các kết quả.

## 4.1 Môi trường thực nghiệm

Việc huấn luyện mô hình được thực hiện trên một **cụm Apache Spark Standalone phân tán** gồm một máy chủ và **hai thiết bị biên NVIDIA Jetson Orin Nano Super Developer Kit**. Cùng một cặp Jetson vừa đóng vai trò worker của cụm Spark trong pha huấn luyện, vừa là thiết bị triển khai trong pha suy luận thời gian thực (Hình 4.6), nhờ đó toàn bộ pipeline được kiểm chứng trên đúng phần cứng mục tiêu.

- **Máy chủ (Spark master):** máy tính xách tay MacBook Air, vi xử lý Apple M3, RAM 16 GB, macOS Sonoma 14.3. Máy này đảm nhận điều phối cụm (master), tiền xử lý dữ liệu thô (từ CSV sang Parquet) và chạy hạ tầng Docker (Kafka, PostgreSQL, InfluxDB, Grafana).
- **Hai nút worker: NVIDIA Jetson Orin Nano Super Developer Kit**, hệ điều hành Ubuntu (NVIDIA JetPack/L4T). Cấu hình chi tiết (CPU/GPU/RAM/lưu trữ) được nêu tại Bảng 3.7, Chương 3. Jetson #1 đồng thời chạy tiến trình *driver* của Spark (driver là tiến trình điều khiển ứng dụng, khác với vai trò *master* điều phối cụm của MacBook).
- **Phần mềm:** Python 3, Apache Spark / PySpark 3.4.1, MLlib. Cấu hình cụm tiêu biểu: `executor.memory`=3 GB, `executor.cores`=4, `driver.memory`=3 GB, `shuffle.partitions`=32. Mỗi Jetson được cấp 8 GB swap trên ổ NVMe để tránh tràn bộ nhớ khi cao điểm (xem `cluster/spark_cluster.env`, `cluster/setup_swap_jetson.sh`).

Các thuật toán học máy, phương pháp giảm chiều và các bước đánh giá đều được triển khai bằng PySpark và thực thi phân tán trên hai nút Jetson. Tập dữ liệu Parquet được đồng

bộ sang cả hai worker trước khi huấn luyện. Kiến trúc cụm huấn luyện phân tán (gồm vai trò *master* điều phối trên MacBook và hai *worker/executor* trên Jetson) được minh họa ở Hình 4.1. Cần phân biệt rõ với Hình 4.6 (pha suy luận): cùng một cặp Jetson nhưng ở pha huấn luyện đóng vai trò worker của cụm Spark, còn ở pha suy luận đóng vai trò cổng lọc (gate) và bộ phân loại.

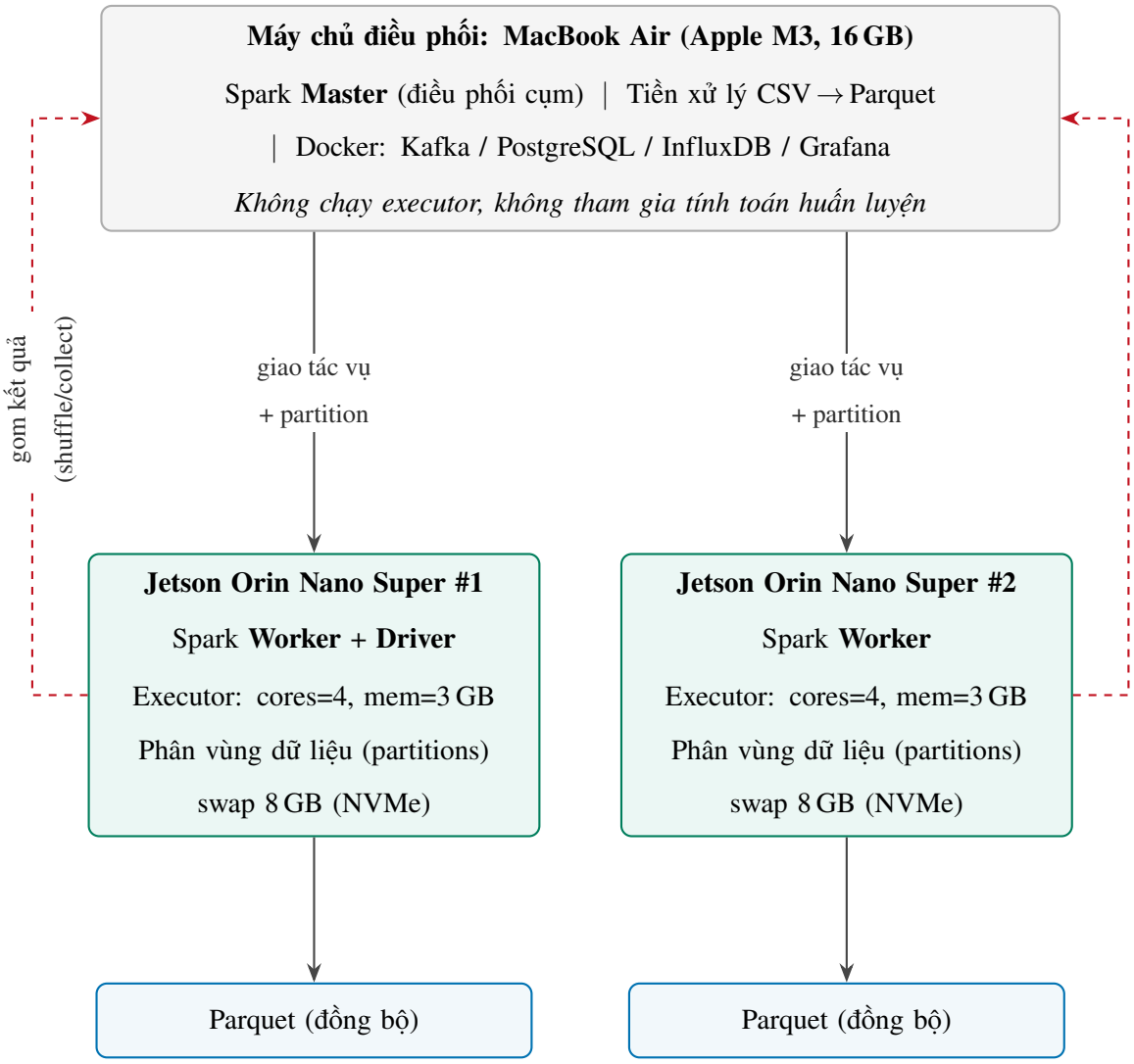
Máy chủ MacBook chỉ đảm nhận vai trò *master* điều phối cụm cùng tiền xử lý và hạ tầng dịch vụ, không tham gia tính toán huấn luyện (không chạy *executor*). Đây là một lựa chọn có chủ đích: số liệu hiệu năng báo cáo trong luận văn phải phản ánh đúng khả năng huấn luyện phân tán ngay trên phần cứng biên Jetson. Nếu đưa MacBook (vi xử lý Apple M3, mạnh hơn đáng kể) vào làm nút tính toán, phần lớn khối lượng công việc sẽ dồn về máy chủ, gây mất cân bằng tải (hiện tượng *straggler*) và làm sai lệch các phép đo thời gian/thông lượng, khiến kết quả không còn phản ánh đúng hiệu năng của cụm biên. Vì vậy, cụm đo đạc được giữ đồng nhất gồm hai nút Jetson, còn máy chủ chỉ được dùng để phát triển và kiểm thử nhanh trong giai đoạn xây dựng pipeline.

## 4.2 Kết quả thực nghiệm

Bảng 4.1 tóm tắt bộ tham số cố định của các mô hình. Bộ tham số này được chọn từ trước, dựa trên các nghiên cứu gần đây và thực nghiệm sơ bộ, và dùng chung cho Kịch bản 1 và 2. Nhờ vậy khác biệt quan sát được giữa các phương pháp giảm chiều phản ánh đúng bản thân phương pháp, thay vì mức độ tinh chỉnh riêng từng mô hình. Riêng Kịch bản 3 tinh chỉnh lại bốn mô hình (Random Forest, Decision Tree, GBT, Logistic Regression) bằng Grid Search kết hợp kiểm định chéo. Trong các bảng kết quả của chương, giá trị in đậm là giá trị tốt nhất trong cột (với bảng chia nhóm dòng: tốt nhất trong từng nhóm), trừ khi chú thích bảng nêu khác.

Bảng 4.1. Tham số cố định của các mô hình học máy

| **Mô hình** | **Tham số** | **Giá trị** | **Ý nghĩa/Mô tả** |
|---|---|---|---|
| Decision Tree | maxDepth | 15 | Bắt các mẫu tấn công phức tạp. |
| | impurity | entropy | Tiêu chí chia nhánh tối ưu. |
| | minInstancesPerNode | 5 | Số mẫu tối thiểu tại mỗi nút lá. |
| Logistic Reg. | maxIter | 200 | Bảo đảm hội tụ trên tập dữ liệu lớn. |
| | regParam | 0,001 | Kiểm soát mức điều chuẩn (regularization). |
| | elasticNetParam | 0,0 | Sử dụng Ridge (L2) ổn định hơn. |
| | threshold | 0,5 | Ngưỡng quyết định nhị phân. |
| SVM (LinearSVC) | maxIter | 200 | Số lần lặp tối ưu hóa biên. |
| | regParam | 0,001 | Tham số điều chuẩn biên quyết định. |
| NB | modelType | gaussian | Phù hợp cho đặc trưng liên tục. |
| | smoothing | 1,0 | Làm mượt Laplace. |
| Random Forest | numTrees | 200 | Bảo đảm độ ổn định kết quả. |
| | maxDepth | 15 | Chiều sâu tối đa của cây trong rừng. |
| | featureSubsetStrategy | sqrt | Chọn đặc trưng ngẫu nhiên mỗi nút. |
| GBT | maxIter | 150 | Số vòng boosting tối đa. |
| | maxDepth | 6 | Chiều sâu của các cây yếu. |
| | stepSize | 0,05 | Tốc độ học (learning rate) của boosting. |
| | subsamplingRate | 0,8 | Tỷ lệ lấy mẫu dữ liệu mỗi vòng boosting. |
| XGB | n_estimators | 300 | Số lượng cây boost trong mô hình. |
| | max_depth | 8 | Chiều sâu tối đa mỗi cây. |
| | learning_rate | 0,05 | Kiểm soát mức độ học của cây mới. |
| | subsample | 0,8 | Tỷ lệ lấy mẫu dữ liệu mỗi cây. |
| | colsample_bytree | 0,8 | Tỷ lệ lấy mẫu đặc trưng mỗi cây. |
| MLP | layers | [$d$, 64, 32, 2] | Mạng nơ-ron 2 lớp ẩn ($d$ = số đặc trưng đầu vào). |
| | maxIter | 150 | Số vòng lặp huấn luyện tối đa. |
| | stepSize | 0,01 | Tốc độ học của mạng nơ-ron. |




Hình 4.1. Cụm huấn luyện phân tán Apache Spark Standalone: máy chủ điều phối (master) và hai Jetson worker/executor

### 4.2.1 Kịch bản 1: Đánh giá hiệu năng cơ sở (Baseline)

Trong kịch bản này, toàn bộ 77 đặc trưng số (sau khi loại các cột cổng) được đưa vào mô hình mà chưa qua bất kỳ bước lựa chọn đặc trưng hay giảm chiều nào. Mục tiêu là thiết lập một mức hiệu năng tham chiếu cho các thử nghiệm tiếp theo.

Kết quả hiệu năng của các thuật toán ML đơn lẻ và phương pháp Ensemble được trình bày trong Bảng 4.2.

Bảng 4.2. Kết quả hiệu năng phân loại kịch bản Baseline (toàn bộ 77 đặc trưng số)

| **Thuật toán** | **Accuracy** | **Precision** | **Recall** | **F1-Score** | **AUC-ROC** | **AUC-PR** |
|---|---|---|---|---|---|---|
| Decision Tree | 0,9986 | 0,9925 | 0,9983 | 0,9954 | 0,9994 | 0,9988 |
| Logistic Regression | 0,9288 | 0,6937 | 0,9422 | 0,7991 | 0,9859 | 0,9509 |
| SVM (LinearSVC) | 0,9199 | 0,6650 | 0,9395 | 0,7788 | 0,9843 | 0,9485 |
| NB | 0,3163 | 0,1793 | 0,9933 | 0,3038 | 0,8829 | 0,6364 |
| Random Forest | 0,9985 | 0,9939 | 0,9960 | 0,9949 | 0,9999 | 0,9997 |
| GBT | 0,9980 | 0,9900 | 0,9965 | 0,9932 | 0,9998 | 0,9991 |
| XGB | **0,9991** | 0,9950 | **0,9993** | **0,9971** | **1,0000** | **0,9999** |
| MLP | 0,9900 | 0,9844 | 0,9487 | 0,9662 | 0,9984 | 0,9936 |
| **Hybrid Bagging** | 0,9990 | 0,9950 | 0,9984 | 0,9967 | **1,0000** | **0,9999** |
| **Soft Voting** | **0,9991** | **0,9952** | 0,9985 | 0,9969 | **1,0000** | **0,9999** |

Nhóm mô hình dựa trên cây dẫn đầu: XGB đạt F1 0,9971, Decision Tree 0,9954, Random Forest 0,9949 và GBT 0,9932. MLP đứng sau với 0,9662. Hai mô hình tuyến tính thấp hơn hẳn (Logistic Regression 0,7991, SVM 0,7788): recall vẫn khoảng 0,94 nhưng precision chỉ 0,69 và 0,67, tức khoảng một phần ba cảnh báo là sai. NB gần như gắn cờ mọi luồng (recall 0,9933, precision 0,1793) nên F1 chỉ 0,3038. Hai mô hình học tổ hợp (0,9967 và 0,9969) không vượt XGB đơn lẻ.

Các con số này thu được sau khi đã loại trừ rò rỉ dữ liệu, nên thấp hơn mức gần tuyệt đối của các cấu hình có rò rỉ và phản ánh đúng hơn độ khó của bài toán. Chênh lệch giữa các mô hình dẫn đầu rất nhỏ, vì vậy kết luận "mô hình nào tốt nhất" chỉ được đưa ra khi chênh lệch vượt khoảng tin cậy (Mục 4.2.7). Do dữ liệu mất cân bằng mạnh, AUC-PR và recall trên lớp tấn công được ưu tiên hơn Accuracy khi diễn giải: một mô hình luôn đoán lớp benign vẫn đạt Accuracy cao.

### 4.2.2 Kịch bản 2: So sánh chiến lược giảm chiều dữ liệu

Luận văn thực hiện so sánh 3 chiến lược chính nhằm giảm số lượng đặc trưng nhưng vẫn bảo đảm hiệu năng:

1. **RF Feature Importance**: Trích xuất 30 đặc trưng có độ quan trọng cao nhất từ Random Forest.

2. **SHAP Importance**: Sử dụng giá trị Shapley từ mô hình XGB để chọn ra 30 đặc trưng có ảnh hưởng lớn nhất.

3. **PCA (k=40)**: Giảm chiều dữ liệu từ không gian đặc trưng gốc (77 chiều) xuống 40 thành phần chính.

Lưu ý rằng PCA dùng $k = 40$, nhiều hơn 30 đặc trưng của RF/SHAP, vì đây là phép trích xuất chứ không phải chọn lọc: mỗi thành phần chính chỉ là một tổ hợp tuyến tính mang một phần phương sai, nên cần nhiều thành phần hơn để giữ lượng thông tin (phương sai tích lũy) tương đương với tập 30 đặc trưng gốc. Trong dải quét chung $\{20, 30, 40\}$, $k = 40$ là mức giữ phương sai cao nhất, nên là điểm vận hành đại diện công bằng nhất cho PCA. Ép $k = 30$ sẽ làm PCA mất thêm phương sai và bất lợi khi so sánh.

Bảng 4.3 so sánh bốn cấu hình. Mô hình đại diện (cột *Mô hình tốt nhất*) của mỗi phương pháp được chọn theo F1 trên tập kiểm định (validation) tách từ tập huấn luyện. Tập kiểm tra không tham gia khâu chọn này.

Bảng 4.3. So sánh hiệu năng tốt nhất của các phương pháp lựa chọn đặc trưng và giảm chiều

| **Phương pháp** | **Số đặc trưng** | **Mô hình tốt nhất** | **F1-Score** | **Huấn luyện (s)** | **Dung lượng (MB)** |
|---|---|---|---|---|---|
| Baseline (Toàn bộ) | 77 | XGB | **0,9971** | 4615,5 | 2,32 |
| RF Importance | 30 | XGB | 0,9922 | 1295,1 | 2,52 |
| SHAP Importance | 30 | XGB | 0,9970 | **1226,1** | 2,37[†] |
| PCA (k=40) | 40 | XGB | 0,9939 | 1444,8 | 3,82 |

[†]Kích thước trong bảng đo bằng cách tuần tự hóa mô hình *trên cụm huấn luyện phân tán*. Riêng với mô hình này, bước lưu phân tán ghi các phân vùng dữ liệu rừng cây trên executor đang giữ chúng, nên chỉ bắt được metadata cục bộ trên driver (0,003 MB). Con số 2,37 MB đo trên bản huấn luyện lại cùng cấu hình ở chế độ đơn nút (F1 kiểm thử 0,9971). Nó xấp xỉ bản RF Top-30, vì dung lượng ensemble cây phụ thuộc số cây và số nút chứ không phụ thuộc số đặc trưng đầu vào. Cũng vì vậy, tập đặc trưng rút gọn có thể cho mô hình *lớn hơn* baseline một chút (2,52 so với 2,32 MB): ít ứng viên tách hơn khiến cây cần nhiều nút hơn để đạt cùng độ tinh khiết. Phép đo tin cậy là bản mô hình xuất ra ở chế độ đơn nút của pipeline triển khai (Mục kết quả biên).

SHAP Top-30 giữ được hiệu năng của Baseline: F1 0,9970 so với 0,9971, trong khi số đặc trưng giảm từ 77 xuống 30 (giảm 61%) và thời gian huấn luyện giảm từ 4615,5 xuống 1226,1 giây, tức còn khoảng một phần tư. Hai phương pháp còn lại cũng rút ngắn thời gian huấn luyện tương tự nhưng mất hiệu năng nhiều hơn: PCA đạt 0,9939 và RF Importance đạt 0,9922. Chênh lệch giữa Baseline và SHAP Top-30 có đủ nhỏ để coi hai cấu hình là tương đương hay không được kiểm định ở Mục 4.2.7.

Bảng 4.4 so sánh trực tiếp các cách chọn 30 đặc trưng trên bốn mô hình cây. Danh sách chọn theo SHAP-XGB cho F1 cao hơn danh sách chọn theo độ quan trọng của RF ở cả bốn mô hình, với mức chênh tương đối từ $+0{,}48\%$ đến $+1{,}15\%$.

Bảng 4.4. So sánh hiệu năng lựa chọn đặc trưng (Top-30): RF Importance, SHAP xếp hạng trên Random Forest (SHAP-RF) và SHAP xếp hạng trên XGB (SHAP-XGB, danh sách được dùng)

| **Thuật toán** | **RF Importance** | **SHAP-RF** | **SHAP-XGB** | **Chênh lệch (%)** |
|---|---|---|---|---|
| Random Forest | 0,9879 | 0,9837 | 0,9955 | +0,77 |
| XGB | **0,9922** | **0,9942** | **0,9970** | +0,48 |
| GBT | 0,9844 | 0,9797 | 0,9918 | +0,75 |
| Decision Tree | 0,9835 | 0,9834 | 0,9948 | +1,15 |

Giá trị là F1 trên tập kiểm tra. Cột chênh lệch là mức chênh tương đối của SHAP-XGB so với RF Importance. Cột SHAP-RF đo trong một lần chạy riêng với cùng giao thức. Trong lần chạy đó, danh sách SHAP-XGB được chạy lại và lệch không quá 0,0002 so với cột SHAP-XGB.

Danh sách SHAP Top-30 ở trên được xếp hạng bằng XGB, trong khi mô hình triển khai ở biên là Random Forest. Để kiểm tra xem tính SHAP trực tiếp trên Random Forest có cho danh sách tốt hơn cho chính mô hình này hay không, luận văn tính thêm giá trị SHAP (TreeExplainer) trên Random Forest huấn luyện với đủ 77 đặc trưng, dùng cùng mẫu 2.000 luồng phân tầng và cùng giao thức đánh giá. Cột SHAP-RF của Bảng 4.4 cho thấy danh sách này kém danh sách SHAP-XGB ở cả bốn mô hình, kể cả với Random Forest (0,9837 so với 0,9955), và chỉ ngang mức RF Importance. Danh sách SHAP-RF trùng 25/30 đặc trưng với RF Importance nhưng chỉ 19/30 với SHAP-XGB, nghĩa là mô hình dùng để xếp hạng quyết định danh sách nhiều hơn cách đo.

Mười một đặc trưng chỉ có trong danh sách SHAP-RF phần lớn là bản sao gần như của các đặc trưng đã được chọn, ví dụ `packet_length_variance` (bình phương của độ lệch chuẩn), `subflow_bwd_bytes` và `avg_fwd_segment_size`. Mười một đặc trưng chỉ có trong danh sách SHAP-XGB là các đặc trưng thời gian và cờ TCP như `idle_min`, `flow_iat_min`, `fin_flag_count` và `psh_flag_count`. Random Forest chia điểm quan trọng cho các đặc trưng tương quan với nhau, còn boosting dồn điểm cho một đại diện rồi chuyển sang tín hiệu khác, nên Top-30 của XGB bao phủ nhiều nguồn thông tin hơn. Đây là lý do luận văn dùng xếp hạng SHAP của XGB cho mô hình Random Forest được triển khai. Kết quả này dựa trên một lần chia dữ liệu nên chưa có kiểm định thống kê, nhưng mức chênh (0,003–0,012) lớn hơn nhiều lần độ lệch giữa hai lần chạy và nhất quán ở cả bốn mô hình.

### 4.2.3 Kịch bản 3: Tối ưu hóa siêu tham số

Kịch bản này tinh chỉnh bốn mô hình Spark ML gốc (Random Forest, Decision Tree, GBT, Logistic Regression) bằng Grid Search kết hợp kiểm định chéo. Việc tinh chỉnh chạy trên cấu hình đặc trưng có F1 cao nhất ở Kịch bản 2, tức Baseline với toàn bộ 77 đặc trưng. Các mô hình còn lại (XGB, MLP, SVM, NB) giữ bộ tham số cố định ở Bảng 4.1.

Kết quả ở Bảng 4.5. Sau tinh chỉnh, ba mô hình cây đạt F1 quanh mức 0,995. Mức cải thiện so với cấu hình cố định là nhỏ (ví dụ Random Forest từ 0,9949 lên 0,9953), vì bài toán nhị phân cùng bộ đã gần bão hòa. Logistic Regression tăng từ 0,7991 lên 0,8226 nhưng vẫn cách xa nhóm cây.

Bảng 4.5. Kết quả hiệu năng phân loại kịch bản Tối ưu hóa siêu tham số

| **Thuật toán** | **Accuracy** | **Precision** | **Recall** | **F1-Score** | **AUC-ROC** | **AUC-PR** |
|---|---|---|---|---|---|---|
| Decision Tree | 0,9987 | 0,9977 | 0,9936 | 0,9957 | 0,9994 | 0,9980 |
| Logistic Regression | 0,9541 | 0,9787 | 0,7094 | 0,8226 | 0,9749 | 0,9371 |
| SVM (LinearSVC)‡ | 0,9199 | 0,6650 | 0,9395 | 0,7788 | 0,9843 | 0,9485 |
| NB‡ | 0,3163 | 0,1793 | 0,9933 | 0,3038 | 0,8829 | 0,6364 |
| Random Forest | 0,9986 | **0,9985** | 0,9922 | 0,9953 | 0,9999 | 0,9997 |
| GBT | 0,9985 | 0,9979 | 0,9924 | 0,9951 | 0,9999 | 0,9993 |
| XGB‡ | **0,9991** | 0,9950 | **0,9993** | **0,9971** | **1,0000** | **0,9999** |
| MLP‡ | 0,9900 | 0,9844 | 0,9487 | 0,9662 | 0,9984 | 0,9936 |
| **Hybrid Bagging** | 0,9986 | 0,9956 | 0,9949 | 0,9953 | 0,9999 | 0,9998 |
| **Soft Voting** | 0,9988 | **0,9985** | 0,9932 | 0,9958 | **1,0000** | 0,9998 |

‡Giữ bộ tham số cố định (Bảng 4.1), không tinh chỉnh. Số liệu là của Kịch bản 1 (Bảng 4.2) ghi lại để tiện so sánh trong cùng bảng.

### 4.2.4 Phân tích khả năng giải thích với SHAP

Tính giải thích (explainability) là một yêu cầu của hệ thống IDS, làm cơ sở để tin tưởng và vận hành mô hình trong thực tế. Trong nghiên cứu này, phương pháp SHAP (SHapley Additive exPlanations) được sử dụng để phân tích mức độ quan trọng của từng đặc trưng đối với quyết định phân loại của mô hình XGB.

Hình 4.2 xếp hạng 30 đặc trưng quan trọng nhất theo giá trị trung bình tuyệt đối của SHAP (mean $|\text{SHAP}|$): thanh càng dài, đặc trưng càng ảnh hưởng lớn đến quyết định phân loại xét trên toàn bộ mẫu. Để tránh rò rỉ, giá trị SHAP được tính trên một mẫu phân tầng theo lớp lấy từ tập huấn luyện (không phải tập kiểm tra). Mẫu này được biến đổi qua đúng bộ chuẩn hóa (StandardScaler) đã fit trên tập huấn luyện trước khi tính SHAP, bảo đảm không gian đầu vào của phép giải thích trùng khớp với không gian đầu vào mà mô hình đã

học.

Bổ sung cho xếp hạng tổng thể, Hình 4.3 trình bày biểu đồ SHAP Summary dạng beeswarm: mỗi điểm đại diện cho một quan sát, trục hoành thể hiện giá trị SHAP (dương đẩy dự đoán về lớp tấn công, âm đẩy về lớp bình thường), còn màu sắc biểu diễn giá trị đặc trưng từ thấp đến cao. Độ phân tán theo trục hoành phản ánh mức độ ảnh hưởng của đặc trưng, độ tập trung thể hiện tính ổn định của tác động.

Bên cạnh đó, giá trị trung bình tuyệt đối của SHAP (mean |SHAP|) được dùng để xếp hạng mức quan trọng tổng thể. Trên tập đặc trưng đã loại trừ rò rỉ, các đặc trưng có mean |SHAP| lớn nhất là: `bwd_packet_length_std`, `init_win_bytes_backward`, `init_win_bytes_forward` và `bwd_packet_length_mean`.

Các đặc trưng như `init_win_bytes_forward`, `init_win_bytes_backward` (kích thước cửa sổ TCP giai đoạn khởi tạo) và `bwd_packet_length_std`, `packet_length_std` (sự biến thiên độ dài gói tin) phản ánh đặc tính phân phối lưu lượng, tức các tín hiệu mạng có ý nghĩa và không phụ thuộc vào định danh dịch vụ. Việc loại `destination_port` buộc mô hình dựa vào các đặc trưng hành vi này thay vì ghi nhớ ánh xạ cổng $\rightarrow$ nhãn.

### 4.2.5 Phân tích rò rỉ dữ liệu (data leakage)

Đặc trưng `destination_port` là một nguồn rò rỉ nhãn tiềm tàng trên CICIDS2017: mỗi loại tấn công được sinh nhằm vào những cổng dịch vụ cố định nên cổng đích tương quan mạnh với nhãn, khiến mô hình dễ "ghi nhớ" ánh xạ cổng–nhãn thay vì học hành vi luồng. Engelen và cộng sự [60] xếp hiện tượng này vào nhóm *shortcut learning* của CICIDS2017: các thuộc tính định danh (Flow ID, địa chỉ IP, cổng nguồn, nhãn thời gian) phải bị loại khỏi biểu diễn đặc trưng, còn riêng việc dùng cổng đích làm đặc trưng cho IDS thì các tác giả xem là *còn gây tranh cãi*. Để định lượng ảnh hưởng, luận văn thực hiện phân tích loại bỏ (ablation) giữ/loại cổng với cùng một cấu hình Random Forest cố định a priori (`numTrees`=200, `maxDepth`=15, có trọng số lớp, bài toán nhị phân), kết quả ở Bảng 4.6. Với bài toán nhị phân trong-miền, sau khi đã loại trùng lặp luồng ở mức đặc trưng, ảnh hưởng riêng của `destination_port` lên F1 chỉ còn nhỏ (chênh khoảng $0{,}001$ điểm F1), bởi tác vụ nhị phân trên CICIDS2017 vốn gần mức bão hòa và nguồn rò rỉ còn lại (các luồng trùng lặp ở mức đặc trưng) đã được loại trước đó. Điều này không có nghĩa rò rỉ vô hại: giá trị của quy trình loại-trừ-rò-rỉ thể hiện rõ hơn ở đánh giá đa lớp với các lớp tấn công hiếm (Mục 4.2.10) và ở tổng quát hóa liên bộ dữ liệu (Mục 4.2.9), nơi các "lối tắt" rò rỉ

không còn giúp mô hình. Pipeline loại bỏ cả `destination_port` lẫn `source_port`, còn phân tích ablation tập trung vào `destination_port` vì đây là cổng chi phối.

Bảng 4.6. Phân tích loại bỏ (ablation) rò rỉ đặc trưng `destination_port` với cấu hình Random Forest cố định

| **Cấu hình** | **F1** | **Recall (Attack)** | **AUC-PR** |
|---|---|---|---|
| Giữ `destination_port` (rò rỉ) | 0,9965 | 0,9988 | 0,9999 |
| Loại `destination_port` (đề xuất) | 0,9955 | 0,9961 | 0,9997 |

Cả hai lượt dùng cùng một cấu hình Random Forest cố định từ trước (200 cây, độ sâu 15, trọng số lớp, khớp Bảng 4.1) để cô lập đúng ảnh hưởng của đặc trưng rò rỉ. Mức chênh lệch F1 giữa hai hàng phản ánh phần hiệu năng bị thổi phồng do đặc trưng rò rỉ. Ở đây mức chênh nhỏ vì đã loại trùng lặp trước và tác vụ nhị phân gần bão hòa, nhưng chiều hướng (giữ cổng cho F1/recall cao hơn) vẫn nhất quán với hiện tượng rò rỉ. Mọi kết quả khác trong luận văn dùng cấu hình đã loại cổng.

**Ảnh hưởng của cổng đích thay đổi theo họ mô hình.** Con số $\approx$0,001 ở trên là đặc tính của họ cây chứ không phải của bộ dữ liệu. Khi lặp lại đúng hai lượt đó trên cùng một phép chia cho cả tám thuật toán (Bảng 4.7, Hình 4.4), mức thổi phồng trải trên gần hai bậc độ lớn: các mô hình cây chỉ được lợi $+0{,}0007$ đến $+0{,}0017$ điểm F1, trong khi SVM và Logistic Regression được lợi tới $+0{,}0415$ và $+0{,}0395$, tức gấp khoảng 40 lần. Thứ tự này bám theo năng lực biểu diễn của mô hình: các ensemble cây dựng lại được cùng một biên quyết định từ các thống kê hành vi của luồng nên gần như không cần đến cổng, còn mô hình tuyến tính không đủ năng lực làm vậy nên bám vào cổng như một chỉ báo định danh dịch vụ. Hệ quả về phương pháp luận: *không thể suy ra mức rò rỉ của một lớp mô hình từ phép đo trên một mô hình đại diện khác*, và con số nhỏ báo cáo ở Bảng 4.6 là con số trung thực cho cấu hình được triển khai, không phải bằng chứng rằng đặc trưng này vô hại nói chung.

### 4.2.6 Phân tích hiệu năng hệ thống

Bên cạnh độ chính xác, hiệu năng hệ thống về thời gian và tài nguyên là yếu tố quyết định khả năng triển khai thực tế. Kết quả trong Bảng 4.8 cho thấy:

- **Thời gian huấn luyện**: Các mô hình Gradient Boosting (GBT, XGB) có thời gian huấn luyện dài nhất nhưng đạt kết quả phân loại cao.

Bảng 4.7. Mức thổi phồng F1 do `destination_port` theo từng thuật toán. Hai lượt huấn luyện trên cùng một phép chia

| **Thuật toán** | **F1 (loại cổng đích)** | **F1 (giữ cổng đích)** | **$\Delta$F1** |
|---|---|---|---|
| SVM (LinearSVC) | 0,7859 | 0,8274 | $+0{,}0415$ |
| Logistic Regression | 0,7986 | 0,8381 | $+0{,}0395$ |
| NB | 0,2985 | 0,3093 | $+0{,}0108$ |
| MLP | 0,9660 | 0,9711 | $+0{,}0052$ |
| GBT | 0,9924 | 0,9941 | $+0{,}0017$ |
| Decision Tree | 0,9953 | 0,9966 | $+0{,}0014$ |
| Random Forest | 0,9954 | 0,9964 | $+0{,}0010$ |
| XGB | 0,9972 | 0,9979 | $+0{,}0007$ |

Đây là một lượt chạy độc lập với Bảng 4.6: hàng Random Forest lệch khoảng $1 \times 10^{-4}$ so với bảng đó do dao động giữa các lần chạy Spark (phân vùng dữ liệu và thứ tự cộng dồn số thực), không phải do khác phương pháp. Kết luận về mức chênh $\approx$0,001 không đổi.

Bảng 4.8. So sánh thời gian xử lý và kích thước mô hình kịch bản Baseline

| **Thuật toán** | **Huấn luyện (s)** | **Dự đoán (s)** | **Kích thước (MB)** |
|---|---|---|---|
| Decision Tree | 33,4 | 6,1 | **0,0004** |
| Logistic Regression | 49,2 | 5,4 | 0,009 |
| SVM (LinearSVC) | 224,3 | 5,4 | 0,002 |
| NB | **8,2** | **5,2** | 0,012 |
| Random Forest | 610,3 | 18,6 | 3,69 |
| GBT | 1346,8 | 8,9 | 0,92 |
| XGB | 4615,5 | 9,0 | 2,32 |
| MLP | 314,7 | 8,0 | **0,0004** |
| **Hybrid Bagging** | 11934,5 | 36,4 | 7,24 |
| **Soft Voting** | – | 25,6 | – |

- **Thời gian dự báo**: XGB có thời gian dự báo nhanh (khoảng 9 giây cho toàn tập kiểm tra), Random Forest chậm hơn (khoảng 19 giây). Cả hai đều phù hợp cho giám sát theo lô gần thời gian thực.

- **Kích thước mô hình**: các mô hình cây chiếm dung lượng lớn hơn hẳn mô hình tuyến tính (LR, SVM, NB chỉ vài KB). RF khoảng 3,7 MB và ensemble Hybrid Bagging lớn nhất (∼7,2 MB) do gộp nhiều bộ học cơ sở. *Lưu ý: kích thước được đo bằng cách tuần tự hóa (serialize)* `PipelineModel` *của Spark ra đĩa. Giá trị này có thể biến thiên theo cách phân mảnh/partition khi lưu nên chỉ mang tính tham khảo tương đối, không dùng làm tiêu chí quyết định lựa chọn mô hình.*

### 4.2.7 Kiểm định ý nghĩa thống kê

Các giá trị F1 của những cấu hình dẫn đầu rất sát nhau, nên luận văn không kết luận "mô hình tốt nhất" từ một lần đo. Mục này kiểm định cặp cấu hình dẫn đầu, Baseline và SHAP Top-30, theo hai câu hỏi: hai cấu hình có khác nhau không, và nếu khác thì mức khác có đủ nhỏ để coi là tương đương không. Kết quả ở Bảng 4.9.

**Cách đo.** Mô hình đại diện của mỗi phương pháp, cũng như cặp phương pháp đưa vào kiểm định, đều được chọn theo F1 trên tập kiểm định (validation, 20% tách từ tập huấn luyện, `seed`=42). Tập kiểm tra không tham gia khâu chọn này. Hai phương pháp sau đó được huấn luyện lại trên $N$=6 lần lấy mẫu lại tập huấn luyện/kiểm tra, ghép cặp trên cùng một phép chia ở mỗi lần. Với mỗi phương pháp, luận văn báo cáo F1 trung bình kèm hai khoảng tin cậy 95%: khoảng $t$ thông thường và khoảng hiệu chỉnh *Nadeau–Bengio* [61]. Khoảng Nadeau–Bengio nhân phương sai với hệ số $(1/N + n_{\text{test}}/n_{\text{train}})$ để bù tương quan giữa các lần lấy mẫu lại chồng lấn, và được dùng làm căn cứ kết luận chính.

**Hai cấu hình có khác nhau không.** Khác biệt được kiểm bằng permutation test ghép cặp hai phía, liệt kê đầy đủ $2^N = 64$ hoán vị đổi dấu (kiểm định chính xác, không xấp xỉ Monte Carlo). Với cách này, $p$ nhỏ nhất có thể đạt là $2/64 \approx 0{,}031$. Độ lớn hiệu ứng được đo bằng Cohen's $d$ ghép cặp. Chênh lệch F1 giữa Baseline và SHAP Top-30 có ý nghĩa thống kê ở đúng mức sàn đó ($p \approx 0{,}031$, $d \approx 1{,}94$), nhưng giá trị tuyệt đối chỉ khoảng $0{,}0001$ điểm F1. Với $N$=6, kiểm định có lực thống kê thấp, nên riêng giá trị $p$ này chưa đủ để kết luận hai cấu hình tương đương hay không.

**Hai cấu hình có tương đương không.** Kiểm định ý nghĩa thông thường chỉ có thể bác bỏ giả thuyết "không khác biệt", không chứng minh được hai cấu hình tương đương. Luận văn vì vậy bổ sung kiểm định tương đương TOST (two one-sided tests [62]). Biên tương

đương được khai báo trước là $0{,}001$ điểm F1: mức chênh nhỏ hơn thế không có ý nghĩa vận hành với IDS, và biên này lớn hơn một bậc so với khoảng cách thực tế giữa hai cấu hình. Kiểm định dùng chính sai số chuẩn đã hiệu chỉnh Nadeau–Bengio ($4{,}58 \times 10^{-5}$), để các lần lấy mẫu lại chồng lấn không làm kết luận đẹp lên giả tạo.

Kết quả là $p_{\text{TOST}} = 3{,}9 \times 10^{-6}$: hai cấu hình tương đương theo nghĩa khẳng định, chứ không chỉ là "chưa phân biệt được". Biên nhỏ nhất mà sáu lần lấy mẫu lại này chứng minh được ở mức $\alpha = 0{,}05$ là $\pm 2{,}30 \times 10^{-4}$ điểm F1, tức chênh lệch thật giữa Baseline và SHAP Top-30 nhỏ hơn ngưỡng đó. Vì hai cấu hình tương đương về hiệu năng, các tiêu chí phụ (khả năng giải thích, kích thước, chi phí trên thiết bị biên) mới là căn cứ lựa chọn. Tăng $N$ kèm kiểm định chéo lồng vẫn là hướng củng cố tiếp theo.

Bảng 4.9. Độ ổn định qua nhiều lần lấy mẫu và kiểm định ghép cặp (từ `statistical_validity_multiseed.csv`)

| **Phương pháp** | **Mô hình** | **F1 (mean ± CI95 $t$)** | **CI95 Nadeau–Bengio** | **$p$ (ghép cặp)** | **Cohen's $d$** |
|---|---|---|---|---|---|
| Baseline (toàn bộ) | XGB | **0,99734 ± 0,00007** | [0,99722; 0,99746] | 0,031 | 1,94 |
| SHAP Top-30 | XGB | 0,99720 ± 0,00009 | [0,99705; 0,99736] | | |

### 4.2.8 Kiểm tra độ ổn định trong phân phối

Mô hình đại diện của mỗi phương pháp được đánh giá lại trên một mẫu con của tập kiểm tra (không giao với tập huấn luyện, xem Bảng 4.10). Đây chỉ là phép kiểm tra độ ổn định cùng phân phối (in-distribution), không phải kiểm thử dịch chuyển phân phối. Đánh giá tổng quát hóa thực sự nằm ở thí nghiệm liên bộ dữ liệu (Mục 4.2.9).

Bảng 4.10. Kiểm tra độ ổn định trong phân phối trên mẫu con tập kiểm tra (từ `robustness_holdout_summary.csv`)

| **Phương pháp** | **Mô hình** | **F1 (holdout)** | **AUC-PR** |
|---|---|---|---|
| Baseline | XGB | **0,9971** | **0,9999** |
| RF Top-30 | XGB | 0,9921 | 0,9998 |
| SHAP Top-30 | XGB | 0,9969 | **0,9999** |
| PCA | XGB | 0,9939 | 0,9998 |

### 4.2.9 Tổng quát hóa liên bộ dữ liệu

Các kết quả từ đầu chương đến đây đều là đánh giá *cùng bộ*: mô hình được huấn luyện và kiểm tra trên hai phần của cùng một bộ dữ liệu. Mục này đánh giá *khác bộ*: huấn luyện trên

một bộ dữ liệu (miền nguồn) rồi kiểm tra trên bộ còn lại (miền đích), theo cả hai chiều giữa CICIDS2017 và CSE-CIC-IDS2018. Đây là phép thử gần nhất với tình huống đưa một hệ thống đã huấn luyện sang một mạng khác. Mục này trả lời ba câu hỏi: hiệu năng giảm bao nhiêu khi đổi bộ dữ liệu, có khắc phục được mà không cần nhãn của miền đích hay không, và nếu cần nhãn thì cần bao nhiêu.

**Thiết lập thí nghiệm.** Mô hình là Random Forest có trọng số lớp (`weightCol`), giống cơ chế xử lý mất cân bằng ở Chương 3. Hai bộ dữ liệu đặt tên cột khác nhau, nên tên cột của CSE-CIC-IDS2018 được ánh xạ về tên chuẩn của CICIDS2017. Mô hình chỉ dùng các đặc trưng có ở cả hai bộ, sau khi đã loại các đặc trưng rò rỉ. Các bước loại trừ rò rỉ giống hệt quy trình cho CICIDS2017 (Thuật toán 3.1). Toàn bộ quy trình cho CSE-CIC-IDS2018 được trình bày ở Thuật toán 4.1.

CSE-CIC-IDS2018 [54] có khoảng 16 triệu luồng ($\sim$6 GB), quá lớn so với bộ nhớ 8 GB của Jetson. Vì vậy thí nghiệm dùng một mẫu 20% phân tầng theo nhãn: khoảng 3,14 triệu luồng, còn khoảng 2,39 triệu sau khi khử trùng lặp ở mức đặc trưng[1]. CICIDS2017 dùng khoảng 2,23 triệu luồng. Việc mở rộng lên toàn bộ CSE-CIC-IDS2018 là hướng phát triển tiếp theo.

Thí nghiệm được chạy 18 lượt trên cùng mã nguồn, cùng dữ liệu và cùng danh sách đặc trưng. Chỉ hạt giống của rừng ngẫu nhiên thay đổi: 13 hạt giống, trong đó 5 hạt giống được chạy hai lượt để đo riêng phần dao động giữa hai lượt có cấu hình giống hệt nhau. Kết quả ở Bảng 4.11.

Bảng 4.11. Tổng quát hóa liên bộ dữ liệu giữa CICIDS2017 và CSE-CIC-IDS2018 (trung bình qua 18 lượt chạy độc lập)

| **Huấn luyện** | **Kiểm tra** | **Loại** | **F1** | **AUC-PR** |
|---|---|---|---|---|
| CICIDS2017 | CICIDS2017 | cùng bộ | 0,9953 | 0,9997 |
| CICIDS2017 | CSE-CIC-IDS2018 | khác bộ | 0,029–0,085 (11) | 0,4778 |
| CICIDS2017 | CSE-CIC-IDS2018 | khác bộ | 0,185–0,301 (7) | 0,4960 |
| CSE-CIC-IDS2018 | CSE-CIC-IDS2018 | cùng bộ | 0,9246 | 0,9679 |
| CSE-CIC-IDS2018 | CICIDS2017 | khác bộ | 0,0544 $\pm$ 0,0019 | 0,1926 |

Chiều 2017$\rightarrow$2018 được ghi thành hai dòng vì F1 của 18 lượt tách thành hai nhóm rời nhau (giải thích ở phần dưới). Mỗi dòng ghi khoảng giá trị F1, số lượt trong ngoặc và AUC-PR trung bình của riêng nhóm đó. AUC-PR trung bình của cả 18 lượt là 0,4849. Ở dòng cuối, $\pm$ là nửa độ rộng khoảng tin cậy 95% (Student-$t$, $n = 18$).

**Hiệu năng khác bộ rơi xuống mức không dùng được.** Cùng bộ, mô hình đạt F1 0,9953 và 0,9246, gần như không đổi qua 18 lượt chạy (hệ số biến thiên không quá 0,1%).

[1]Mẫu được lấy với `seed`=42 qua công tắc `IDS_SAMPLE_FRAC`. Danh sách đặc trưng chung sau ánh xạ được lưu tại `common_features.txt`.

**Thuật toán 4.1.** Tiền xử lý và căn chỉnh CSE-CIC-IDS2018 cho đánh giá liên bộ dữ liệu

**Đầu vào:** Tập tệp CSV của CSE-CIC-IDS2018; lược đồ tên cột chuẩn của CICIDS2017 $\mathcal{S}_{2017}$;
bảng bí danh tên cột $\mathcal{A}$ (2018 $\rightarrow$ 2017); tỷ lệ lấy mẫu $r = 0.2$; $\texttt{seed} = 42$

**Đầu ra:** Bộ dữ liệu $D_{2018}$ trên tập đặc trưng chung đã loại rò rỉ $\mathcal{F}_{\text{chung}}$, sẵn sàng cho thí nghiệm khác bộ

**Bước 1: Nạp và làm sạch** (giống Thuật toán 3.1)
Đọc các tệp CSV; CLEANCOLUMNNAMES; HANDLEINFINITYVALUES; loại null và bản ghi trùng lặp hoàn toàn
**Bước 2: Căn chỉnh tên cột**
$\forall c \in \text{columns}(D) : \; c \leftarrow \mathcal{A}(c)$ ▷ ánh xạ tên cột CICFlowMeter v3 về tên chuẩn của CICIDS2017
**Bước 3: Tạo nhãn nhị phân (không phân biệt hoa/thường)**
$y_i \leftarrow 0$ nếu LOWER(label$_i$) = "benign", ngược lại $y_i \leftarrow 1$ ▷ 2018 ghi "Benign"; so khớp không phân biệt hoa/thường
**→ Loại trừ rò rỉ dữ liệu: Bước 4 và Bước 6**
**Bước 4 [loại rò rỉ – I]: Giao đặc trưng chung, đã loại trừ rò rỉ**
$\mathcal{F}_{\text{chung}} \leftarrow (\mathcal{F}_{2017} \cap \mathcal{F}_{2018}) \setminus \mathcal{C}_{\text{exclude}}$ ▷ $\mathcal{C}_{\text{exclude}}$: destination_port, source_port, protocol, IP, timestamp, flow_id
Lưu $\mathcal{F}_{\text{chung}}$ vào `common_features.txt` ▷ cố định cho cả hai chiều huấn luyện/kiểm tra
**Bước 5: Lấy mẫu phân tầng theo nhãn**
$D \leftarrow \text{STRATIFIEDSAMPLE}(D, r, \text{seed} = 42)$ ▷ bảo toàn tỷ lệ lớp; còn ∼3,14 triệu luồng
**Bước 6 [loại rò rỉ – II]: Loại trùng lặp ở mức đặc trưng** (tất định, như Bước 4b của Thuật toán 3.1)
Gom nhóm theo $\mathcal{F}_{\text{chung}}$; loại toàn bộ nhóm mâu thuẫn nhãn; giữ đúng một đại diện của mỗi nhóm nhất quán ▷ còn ∼2,39 triệu luồng
**return** $D_{2018}$ trên tập đặc trưng $\mathcal{F}_{\text{chung}}$

Khác bộ, F1 cao nhất quan sát được chỉ là 0,3009.

AUC-PR cho thấy mức rơi này rõ nhất, vì nó không phụ thuộc ngưỡng quyết định và ổn định giữa các lượt chạy (ở chiều 2017→2018 nó nằm trong khoảng 0,4444–0,5397, hệ số biến thiên 5,5%). AUC-PR giảm từ 0,9997 xuống 0,4849 ở chiều 2017→2018 và từ 0,9679 xuống 0,1926 ở chiều ngược lại. Mô hình mất từ một nửa đến bốn phần năm khả năng xếp hạng luồng độc hại.

Mô hình bỏ sót gần hết tấn công của bộ dữ liệu kia: recall trung bình chỉ 0,073 ở chiều 2017→2018 và 0,030 ở chiều ngược lại. Precision cao hơn recall nhiều lần (0,772 và 0,272), nên vấn đề chính là bỏ sót tấn công chứ không phải cảnh báo sai tràn lan.

Nguyên nhân là hai testbed khác nhau ở ba điểm: phiên bản CICFlowMeter (thang đo và cách tính một số đặc trưng thay đổi), hạ tầng mạng cùng cấu hình dịch vụ, và tổ hợp loại tấn công (HOIC, LOIC-HTTP của 2018 không có trong 2017). Mức sụt giảm này nhất quán với các nghiên cứu khác bộ trên IDS dựa trên đặc trưng luồng [63, 64, 65]. Nó củng cố luận điểm trung tâm của luận văn: F1 cùng bộ gần hoàn hảo, kể cả khi đã loại rò rỉ, không bảo đảm mô hình dùng được ở môi trường khác. Goldschmidt và Chudá cũng khuyến nghị không rút kết luận từ một bộ dữ liệu duy nhất mà nên đánh giá trên nhiều bộ dữ liệu [52].

**F1 chiều 2017→2018 tách thành hai nhóm rời nhau.** Mười tám lượt chạy chỉ khác nhau ở hạt giống, nhưng F1 của chúng không dao động quanh một giá trị trung tâm. Mười một lượt cho F1 0,0291–0,0843 với recall 0,0148–0,0447, gọi là *nhóm thấp*. Bảy lượt cho F1 0,1853–0,3009 với recall 0,1036–0,1778, gọi là *nhóm cao*. Không lượt nào rơi vào khoảng giữa 0,0843 và 0,1853. Chiều 2018→2017 không có hiện tượng này: mười tám giá trị F1 nằm liền nhau trong khoảng 0,0460–0,0593, hai giá trị kề nhau cách nhau nhiều nhất 0,0023.

Hạt giống không quyết định một lượt rơi vào nhóm nào. Trong năm hạt giống được chạy hai lượt, ba hạt giống cho hai lượt cùng nhóm và lệch nhau rất ít (seed 42: 0,3009 và 0,2936; seed 23: 0,0411 và 0,0434; seed 31: 0,0291 và 0,0303). Hai hạt giống còn lại đổi nhóm giữa hai lượt: seed 5 cho 0,0495 rồi 0,1854, seed 11 cho 0,0843 rồi 0,2606. Cùng hạt giống và cùng dữ liệu, hai lượt vẫn chênh nhau 3,7 lần và 3,1 lần. Nguồn dao động là quá trình huấn luyện phân tán: thứ tự Spark gộp các phân vùng khi tính biểu đồ tần suất để chia nút không cố định giữa các lượt, nên một vài ngưỡng chia nút được chọn khác đi.

Sai lệch nhỏ đó trở thành hai nhóm là do ngưỡng quyết định 0,5. Xác suất dự đoán của gần như toàn bộ luồng tấn công miền đích nằm sát ngưỡng này. Các luồng đó lại rất giống nhau, vì thuộc cùng vài họ tấn công và do cùng bộ công cụ sinh ra. Vì vậy chúng

không vượt ngưỡng lẻ tẻ từng luồng, mà cả một cụm lớn cùng nằm trên hoặc cùng nằm dưới ngưỡng. Recall nhảy thẳng từ ≈0,03 lên ≈0,15 mà không đi qua các giá trị trung gian.

AUC-PR xác nhận cách giải thích này, vì nó không dùng ngưỡng. AUC-PR trung bình là 0,4778 ở nhóm thấp và 0,4960 ở nhóm cao. Hai khoảng giá trị (0,4444–0,5163 và 0,4532–0,5397) chồng lên nhau trên phần lớn miền: 88% khoảng của nhóm thấp và 73% khoảng của nhóm cao. Kiểm định Welch trên 11 so với 7 lượt không đủ bằng chứng để nói hai nhóm khác nhau ($|t| = 1{,}30$, $\mathrm{df} \approx 9$). Khả năng xếp hạng của mô hình không đổi giữa các lượt. Chỉ vị trí của cụm luồng đó so với ngưỡng 0,5 là đổi.

**Hệ quả đối với cách báo cáo.** Thứ nhất, không nên báo cáo F1 trung bình cho chiều 2017→2018. Trung bình của 18 lượt là 0,1276, rơi đúng vào khoảng trống giữa hai nhóm nên không lượt nào cho giá trị ấy. Khoảng tin cậy 95% của nó ([0,0732; 0,1820], Student-$t$, $n = 18$) chỉ chứa 1 trong 18 lượt. Ghi con số này sẽ gợi ý sai rằng kết quả dao động quanh 0,13. Thứ hai, F1 khác bộ từ một lượt chạy duy nhất chủ yếu cho biết lượt đó rơi vào nhóm nào, chứ không mô tả mô hình.

Vì vậy Bảng 4.11 ghi khoảng giá trị và số lượt của từng nhóm, và luận văn dùng AUC-PR làm đại lượng chính khi so sánh khác bộ. Kết luận về mức sụt giảm không phụ thuộc vào nhóm: ngay ở nhóm cao, F1 khác bộ 0,30 và AUC-PR 0,50 vẫn cách rất xa mức cùng bộ 0,99 và 0,9997.

**Căn chỉnh đặc trưng không cần nhãn không thu hẹp được khoảng cách.** Nếu mức sụt giảm chỉ do hai testbed lệch thang đo thì một phép căn chỉnh tuyến tính là đủ để khắc phục. Luận văn kiểm chứng giả thuyết này bằng hai kỹ thuật thích ứng miền không giám sát kinh điển. Cả hai không dùng nhãn nào của miền đích và không huấn luyện lại bộ phân loại, đúng với tình huống một hệ đã triển khai gặp mạng mới:

- **Chuẩn hóa theo miền**: giữ nguyên rừng cây, chỉ fit lại `StandardScaler` trên phần đặc trưng (không nhãn) của tập huấn luyện miền đích.
- **CORAL** [66]: làm trắng miền đích bằng hiệp phương sai của chính nó rồi "tô" lại bằng hiệp phương sai miền nguồn, tức căn khớp cả moment bậc hai. Bài gốc biến đổi miền nguồn về phía miền đích rồi huấn luyện lại, còn ở đây phép biến đổi được áp dụng theo chiều ngược lại vì bộ phân loại được giữ nguyên.

Bảng 4.12 cho thấy cả hai kỹ thuật đều đưa F1 về 0: sau khi căn chỉnh, mô hình gần như ngừng báo tấn công. Kết quả này bác bỏ giả thuyết "chỉ lệch thang đo". Nếu hai miền chỉ khác nhau ở trung bình và hiệp phương sai (moment bậc một và bậc hai) thì hai phép căn

Bảng 4.12. Hai kỹ thuật thích ứng miền không giám sát trên tác vụ liên bộ dữ liệu (bộ phân loại giữ nguyên, không dùng nhãn miền đích)

| Chiều | Cấu hình | F1 | AUC-PR |
|---|---|---|---|
| 2017 → 2018 | Không thích ứng | **0,2978** | **0,5261** |
| | Chuẩn hóa theo miền | 0,0000 | 0,3169 |
| | CORAL | 0,0000 | 0,2015 |
| 2018 → 2017 | Không thích ứng | **0,0575** | 0,1791 |
| | Chuẩn hóa theo miền | 0,0000 | 0,1068 |
| | CORAL | 0,0000 | **0,2496** |

Ba dòng của mỗi chiều đến từ cùng một lượt chạy (hạt giống 42, thuộc nhóm cao, từ `cross_dataset_adaptation_high_regime.csv`) nên so sánh được với nhau. Kết quả F1 về 0 lặp lại ở cả bốn lượt chạy độc lập đã thử: ba lượt nhóm cao (F1 khi không thích ứng 0,2938–0,3001, trong đó có lượt của Bảng 4.13) và một lượt nhóm thấp (0,0423, lưu tại `sweep/logs/xd_clean_run.log`). Sau ánh xạ CORAL, trung bình và hiệp phương sai của miền đích khớp miền nguồn với sai số tương đối $4 \times 10^{-12}$.

chỉnh này đã đủ. Phần khác biệt thực sự nằm ở phân phối có điều kiện, tức ở chỗ điều gì làm một luồng trở thành độc hại, và hai testbed còn dùng bộ công cụ tấn công không trùng nhau.

Có một chi tiết đáng chú ý. Ở chiều 2018→2017, CORAL nâng AUC-PR từ 0,1791 lên 0,2496 trong khi F1 vẫn bằng 0. Phép căn chỉnh có làm thứ tự xếp hạng tốt lên, nhưng ngưỡng quyết định 0,5 học từ miền nguồn không còn hiệu lực trong không gian đặc trưng đã đổi. Với triển khai thực tế, thích ứng miền cho IDS vì vậy không thể dừng ở căn chỉnh đặc trưng mà phải kèm hiệu chỉnh lại ngưỡng trên miền đích.

**Gán nhãn 1% dữ liệu miền đích là đủ.** Khi không kỹ thuật không giám sát nào hiệu quả, câu hỏi thực tế là phải gán nhãn bao nhiêu lưu lượng của mạng mới để có bộ phát hiện dùng được. Luận văn đo trực tiếp: lấy ngẫu nhiên 1% tập huấn luyện của miền đích kèm nhãn, huấn luyện lại mô hình từ đầu, rồi đánh giá trên tập kiểm tra của miền đích. Mẫu 1% giữ tỷ lệ tấn công tự nhiên chứ không cân bằng 50/50, vì kịch bản là gán nhãn những gì đi qua mạng.

Hai cấu hình được so sánh: gộp dữ liệu nguồn với 1% nhãn đích, và chỉ dùng 1% nhãn đích. Cấu hình thứ hai là đối chứng, cho biết dữ liệu nguồn có đóng góp gì không. Kết quả ở Bảng 4.13.

Có hai kết quả. Thứ nhất, **khoảng 19 nghìn luồng có nhãn là đủ**: F1 tăng từ 0,29 lên 0,92 ở chiều 2017→2018 và từ 0,06 lên 0,99 ở chiều ngược lại. Mạng mới không khó phát hiện. Cái không chuyển sang được là kiến thức của mô hình cũ.

Thứ hai, và bất ngờ hơn, **dữ liệu miền nguồn không những vô ích mà còn có hại**. Ở chiều 2017→2018, chỉ dùng 19 nghìn luồng có nhãn của miền đích cho F1 0,9223. Gộp

Bảng 4.13. Ngân sách nhãn miền đích: huấn luyện lại với 1% dữ liệu có nhãn của miền đích

| Chiều | Cấu hình | Nhãn đích | F1 | Recall |
|---|---|---|---|---|
| 2017 → 2018 | Không thích ứng | 0 | 0,2939 | 0,1749 |
| | Nguồn + 1% nhãn đích | 19.276 | 0,8034 | 0,6750 |
| | **Chỉ 1% nhãn đích** | 19.276 | **0,9223** | **0,9198** |
| 2018 → 2017 | Không thích ứng | 0 | 0,0574 | 0,0322 |
| | Nguồn + 1% nhãn đích | 18.011 | 0,9607 | **0,9853** |
| | **Chỉ 1% nhãn đích** | 18.011 | **0,9897** | 0,9849 |

1% được lấy từ tập huấn luyện của miền đích, không lấy từ tập kiểm tra. Dòng "không thích ứng" lấy từ chính lượt chạy này nên ba dòng so sánh được với nhau. Lượt này thuộc nhóm cao (F1 0,2939). Kết luận không phụ thuộc vào mức đó: F1 sau khi có 1% nhãn vẫn cao gấp khoảng ba lần giá trị khác bộ cao nhất quan sát được (0,3009).

thêm 1,79 triệu luồng có nhãn của miền nguồn kéo F1 xuống 0,8034 và recall từ 0,92 xuống 0,68. Chiều ngược lại cùng xu hướng (0,9897 so với 0,9607). Dữ liệu nguồn nhiều áp đảo nên kéo mô hình về những mẫu hình đặc thù của testbed cũ, lấn át phần dữ liệu đích ít nhưng đúng phân phối.

Khuyến nghị triển khai rút ra từ đây rất cụ thể. Khi đưa hệ thống sang một mạng mới, không nên dùng lại mô hình cũ hay gộp dữ liệu cũ vào. Nên gán nhãn một lượng nhỏ lưu lượng tại chỗ (cỡ 1%, tương đương vài giờ công của một phân tích viên) rồi huấn luyện lại từ đầu. Kết quả này cũng giải thích vì sao hai kỹ thuật không giám sát ở trên thất bại: khoảng cách giữa hai miền nằm ở chính ranh giới quyết định chứ không ở một phép biến đổi tuyến tính của không gian đặc trưng, và ranh giới đó chỉ học được từ nhãn của miền đích.

### 4.2.10 Đánh giá đa lớp theo loại tấn công

Phân loại nhị phân (Benign/Attack) trên CICIDS2017 vốn dễ tách. Để bộc lộ điểm yếu thực sự, luận văn bổ sung đánh giá đa lớp theo từng loại tấn công gốc (precision/recall/F1 từng lớp, macro-F1, weighted-F1 và ma trận nhầm lẫn), gồm lớp Benign và 14 loại tấn công của CICIDS2017. Kết quả cho thấy tương phản rất rõ giữa hai chỉ số tổng hợp: weighted-F1 $\approx 0{,}998$ (bị chi phối bởi các lớp đa số) nhưng macro-F1 chỉ $\approx 0{,}806$. Đây là bằng chứng cho thấy độ chính xác nhị phân gần hoàn hảo đã che giấu hiệu năng phát hiện kém ở các lớp hiếm. Cụ thể, các lớp lớn (Benign, DoS Hulk, DDoS) và cả PortScan (chỉ còn 383 mẫu sau khử trùng lặp) đạt F1 rất cao ($\geq 0{,}97$), trong khi các lớp cực hiếm suy giảm nghiêm trọng: Web Attack – XSS recall chỉ $0{,}027$, Web Attack – SQL Injection F1 $= 0$ (chỉ 6 mẫu), Bot

recall $0{,}46$. Đây là hướng cần cải thiện (xử lý mất cân bằng, tăng cường dữ liệu lớp hiếm). Đánh giá đa lớp hiện mới chạy cho Random Forest dùng toàn bộ đặc trưng. So sánh các phương pháp lựa chọn đặc trưng ở chế độ đa lớp là hướng tiếp theo. Kết quả chi tiết theo từng lớp được trình bày trong Bảng 4.14, và ma trận nhầm lẫn đa lớp tương ứng được minh họa ở Hình 4.5.

Bảng 4.14. Hiệu năng theo từng lớp tấn công, Random Forest dùng toàn bộ đặc trưng (từ `per_class_metrics.csv`)

| **Lớp** | **Support** | **Precision** | **Recall** | **F1** |
|---|---|---|---|---|
| Benign | 380.119 | 0,9981 | 0,9997 | 0,9989 |
| DoS Hulk | 34.446 | 0,9990 | 0,9904 | 0,9947 |
| DDoS | 25.694 | 0,9996 | 0,9992 | 0,9994 |
| DoS GoldenEye | 2.020 | 1,0000 | 0,9757 | 0,9877 |
| FTP-Patator | 1.147 | 0,9965 | 0,9965 | 0,9965 |
| DoS slowloris | 1.068 | 0,9953 | 0,9925 | 0,9939 |
| DoS Slowhttptest | 1.023 | 0,9286 | 0,9912 | 0,9589 |
| SSH-Patator | 642 | 1,0000 | 0,9346 | 0,9662 |
| PortScan | 383 | 1,0000 | 0,9530 | 0,9759 |
| Web Attack – Brute Force | 311 | 0,7346 | 0,7299 | 0,7323 |
| Bot | 290 | 1,0000 | 0,4552 | 0,6256 |
| Web Attack – XSS | 111 | 1,0000 | 0,0270 | 0,0526 |
| Infiltration | 12 | 1,0000 | 0,6667 | 0,8000 |
| Web Attack – SQL Injection | 6 | 0,0000 | 0,0000 | 0,0000 |
| Heartbleed | 3 | 1,0000 | 1,0000 | 1,0000 |
| **macro-F1 / weighted-F1** | | | 0,806 / 0,998 | |

**Phân tích hướng nhầm lẫn giữa các lớp hiếm.** Ma trận nhầm lẫn cho thấy các lớp hiếm suy giảm theo hai kiểu khác nhau, tùy dự đoán sai rơi vào lớp nào.

Kiểu thứ nhất là nhầm loại nhưng vẫn bị phát hiện. 81/111 mẫu Web Attack – XSS bị gán thành Web Attack – Brute Force. Ở bài toán nhị phân triển khai trên biên, các luồng này vẫn bị gắn cờ tấn công (76% XSS vẫn bị chặn), chỉ sai ở khâu định danh loại.

Kiểu thứ hai nguy hiểm hơn: lọt hẳn thành Benign. Toàn bộ 6/6 mẫu SQL Injection, 158/290 mẫu Bot (54%) và 84/311 mẫu Web Brute Force bị dự đoán là lưu lượng bình thường.

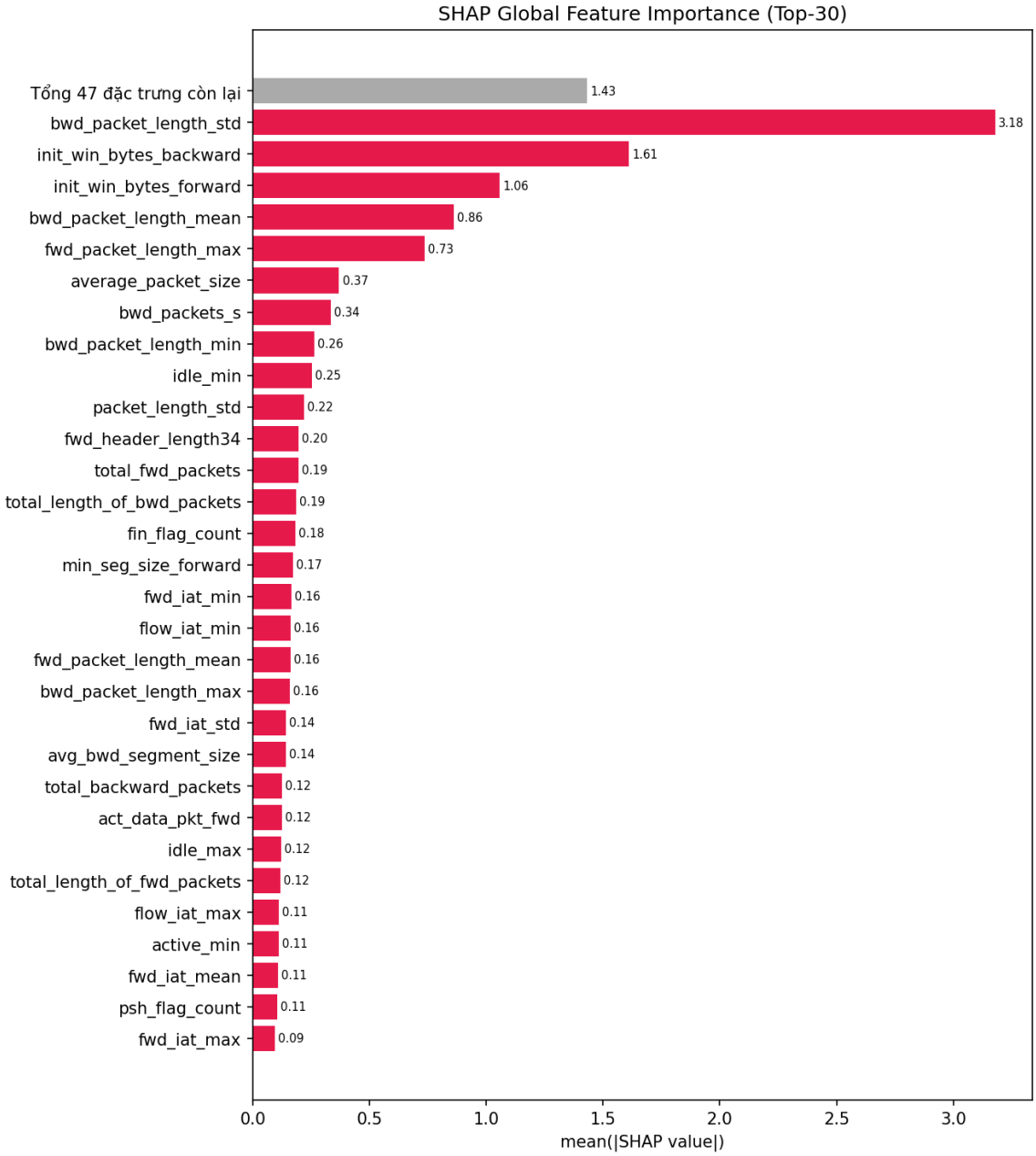
Có ba nguyên nhân:

- **Mất cân bằng cực đoan**: mỗi lớp hiếm chỉ có 6–311 mẫu so với 380.119 mẫu Benign, nên gần như không có vùng quyết định riêng.
- **Loại `destination_port`**: ảnh hưởng nặng nhất đến nhóm tấn công web. Khi mất "lối tắt" cổng 80, các luồng HTTP tấn công (request/response ngắn) có đặc trưng

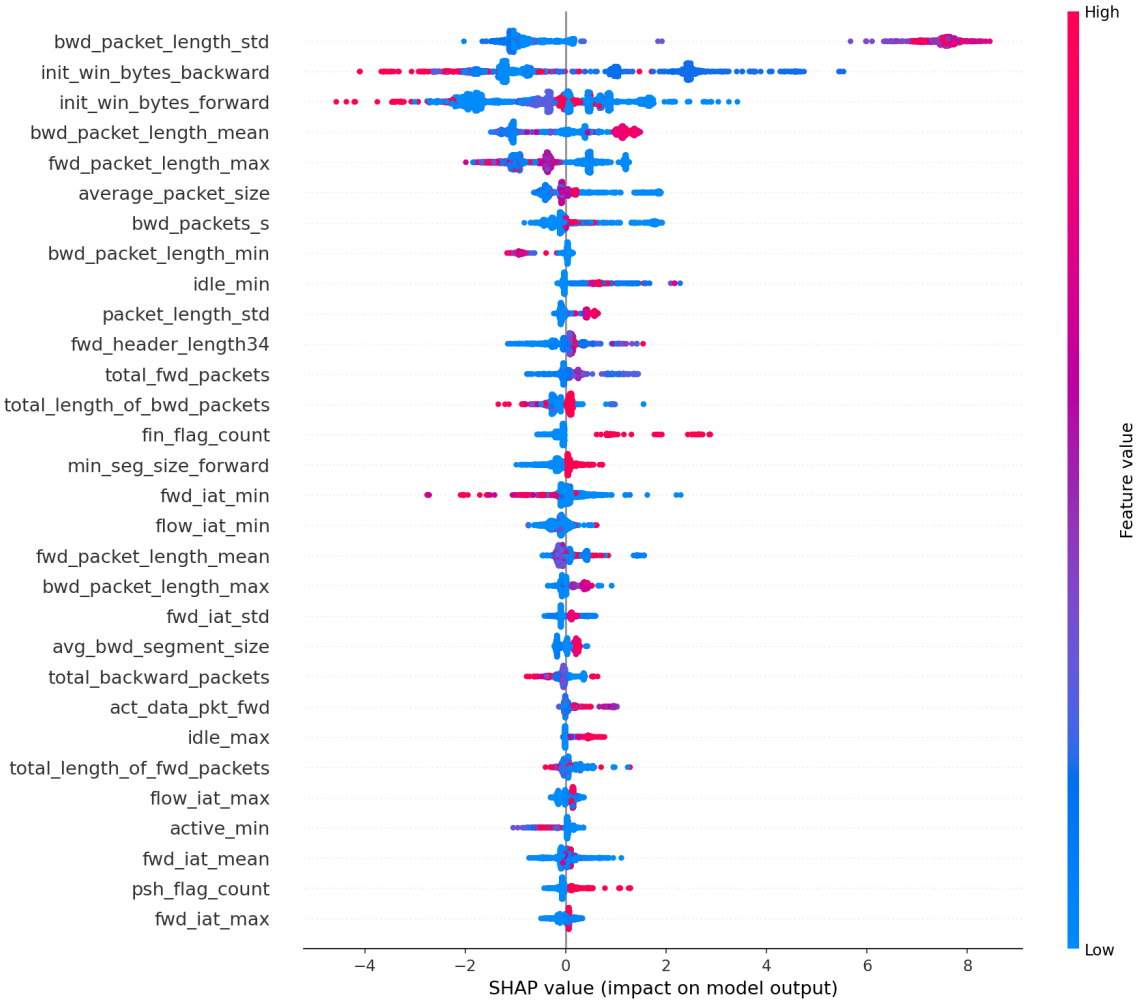
hành vi rất gần lưu lượng duyệt web thường và gần lẫn nhau. Recall 0,027 của XSS vì vậy là con số trung thực, thay cho con số đẹp nhờ rò rỉ.

- **Bot giống Benign về bản chất**: các luồng beaconing C&C nhỏ và đều đặn, giống lưu lượng nền khi không còn định danh cổng/dịch vụ.

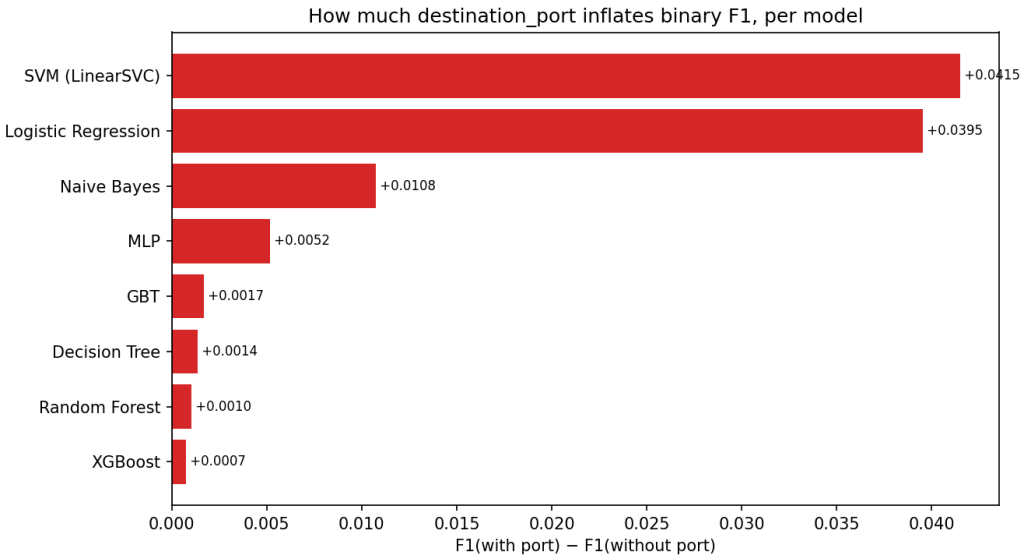
Các nguyên nhân này dẫn thẳng tới các hướng phát triển: xử lý mất cân bằng có chủ đích (tăng cường/oversampling lớp hiếm), bổ sung đặc trưng chuỗi thời gian cho hành vi beaconing, và kiểm chứng liên bộ dữ liệu (Mục 4.2.9).



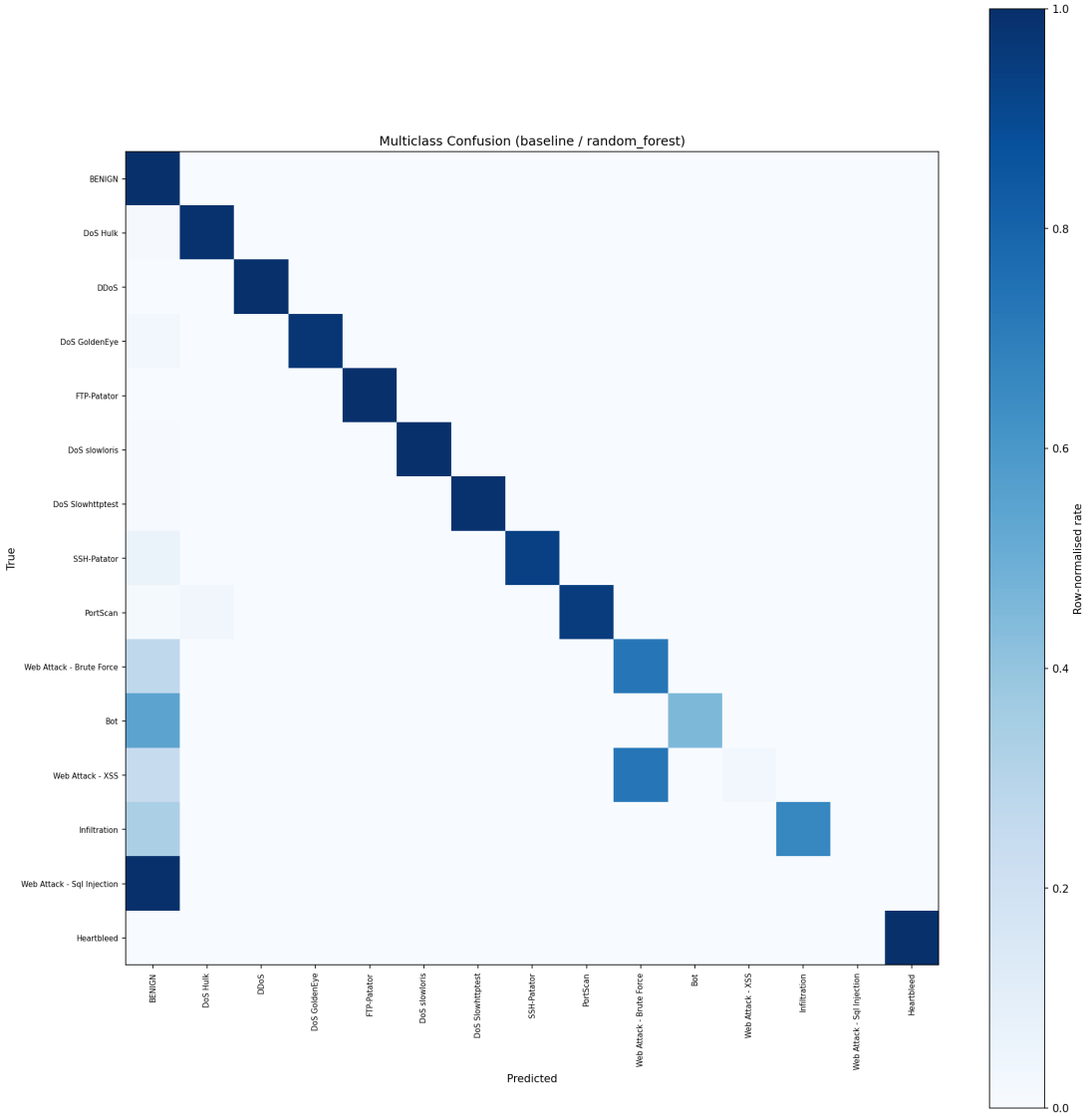

Hình 4.2. Top 30 đặc trưng quan trọng theo giá trị trung bình tuyệt đối SHAP của mô hình XGB




Hình 4.3. Biểu đồ SHAP Summary (beeswarm): phân bố và hướng tác động của từng đặc trưng




Hình 4.4. Mức thổi phồng F1 do `destination_port` theo từng thuật toán: mô hình tuyến tính phụ thuộc cổng nặng hơn hẳn các ensemble cây




Hình 4.5. Ma trận nhầm lẫn đa lớp theo loại tấn công, Random Forest dùng toàn bộ đặc trưng (chuẩn hóa theo hàng)

## 4.3 Thực nghiệm và đánh giá hiệu năng mô hình trên Jetson Orin Nano Super

Trong phần này, luận văn đề xuất xây dựng một hệ thống phát hiện xâm nhập mạng IDS phân tán triển khai trên một cụm **hai thiết bị biên NVIDIA Jetson Orin Nano Super** [67] nhằm đánh giá khả năng hoạt động trong môi trường tài nguyên hạn chế. Đây là hướng tiếp cận điện toán biên đưa phân tích về gần nguồn dữ liệu để giảm băng thông và thời gian phản hồi [68, 69], trong khi các nghiên cứu IDS biên trước đây chủ yếu đánh giá suy luận đơn nút [70]. Hệ thống được thiết kế theo kiến trúc xử lý dữ liệu thời gian thực hai tầng: một cổng lọc bất thường (anomaly gate) bằng autoencoder nhẹ lọc bớt lưu lượng bình thường trên Jetson #1, và một bộ phân loại PySpark `PipelineModel` chỉ xử lý các luồng nghi vấn trên Jetson #2. Hạ tầng trung tâm (Apache Kafka cho truyền tải thông điệp, PostgreSQL để quản lý dữ liệu, InfluxDB để lưu chỉ số hiệu năng theo thời gian, Grafana để trực quan hóa và Mailtrap để kiểm thử cảnh báo qua email) chạy trên một máy chủ điều phối.

NVIDIA Jetson Orin Nano Super là một máy tính nhúng (single-board computer) tích hợp CPU kiến trúc ARM 64-bit và GPU NVIDIA, hỗ trợ tăng tốc suy luận học máy ngay tại biên. Cấu hình chi tiết đã nêu tại Bảng 3.7 (Chương 3) và mục Môi trường thực nghiệm. Do hạn chế về tài nguyên tính toán so với máy chủ truyền thống, việc triển khai trên Jetson Orin Nano Super cho phép đánh giá khả năng tối ưu hóa mô hình và hiệu năng suy luận trong điều kiện phần cứng nhúng (edge computing).

Mô hình có hiệu năng tối ưu được xác định ở phần đánh giá hiệu năng phân loại phía trên sẽ được tích hợp vào pipeline xử lý nhằm thực hiện nhiệm vụ phát hiện và phân loại lưu lượng mạng thành hai nhóm: *Benign* và *Attack*. Các chỉ số đánh giá bao gồm độ chính xác (Accuracy), Precision, Recall, F1-Score, thời gian dự đoán trung bình cho mỗi mẫu, thông lượng xử lý (throughput) và mức tiêu thụ tài nguyên hệ thống (CPU, RAM). Kết quả thực nghiệm cho phép phân tích tính khả thi của việc triển khai hệ thống IDS dựa trên học máy trong môi trường thiết bị biên có tài nguyên giới hạn.

### 4.3.1 Các công cụ sử dụng trong hệ thống

Trong hệ thống phát hiện xâm nhập mạng IDS được đề xuất, ngoài các công nghệ xử lý dữ liệu lớn đã trình bày ở Chương 2, luận văn sử dụng thêm một số công cụ và nền tảng mã nguồn mở nhằm hỗ trợ truyền tải dữ liệu, lưu trữ, giám sát hiệu năng và cảnh báo theo thời gian thực. Cụ thể:

- **Apache Kafka** [71] – Là nền tảng truyền tải thông điệp phân tán (distributed event

streaming platform), được sử dụng để tiếp nhận và phân phối dữ liệu luồng theo cơ chế publish-subscribe. Trong hệ thống, Kafka đóng vai trò trung gian giữa nguồn dữ liệu (CICIDS2017 được mô phỏng bằng Python script) và thiết bị Jetson Orin Nano Super, bảo đảm khả năng truyền tải bất đồng bộ, chịu lỗi và mở rộng linh hoạt.

- **Apache Spark Streaming** – Là thành phần xử lý dữ liệu luồng thời gian thực. Spark Streaming nhận dữ liệu từ Kafka Consumer, thực hiện các bước tiền xử lý đặc trưng và chuyển dữ liệu đến mô hình học máy đã được huấn luyện để thực hiện suy luận.

- **PostgreSQL** – Là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở có độ ổn định và khả năng mở rộng cao. Trong hệ thống, PostgreSQL được sử dụng để lưu trữ kết quả dự đoán, thông tin cảnh báo và các bản ghi phục vụ báo cáo tổng hợp.

- **InfluxDB** – Là cơ sở dữ liệu chuỗi thời gian (Time-Series Database), tối ưu cho việc lưu trữ và truy vấn các chỉ số theo thời gian. Hệ thống sử dụng InfluxDB để ghi nhận các thông số hiệu năng như thời gian suy luận, mức sử dụng CPU, bộ nhớ và thông lượng xử lý.

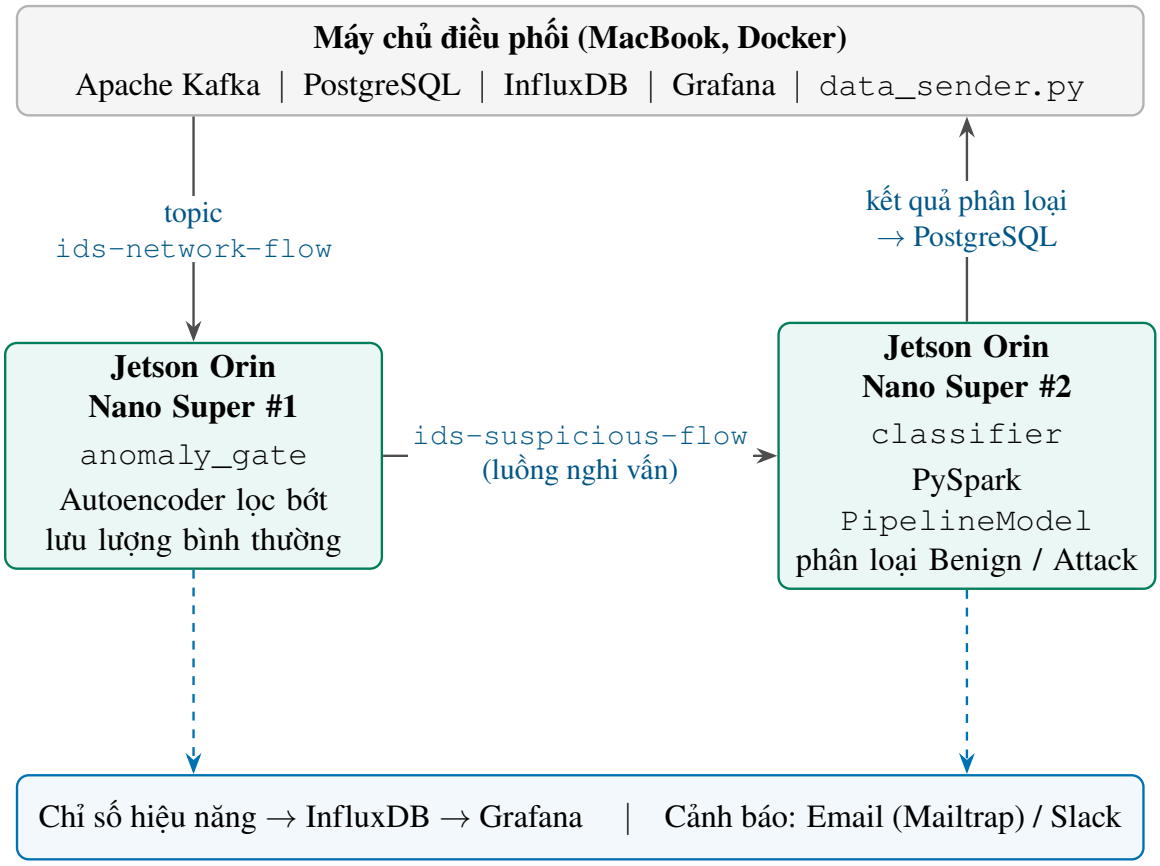
- **Grafana** – Là nền tảng trực quan hóa dữ liệu mã nguồn mở, cho phép xây dựng các dashboard theo thời gian thực. Grafana kết nối với InfluxDB và PostgreSQL để hiển thị tình trạng hoạt động của hệ thống IDS cũng như thống kê các sự kiện tấn công.

- **Mailtrap** – Là công cụ kiểm thử email trong môi trường giả lập. Trong giai đoạn thực nghiệm, Mailtrap được sử dụng để kiểm tra cơ chế gửi cảnh báo khi hệ thống phát hiện lưu lượng mạng bất thường trước khi triển khai môi trường thực tế.

Các công nghệ trên bao phủ đủ bốn nhu cầu vận hành của hệ thống: truyền tải luồng, lưu trữ, giám sát hiệu năng và cảnh báo.

### 4.3.2 Mô hình hệ thống đề xuất

Hệ thống phát hiện xâm nhập được thiết kế theo kiến trúc xử lý dữ liệu luồng như minh họa ở Hình 4.6. Luồng xử lý tổng thể bao gồm các bước sau:




Hình 4.6. Kiến trúc hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm 2 Jetson Orin Nano Super

1. **Nguồn dữ liệu**: Dữ liệu từ bộ CICIDS2017 được mô phỏng theo thời gian thực thông qua một Python script và gửi vào Kafka Producer trên topic `ids-network-flow`.

2. **Cổng lọc bất thường (Jetson #1)**: Jetson #1 đảm nhận vai trò `anomaly_gate`: một autoencoder nhẹ (huấn luyện chỉ trên lưu lượng bình thường, trên toàn bộ 77 đặc trưng số không rò rỉ) tính sai số tái tạo (reconstruction error) cho từng luồng. Các luồng có sai số dưới ngưỡng được coi là bình thường và bị lọc bỏ. Chỉ các luồng nghi vấn mới được chuyển tiếp sang topic `ids-suspicious-flow`. Cơ chế này giảm đáng kể khối lượng phải đưa vào suy luận Spark.

3. **Bộ phân loại (Jetson #2)**: Jetson #2 đảm nhận vai trò `classifier`: tải mô hình PySpark `PipelineModel` (với tập đặc trưng SHAP Top-30 đã chọn) và phân loại các luồng nghi vấn thành *Benign* hoặc *Attack*. Vector đầu vào khi triển khai gồm đúng 30 đặc trưng SHAP Top-30 của pha huấn luyện loại trừ rò rỉ: các cột cổng (`destination_port`, `source_port`) đã bị loại trước khi xếp hạng (Mục 4.2.5). Random Forest triển khai (200 cây, độ sâu 15, huấn luyện trên toàn bộ tập huấn luyện) đạt F1 0,9959 trên tập kiểm tra, khớp với kết quả 0,9955 ở pha huấn luyện phân tán (Bảng 4.3).

4. **Gom lô có giới hạn thời gian**: Mỗi tầng xử lý theo lô tối đa 20 luồng, nhưng một lô chưa đầy cũng được xử lý sau tối đa 1 giây. Nếu không có giới hạn này, bộ phân loại (chỉ nhận khoảng 11% lưu lượng sau cổng lọc) có thể giữ một lô dở dang chứa luồng tấn công trong thời gian không xác định.

5. **Ghi nhận hiệu năng**: Mỗi nút ghi nhận thời gian dự đoán/mẫu, mức sử dụng CPU/RAM/nhiệt độ và thông lượng (gắn nhãn theo `EDGE_NODE_ID`) và lưu các chỉ số này vào InfluxDB.

6. **Lưu trữ và cảnh báo**: Kết quả dự đoán và sự kiện tấn công được lưu vào PostgreSQL. Khi phát hiện lưu lượng bất thường, hệ thống kích hoạt cảnh báo qua Email/Webhook (Mailtrap, Slack trong môi trường kiểm thử) từ một nút chính được chỉ định để tránh trùng lặp.

7. **Trực quan hóa và giám sát**: Grafana truy xuất dữ liệu từ InfluxDB và PostgreSQL để xây dựng dashboard theo thời gian thực, giám sát trạng thái hệ thống và thống kê các sự kiện tấn công.

**Ba chế độ triển khai phân tán.** Cụm hai Jetson hỗ trợ ba chế độ, lựa chọn theo ràng buộc tải và độ trễ:

- **Chế độ A – Pipeline split** (khuyến nghị): Jetson #1 = cổng lọc bất thường (anomaly gate), Jetson #2 = bộ phân loại (classifier) (như mô tả trên), giúp giảm tải Spark nhờ lọc lưu lượng bình thường sớm.
- **Chế độ B – Mở rộng theo chiều ngang (Horizontal scaling)**: cả hai Jetson chạy pipeline đầy đủ trong cùng một nhóm tiêu thụ (consumer group) của Kafka nhằm tăng thông lượng.
- **Chế độ C – Spark cluster**: máy chủ MacBook làm Spark master, hai Jetson làm worker, phân tán suy luận Spark trên cụm. Nhất quán với nguyên tắc ở mục Môi trường thực nghiệm, MacBook chỉ đảm nhận điều phối (master), không chạy executor. Toàn bộ tính toán suy luận vẫn nằm trên hai Jetson.

Kiến trúc này tách các thành phần xử lý thành những mô-đun độc lập, phù hợp với triển khai phân tán trên thiết bị biên tài nguyên hạn chế như Jetson Orin Nano Super.

**Điểm vận hành của cổng lọc bất thường.** Cổng lọc autoencoder là một bộ lọc điều chỉnh được chứ không phải một điểm vận hành cố định. Nâng ngưỡng thì cổng lọc bỏ qua nhiều lưu lượng bình thường hơn, giảm tải cho Spark, nhưng cũng có thể bỏ qua nhiều luồng tấn công hơn. Recall của cả hệ thống bị chặn trên bởi recall của cổng lọc, vì luồng tấn công bị cổng lọc bỏ qua không bao giờ đến bộ phân loại.

Ngưỡng được chọn theo phân vị (quantile) của sai số tái tạo, tính trên một tập kiểm định benign gồm 10% tách từ phần lưu lượng bình thường của tập huấn luyện (`seed`=42). Tập kiểm tra không tham gia việc chọn ngưỡng, tránh rò rỉ kiểu "ngưỡng oracle". Bảng 4.15 trình bày đường đánh đổi giữa recall và mức giảm tải khi quét các mức quantile. Điểm vận hành được chọn theo ngân sách độ trễ và độ chính xác của triển khai. Tỷ lệ bỏ qua và recall trong bảng được tính trên tập kiểm tra đã loại trùng lặp ở mức đặc trưng (380.119 benign / 67.156 attack). FPR trong bảng là tỷ lệ luồng benign bị cổng lọc chuyển nhầm sang bộ phân loại, không phải FPR của toàn hệ thống.

Hệ thống triển khai dùng quantile 0,995, cố định trước khi đo. Ở điểm này cổng lọc chuyển tiếp 68,1% luồng tấn công sang bộ phân loại, chỉ chuyển nhầm 0,49% luồng benign và bỏ qua 89,4% lưu lượng. Đây là cái giá của việc giảm tải, và là lý do đường đánh đổi được báo cáo đầy đủ: hạ quantile xuống 0,90 nâng recall của cổng lọc lên 84,5% nhưng chuyển nhầm 10% luồng benign sang bộ phân loại.

Luồng phát lại khi triển khai là 10.000 luồng lấy ngẫu nhiên từ chính tập kiểm tra này (`seed`=42, 15,2% tấn công). Vì vậy các con số thực đo (bỏ qua 88,8%, recall tấn công 70,0% ở chế độ A, Bảng 4.21) khớp với điểm vận hành của bảng.

Bảng 4.15. Điểm vận hành của cổng lọc bất thường theo ngưỡng quantile trên tập kiểm định benign (dòng in đậm: điểm triển khai)

| Quantile (validation) | Ngưỡng | Recall (Attack) | FPR | Bỏ qua tại cổng lọc (%) |
|---|---|---|---|---|
| 0,90 | 0,0145 | 0,8449 | 0,1009 | 78,74 |
| 0,95 | 0,0317 | 0,7571 | 0,0503 | 84,36 |
| 0,975 | 0,0607 | 0,7506 | 0,0249 | 86,62 |
| 0,99 | 0,1219 | 0,7235 | 0,0099 | 88,29 |
| **0,995** | **0,2021** | **0,6807** | **0,0049** | **89,37** |
| 0,999 | 0,5917 | 0,3981 | 0,0009 | 93,95 |

### 4.3.3 Lựa chọn mô hình cho thiết bị biên

Triển khai trên thiết bị biên đặt ra các ràng buộc về tài nguyên: RAM 8 GB chia sẻ giữa CPU và GPU (trong đó PySpark JVM chiếm một phần đáng kể), CPU ARM, cùng ràng buộc về nhiệt độ/băng thông tại biên. Việc chọn Apache Spark trên Jetson 8 GB phải chịu chi phí JVM/Spark đáng kể. Spark được dùng ở đây nhằm nhất quán với pha huấn luyện phân tán, chứ không nhằm tối ưu độ trễ suy luận trên một nút. Chi phí đánh đổi này được định lượng bằng cách chạy cùng mô hình qua NumPy và ONNX Runtime ở mục 4.3.7, còn tăng tốc bằng TensorRT là hướng nghiên cứu tương lai. Ba mô hình đại diện cho ba thử nghiệm khác nhau được chọn để đánh giá:

1. **Decision Tree** – Mô hình cây đơn lẻ, đại diện cho cách tiếp cận đơn giản nhất.
2. **GBT (Gradient-Boosted Trees)** – Đại diện cho phương pháp Boosting, sử dụng nhiều cây yếu kết hợp tuần tự.
3. **Random Forest** – Đại diện cho phương pháp Bagging, sử dụng ensemble 200 cây song song.

XGB đạt F1 ngoại tuyến cao nhất trên SHAP Top-30 (0,9970, Bảng 4.3, so với 0,9955 của Random Forest, Bảng 4.4) nhưng không được đưa vào nhóm ứng viên. Spark ML không có sẵn XGB: tích hợp XGB của Spark là một thư viện ngoài, cần thêm thư viện native khi nạp mô hình, trong khi ba mô hình trên là estimator gốc của Spark ML nên

`PipelineModel` xuất ra nạp được thẳng trên bo mạch ARM64. Mức chênh F1 khoảng 0,0015 không đủ bù cho rủi ro triển khai đó.

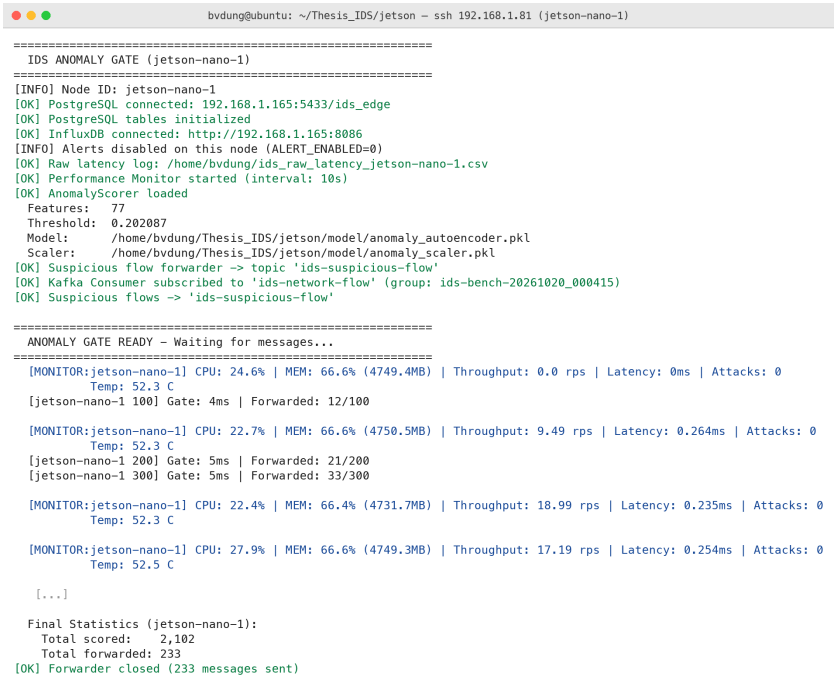
Cả ba mô hình đều được huấn luyện trên một danh sách SHAP Top-30 và triển khai dưới dạng PySpark PipelineModel. Lưu ý về phạm vi: phép so sánh ba ứng viên (Bảng 4.16 và 4.17) được thực hiện ở giai đoạn đầu, với danh sách đặc trưng xếp hạng *trước* khi áp dụng quy trình loại trừ rò rỉ (29 đặc trưng, chỉ trùng 15 với SHAP Top-30 hiện tại). Kết quả so sánh tương đối giữa ba mô hình vẫn là căn cứ chọn Random Forest, nhưng mô hình thực sự triển khai đã được huấn luyện lại trên SHAP Top-30 loại trừ rò rỉ (F1 0,9959), và mọi số đo hệ thống từ Mục 4.3.7 trở đi là của mô hình huấn luyện lại này.

Hình 4.7 là nhật ký terminal của hai nút trong một lượt phát lại 20 luồng/giây với mô hình đang triển khai. Cổng lọc trên Jetson #1 chấm điểm khoảng 19–21 luồng/giây và chỉ chuyển tiếp 233 trên 2.102 luồng. Bộ phân loại PySpark trên Jetson #2 xử lý các luồng được chuyển tiếp theo từng lô. Cứ 10 giây, mỗi nút in một dòng MONITOR gồm CPU, bộ nhớ, nhiệt độ, độ trễ và số tấn công. Hai dashboard Grafana (Hình 4.8 và Hình 4.9) cho phép giám sát hiệu năng hệ thống và chi tiết các cuộc tấn công theo thời gian thực. Hình 4.10 ghi lại thời điểm tải đỉnh, cho thấy rõ sự phân bố tải lệch đặc trưng của chế độ tách pipeline: nút cổng lọc chỉ sử dụng khoảng 6% CPU trong khi nút phân loại PySpark đạt tới 94%, xác nhận bộ phân loại là thành phần chiếm tài nguyên chủ đạo. Các ảnh Grafana được chụp ở một lượt chạy trước với mô hình cũ. Chúng minh họa cách hệ thống vận hành và giám sát, còn các con số hiệu năng chính thức là số đo lại ở Bảng 4.18 và 4.21. Khi phát hiện tấn công, hệ thống tự động gửi cảnh báo qua nhiều kênh: email thông qua Mailtrap SMTP (Hình 4.11) và tin nhắn Slack (Hình 4.12). Hai ảnh cảnh báo này chụp từ giai đoạn thử nghiệm sớm của hệ thống. Mô-đun cảnh báo và định dạng thông điệp giữ nguyên khi triển khai trên cụm Jetson.

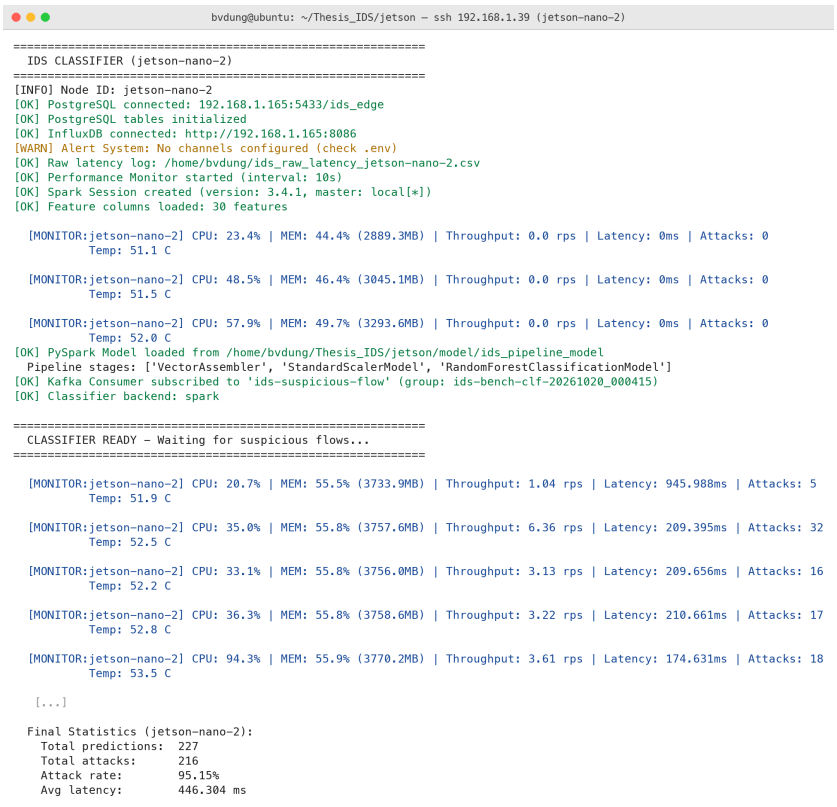
### 4.3.4 Kết quả huấn luyện các mô hình ứng viên trên cụm Spark

Bảng 4.16 trình bày kết quả huấn luyện và đánh giá ba mô hình ứng viên trên cụm Spark (mô tả ở mục Môi trường thực nghiệm) trước khi triển khai xuống thiết bị biên.

Về tiêu chí nhận biết ràng buộc biên (edge-aware), ba mô hình ứng viên đánh đổi khác nhau: Decision Tree có kích thước nhỏ nhất (0,046 MB, nhỏ hơn Random Forest khoảng 96 lần) và huấn luyện nhanh hơn khoảng 28 lần, trong khi Random Forest đạt F1 cao nhất. Riêng F1/Accuracy chưa đủ để kết luận: quyết định cuối cùng dựa trên số đo vận hành thực tế trên thiết bị (thông lượng, độ trễ, CPU, RAM) ở mục tiếp theo. (Các mô hình ứng viên

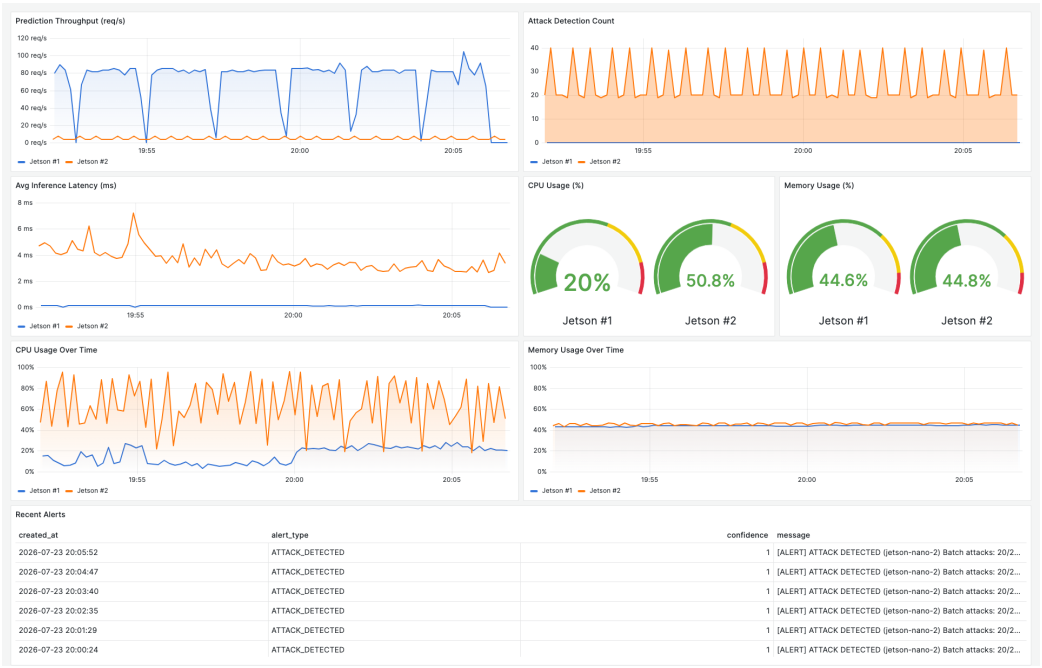



(a) Jetson #1 (`jetson-nano-1`): cổng lọc bất thường, khoảng 19–21 luồng/giây, chuyển tiếp 233 trên 2.102 luồng

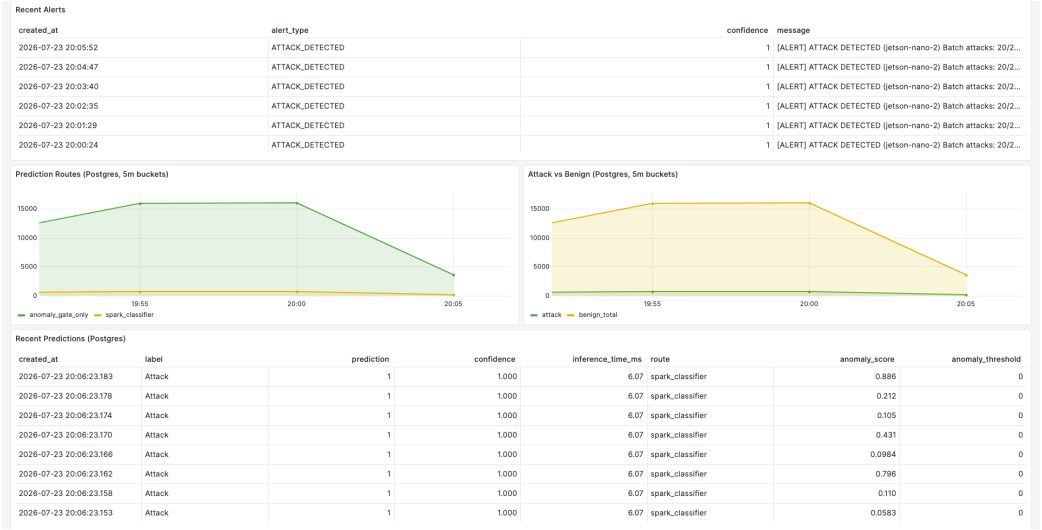



(b) Jetson #2 (`jetson-nano-2`): bộ phân loại PySpark (Random Forest, 30 đặc trưng) xử lý các luồng được chuyển tiếp

Hình 4.7. Nhật ký terminal của pipeline IDS trên hai nút Jetson trong một lượt phát lại 20 luồng/giây ở chế độ pipeline split: (a) cổng lọc bất thường, (b) bộ phân loại PySpark. Dấu [...] đánh dấu các dòng được lược bớt



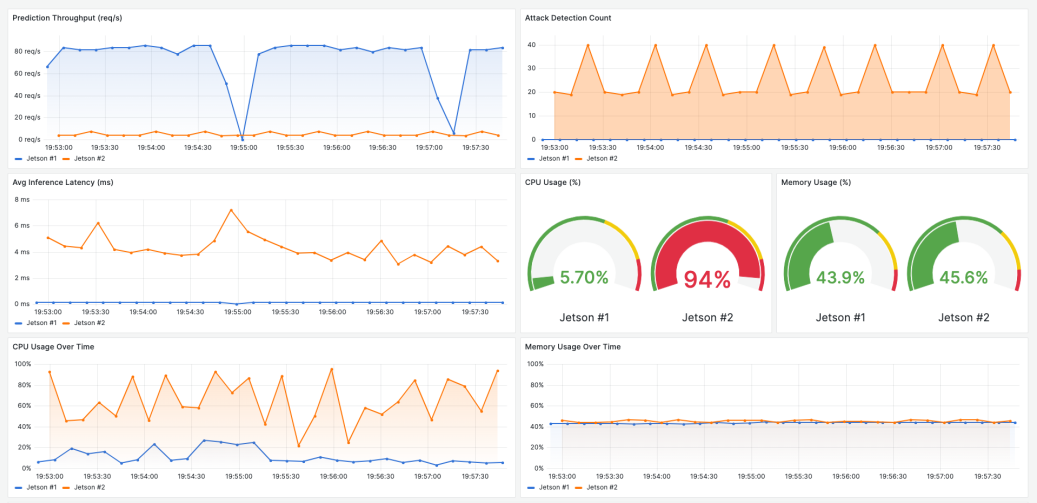

Hình 4.8. Dashboard Grafana giám sát hiệu năng IDS theo thời gian thực



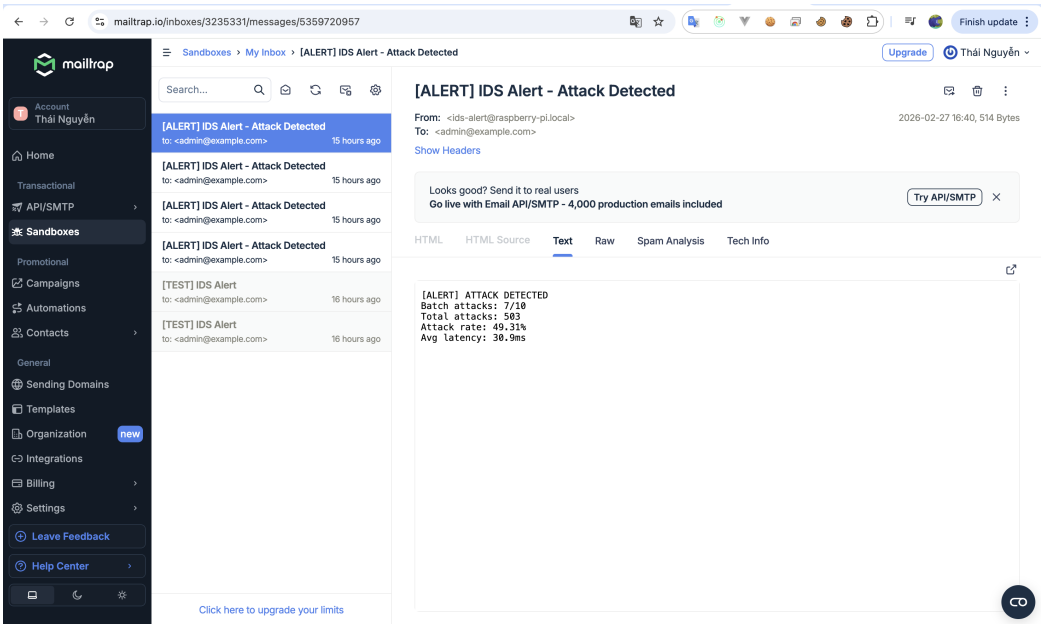

Hình 4.9. Dashboard Grafana hiển thị chi tiết các cuộc tấn công được phát hiện và phân loại theo thời gian

Bảng 4.16. Kết quả huấn luyện các mô hình ứng viên cho triển khai biên (giai đoạn chọn mô hình, danh sách đặc trưng trước khi loại trừ rò rỉ)

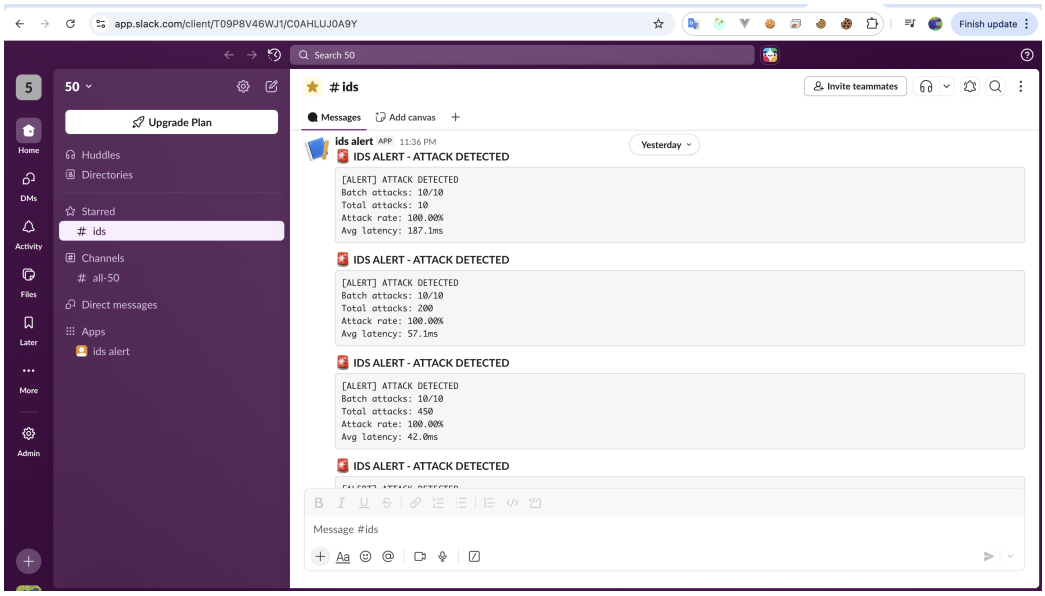
| **Mô hình** | **F1-Score** | **Accuracy** | **Thời gian huấn luyện (s)** | **Kích thước (MB)** |
|---|---|---|---|---|
| Decision Tree | 0,9814 | 0,9943 | **24,4** | **0,046** |
| GBT | 0,9753 | 0,9925 | 341,8 | 0,865 |
| Random Forest | **0,9826** | **0,9947** | 687,3 | 4,427 |




Hình 4.10. Dashboard Grafana tại thời điểm tải đỉnh của chế độ tách pipeline




Hình 4.11. Email cảnh báo tấn công gửi qua Mailtrap trong môi trường kiểm thử




Hình 4.12. Thông báo cảnh báo tấn công gửi qua Slack Incoming Webhook

biên được huấn luyện trên mẫu con phân tầng khi xuất để vừa bộ nhớ một nút, nên F1 thấp hơn chút ít so với kết quả huấn luyện phân tán đầy đủ ở Bảng 4.3. Độ chính xác chính thức lấy từ pha huấn luyện đầy đủ.)

### 4.3.5 Kết quả đo hiệu năng (benchmark) trên Jetson Orin Nano Super

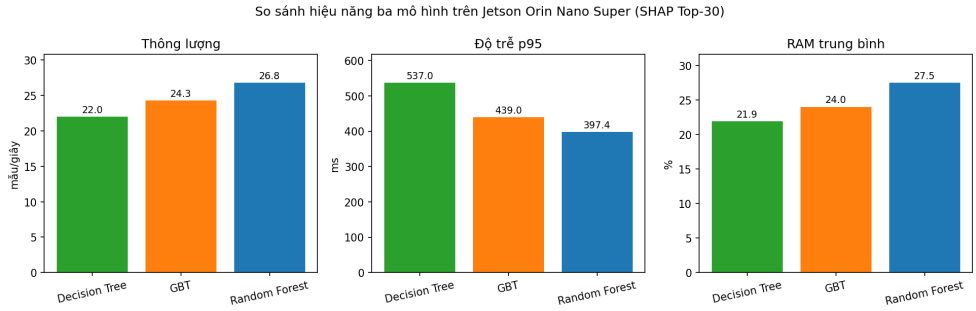
Bảng 4.17 trình bày kết quả đo hiệu năng (benchmark) thực tế trên Jetson Orin Nano Super. Mỗi cấu hình được đo lặp lại 5 lần. Trước mỗi cửa sổ đo có giai đoạn khởi động (warm-up) bằng chính lưu lượng thật để làm nóng JVM/JIT, và dữ liệu warm-up bị loại khỏi cửa sổ đo. Kết quả báo cáo là trung bình qua các lần chạy. Hai loại độ trễ được ghi nhận: (i) *độ trễ suy luận* (tiền xử lý + dự đoán) trên từng nút, và (ii) *độ trễ đầu cuối* (từ thời điểm producer gửi đến khi có kết quả phân loại) tính từ nhãn thời gian của producer. Giá trị tuyệt đối của loại (ii) chỉ có ý nghĩa khi các máy đồng bộ đồng hồ NTP. Các phân vị độ trễ (p50/p95/p99) được tính ở mức từng lần chạy từ log thô theo từng luồng trong đúng cửa sổ tải. Với cụm nhiều nút, p95 báo cáo là p95 của nút xấu nhất (không lấy phân vị của các giá trị trung bình theo nút).

Bảng 4.17. So sánh hiệu năng ba mô hình ứng viên trên Jetson Orin Nano Super (giai đoạn chọn mô hình, độ trễ suy luận theo lô, không gồm thời gian chờ hàng đợi)

| **Mô hình** | **Thông lượng (mẫu/giây)** | **Độ trễ suy luận (ms) p50 / p95 / p99** | **CPU (%)** | **RAM (%)** | **Kích thước (MB)** |
|---|---|---|---|---|---|
| Decision Tree | 22,0 | 441 / 537 / – | 46,4 | **21,9** | **0,046** |
| GBT | 24,3 | 414 / 439 / – | 40,9 | 24,0 | 0,865 |
| Random Forest | **26,8** | **378 / 397 / –** | **35,2** | 27,5 | 4,427 |

### 4.3.6 Phân tích và lựa chọn mô hình

Hình 4.13 minh họa trực quan sự khác biệt giữa ba mô hình về thông lượng, độ trễ và mức sử dụng RAM trên Jetson Orin Nano Super.




Hình 4.13. So sánh hiệu năng ba mô hình trên Jetson Orin Nano Super

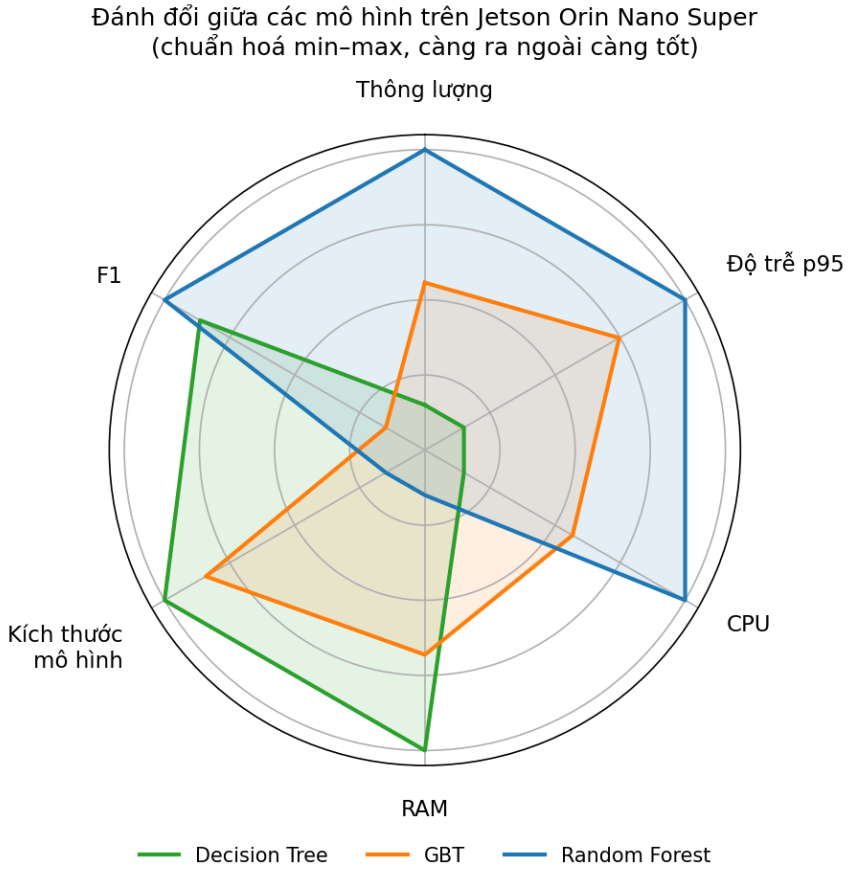
Kết quả benchmark cho thấy sự đánh đổi rõ ràng giữa các chỉ số:

- **Random Forest** thắng ở cả ba chỉ số vận hành quan trọng nhất đối với phát hiện xâm nhập thời gian thực: thông lượng cao nhất (26,8 mẫu/giây), độ trễ p95 thấp nhất (397 ms) và CPU thấp nhất (35,2%), nhờ Spark song song hóa tốt trên nhiều cây. Cái giá phải trả là RAM cao nhất (27,5%) và mô hình lớn nhất (≈4,4 MB).
- **GBT** có hiệu năng trung bình ở tất cả các chỉ số, đóng vai trò cầu nối giữa Decision Tree và Random Forest.
- **Decision Tree** nhẹ nhất về bộ nhớ (RAM 21,9%, mô hình ≈0,05 MB) nhưng chậm nhất: thông lượng 22,0 mẫu/giây và độ trễ p95 537 ms, cao hơn Random Forest khoảng 35%.

Hình 4.14 thể hiện sự đánh đổi đa chiều giữa các tiêu chí lựa chọn mô hình: Decision Tree chiếm ưu thế về bộ nhớ, còn Random Forest chiếm ưu thế về tốc độ xử lý và tải CPU. Thông lượng, độ trễ p95, CPU và RAM lấy từ Bảng 4.17, F1 và kích thước lấy từ Bảng 4.16. Mỗi trục được chuẩn hóa min–max giữa ba mô hình, nên khoảng cách trên hình chỉ cho biết thứ hạng chứ không cho biết độ lớn chênh lệch: chẳng hạn F1 của ba mô hình chỉ chênh nhau 0,007.

**Random Forest được chọn** làm mô hình triển khai chính trên thiết bị biên dựa trên ba tiêu chí:

1. **Hiệu năng xử lý thời gian thực**: thông lượng cao nhất (26,8 mẫu/giây) và độ trễ p95 thấp nhất (397 ms). Với IDS, độ trễ dự đoán là ràng buộc trực tiếp: một mô hình nhỏ nhưng chậm hơn 35% về p95 sẽ kéo dài cửa sổ mà tấn công chưa bị chặn.




Hình 4.14. Đánh đổi giữa ba mô hình ứng viên trên Jetson Orin Nano Super

2. **Chất lượng phát hiện**: cao nhất trong ba mô hình ứng viên ở cả hai phép đo: F1 0,9955 trên đánh giá ngoại tuyến đầy đủ (Bảng 4.4) và 0,9826 ở bản xuất ứng viên (Bảng 4.16). Bản huấn luyện lại trên SHAP Top-30 loại trừ rò rỉ để triển khai đạt F1 0,9959.

3. **Tương thích nền tảng**: Random Forest là estimator gốc của Spark ML. Nhờ vậy `PipelineModel` xuất ra nạp được trực tiếp trên bo mạch ARM64, không cần các thư viện native mà tích hợp XGB của Spark đòi hỏi.

Đánh đổi được chấp nhận là mức tiêu thụ bộ nhớ: Random Forest dùng 27,5% RAM so với 21,9% của Decision Tree và mô hình nặng ≈4,4 MB so với 0,05 MB. Trên bo mạch 8 GB, khoảng chênh này vẫn còn dư địa lớn, nên bộ nhớ không phải là ràng buộc quyết định. Decision Tree vẫn là phương án dự phòng hợp lý nếu triển khai xuống các thiết bị IoT eo hẹp bộ nhớ hơn.

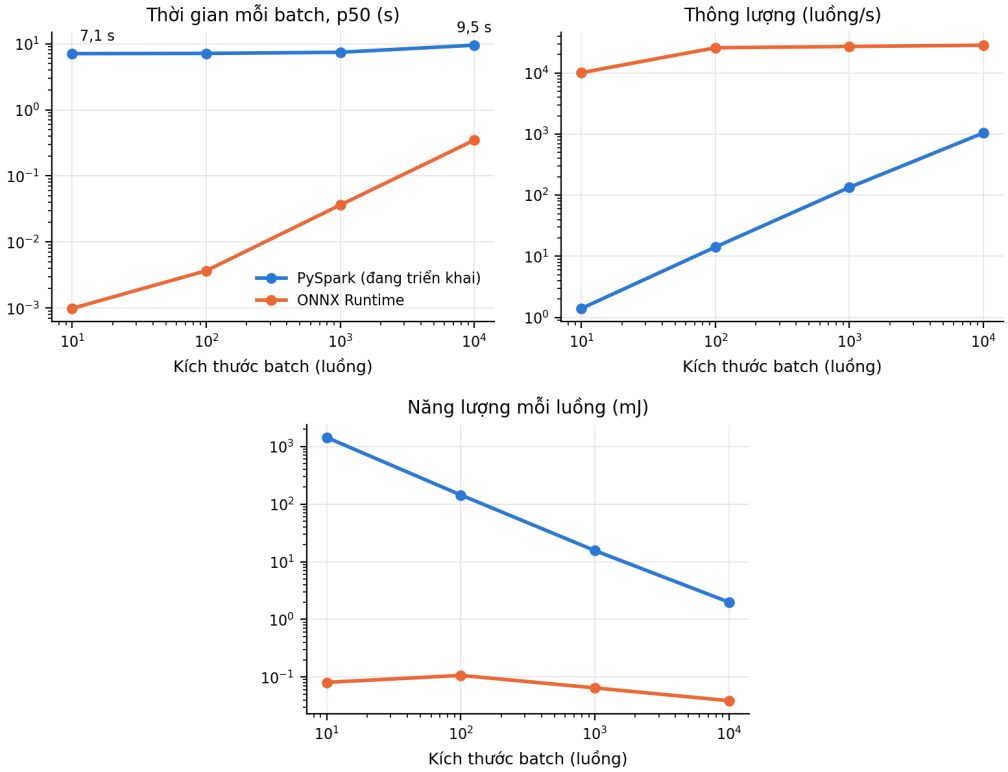
### 4.3.7 So sánh engine suy luận: chi phí của việc đồng nhất stack

Luận văn triển khai mô hình dưới dạng `PipelineModel` của Spark để đồng nhất với pha huấn luyện phân tán: cùng một stack vừa huấn luyện trên cụm vừa suy luận trên thiết bị. Để định lượng chi phí của sự đồng nhất này, luận văn chạy *cùng một* rừng ngẫu nhiên đã triển khai (cùng 200 cây, cùng 30 đặc trưng) qua ba engine suy luận của chính hệ thống biên: PySpark (bản triển khai), NumPy (duyệt cây trực tiếp trên mảng, không JVM) và ONNX Runtime (CPU execution provider). Hai bản NumPy và ONNX được xuất từ chính mô hình Spark, và cả ba engine cho dự đoán trùng nhau 100% trên 1.000 luồng phát lại, nên độ chính xác là như nhau và phép so sánh chỉ đo chi phí suy luận. Phép đo chạy trên nút phân loại với lô 10 luồng (`benchmark_same_forest.py`). Năng lượng báo cáo là phần hoạt động, đã trừ công suất nghỉ. Kết quả (Bảng 4.18) cho thấy chi phí JVM/Spark rất lớn trên bo mạch ARM 8 GB: một lô 10 luồng qua PySpark mất vài giây (p95 7,5 s), chủ yếu do chi phí cố định của mỗi lần gọi Spark chứ không phải do tính toán cây. Cùng mô hình qua ONNX Runtime đạt ≈10.000 luồng/giây với p95 1,2 ms, tức gấp ≈7.200 lần PySpark, ở mức năng lượng mỗi luồng thấp hơn khoảng bốn bậc độ lớn. Khoảng cách này giải thích vì sao thông lượng của mọi chế độ triển khai ở mục sau bị giới hạn bởi bộ phân loại PySpark, và là cơ sở cho hướng phát triển xuất mô hình sang ONNX/TensorRT để suy luận trong khi vẫn giữ Spark cho huấn luyện.

Bảng 4.18. Chi phí suy luận một nút của cùng một rừng ngẫu nhiên qua ba engine (Jetson #2, lô 10 luồng, dự đoán trùng nhau 100%)

| **Engine** | **Thông lượng (luồng/giây)** | **Độ trễ một lô p95 (ms)** | **Năng lượng hoạt động /luồng (mJ)** |
|---|---|---|---|
| PySpark (triển khai) | 1,4 | 7515 | 1506 |
| NumPy | 54,8 | 194 | 11,3 |
| ONNX Runtime | **10.092** | **1,2** | **0,087** |

**Ảnh hưởng của kích thước lô.** Để kiểm chứng rằng chi phí của PySpark nằm ở mỗi lần gọi chứ không ở việc duyệt cây, cùng mô hình được chạy lại với lô 10, 100, 1.000 và 10.000 luồng (Bảng 4.19, Hình 4.15). Thời gian một lô qua PySpark gần như không đổi: 7,1 s ở lô 10, 7,4 s ở lô 1.000, và chỉ tăng lên 9,5 s ở lô 10.000. Vì vậy thông lượng PySpark tăng gần đúng tỷ lệ với kích thước lô, từ 1,4 lên 1.046 luồng/giây, trong khi công suất hoạt động giữ ở 2,0–2,1 W. ONNX Runtime bão hòa quanh 26.000–28.600 luồng/giây từ lô 100 trở lên. Do đó tỷ lệ ≈7.200 lần nêu trên gắn với lô 10, đúng cấu hình đang triển khai. Ở lô 10.000 khoảng cách chỉ còn khoảng 27 lần. Hệ thống phát hiện xâm nhập dạng luồng lại



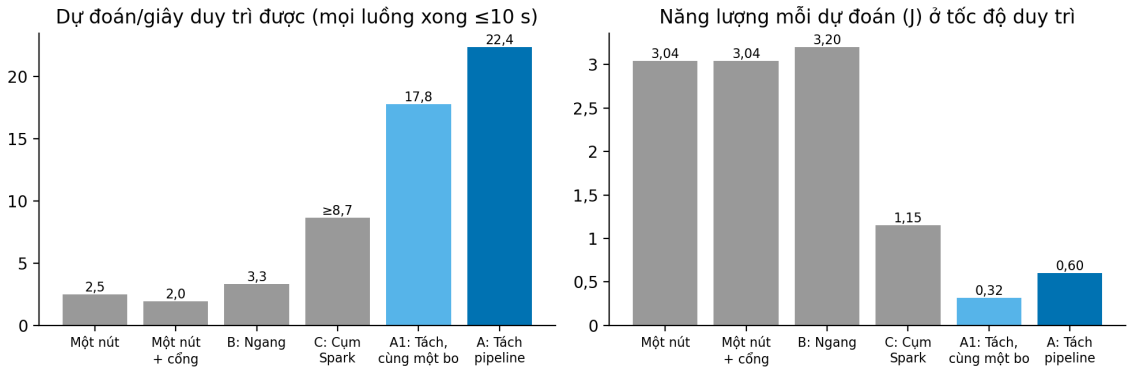

Hình 4.15. Thời gian mỗi lô (p50), thông lượng và năng lượng hoạt động mỗi luồng của cùng một rừng ngẫu nhiên qua PySpark và ONNX Runtime theo kích thước lô (Jetson #2, mỗi điểm một lượt đo, dự đoán trùng nhau 100%, trục log)

không thể dùng lô lớn để bù: ở 20 luồng/giây, gom đủ 10.000 luồng mất hơn 8 phút. Năng lượng của ONNX dao động giữa các lượt đo đơn (0,04–0,11 mJ/luồng) và không giảm đều theo kích thước lô, nên chỉ bậc độ lớn của nó có ý nghĩa.

Bảng 4.19. Chi phí suy luận của cùng một rừng ngẫu nhiên qua PySpark và ONNX Runtime theo kích thước lô (Jetson #2, mỗi điểm một lượt đo, dự đoán trùng nhau 100%)

| | PySpark (đang triển khai) | | | ONNX Runtime | | |
|---|---|---|---|---|---|---|
| **Lô (luồng)** | **p50 mỗi lô (s)** | **Thông lượng (luồng/giây)** | **Năng lượng (mJ/luồng)** | **p50 mỗi lô (ms)** | **Thông lượng (luồng/giây)** | **Năng lượng (mJ/luồng)** |
| 10 | 7,09 | 1,4 | 1.429 | 0,99 | 10.160 | 0,081 |
| 100 | 7,14 | 14,2 | 144 | 3,68 | 25.987 | 0,107 |
| 1.000 | 7,43 | 134,5 | 15,6 | 36,3 | 27.281 | 0,065 |
| 10.000 | 9,50 | 1.046 | 2,0 | 347 | 28.589 | 0,039 |

Thông lượng gồm cả việc dựng ma trận đặc trưng từ bản ghi Python, như trong consumer biên. Năng lượng là phần công suất hoạt động (đã trừ công suất nghỉ) chia cho số luồng. Với ONNX, công suất hoạt động dao động giữa các lượt đo đơn nên chỉ bậc độ lớn có ý nghĩa.




Hình 4.16. Thông lượng duy trì (trái) và năng lượng mỗi dự đoán ở tốc độ duy trì (phải) của sáu cấu hình triển khai trên cụm 2 Jetson. Cột của cụm Spark là cận dưới, cột một nút + cổng lọc xem chú thích Bảng 4.21

### 4.3.8 Kết quả ba chế độ triển khai phân tán

Ngoài việc so sánh engine trên một nút, luận văn đánh giá ba chế độ triển khai phân tán trên cụm hai Jetson Orin Nano Super, cùng hai cấu hình một nút làm cơ sở: một nút chạy toàn bộ pipeline không có cổng lọc, và một nút chạy cả cổng lọc lẫn bộ phân loại (Bảng 4.21). Ở chế độ B (mở rộng theo chiều ngang), mỗi nút chạy vai trò `full` không có cổng lọc, đúng như cấu hình đã dùng trong các lượt đo trước.

**Phương pháp đo có tính hàng đợi.** Mỗi luồng phát lại mang theo nhãn thời gian gửi và số thứ tự dòng, và hai giá trị này được lưu cùng dự đoán trong PostgreSQL. Nhờ đó, phép đo theo dõi *mọi luồng được gửi* trong cửa sổ tải đến khi nó có kết quả, kể cả khi kết quả đến sau khi cửa sổ tải đã kết thúc. Độ trễ đầu cuối tính từ lúc gửi đến lúc có kết quả phân loại, nên đã bao gồm thời gian chờ trong hàng đợi Kafka và thời gian chờ gom lô. Tỷ lệ bỏ qua tại cổng lọc và recall tấn công được tính trực tiếp trên các luồng đó. Mỗi lượt chạy dùng một nhóm tiêu thụ Kafka riêng để lượt sau không xử lý phần tồn đọng của lượt trước. Với mỗi chế độ, tốc độ gửi được nâng dần (Bảng 4.20). Một tốc độ gửi được coi là *theo kịp* khi ở cả 2 lần lặp mọi luồng đều có kết quả trong vòng 10 giây sau cửa sổ tải (cửa sổ tải 45 giây sau 15 giây khởi động). *Thông lượng duy trì* là tốc độ dự đoán ở tốc độ gửi cao nhất còn theo kịp.

Kết quả (Bảng 4.21, Hình 4.16) cho thấy:

- **Thông lượng**: chế độ *pipeline split* duy trì được 22,4 luồng/giây, gấp khoảng 9 lần một nút chạy toàn bộ pipeline (2,5 luồng/giây). Mức tăng đến từ việc tách hai tầng thành hai tiến trình độc lập, không phải từ riêng cổng lọc hay từ bo mạch thứ hai. Khi cổng lọc chạy chung tiến trình với bộ phân loại, một nút không nhanh hơn chút

Bảng 4.20. Kiểm tra chịu tải của từng chế độ triển khai trên cụm 2 Jetson Orin Nano Super theo tốc độ gửi tăng dần: mỗi mức chạy 2 lần lặp, một mức đạt khi cả 2 lần mọi luồng đều có kết quả trong vòng 10 giây sau cửa sổ tải

| Chế độ | Tốc độ gửi (luồng/giây) | Thông lượng (luồng/giây) | Độ trễ p50 (s) | Độ trễ p95 (s) | Xả hàng đợi lần 1 / lần 2 (s) | Quy tắc 10 giây |
|---|---|---|---|---|---|---|
| Một nút | 1 | 0,9 | 4,7 | 8,2 | 7,6 / 5,8 | Đạt |
| | 2 | 1,8 | 4,3 | 8,1 | 6,5 / 0,4 | Đạt |
| | 3 | 2,5 | 5,9 | 11,3 | 7,7 / 8,6 | Đạt |
| Một nút + cổng lọc | 2 | 2,0 | 3,8 | 7,8 | 1,2 / 0,0 | Đạt |
| | 3 | 2,4 | 6,3 | 11,7 | 10,6 / 8,7 | Không |
| | 5 | 2,7 | 23,1 | 36,7 | 42,6 / 23,7* | Không |
| B: Horizontal | 2 | 1,7 | 4,9 | 9,1 | 8,1 / 6,5 | Đạt |
| | 4 | 3,3 | 4,1 | 8,2 | 7,6 / 7,9 | Đạt |
| | 6 | 4,8 | 7,1 | 11,7 | 10,5 / 9,4 | Không |
| C: Spark cluster | 10 | 8,7 | 0,7 | 5,2 | 5,6 / 1,8 | Đạt |
| | 20 | 15,8 | 0,7 | 10,3 | 12,3 / 3,4 | Không |
| A: Pipeline split | 5 | 4,2 | 1,1 | 5,1 | 7,1 / 4,4 | Đạt |
| | 10 | 9,2 | 1,2 | 4,4 | 1,4 / 1,5 | Đạt |
| | 20 | 15,8 | 1,1 | 5,9 | 7,0 / 7,5 | Đạt |
| | 30 | 22,4 | 1,0 | 7,1 | 8,7 / 3,8 | Đạt |
| | 40 | 22,0 | 0,4 | 15,2 | 29,0 / 22,2 | Không |

Thông lượng và độ trễ là trung bình 2 lần lặp. Độ trễ đầu cuối tính từ lúc gửi, gồm thời gian chờ hàng đợi. Xả hàng đợi là thời gian từ khi ngừng gửi đến khi luồng cuối có kết quả. *Lần lặp 2 chỉ hoàn thành 93,2% số luồng. Dòng in ở Bảng 4.21 là mức đạt cao nhất của mỗi chế độ. Với cụm Spark, mức 20 có một lần lặp theo kịp (17,1 luồng/giây) nên thông lượng của nó chỉ biết được cận dưới. Nguồn: hai tệp `capacity_sweep` của lần đo 30/9/2026.

Bảng 4.21. Thông lượng, độ trễ, tỷ lệ bỏ qua và năng lượng của từng chế độ triển khai ở mức tải cao nhất mà hệ thống còn theo kịp (cụm 2 Jetson Orin Nano Super)

| Chế độ | Số bo | Thông lượng duy trì (luồng/giây) | Độ trễ đầu cuối p50 / p95 (s) | Bỏ qua tại cổng lọc (%) | Năng lượng /dự đoán (J) |
|---|---|---|---|---|---|
| Một nút | 1 | 2,5 | 5,9 / 11,3 | – | 3,04 |
| Một nút + cổng lọc | 1 | 2,0* | 3,8 / 7,8 | 89,0 | 3,04 |
| B: Horizontal | 2 | 3,3 | 4,1 / 8,2 | – | 3,20 |
| C: Spark cluster | 2 | $\geq$8,7† | **0,7 / 5,2** | 88,6 | 1,15 |
| A1: Tách, cùng một bo‡ | 1 | 17,8 | 0,6 / 5,3 | 89,0 | **0,32** |
| **A: Pipeline split** | 2 | **22,4** | 1,0 / 7,1 | 88,8 | 0,60 |

Phát lại 10.000 luồng lấy từ tập kiểm tra CICIDS2017 (15,2% tấn công). Thông lượng duy trì và độ trễ là trung bình 2 lần lặp ở tốc độ gửi cao nhất còn theo kịp (Bảng 4.20). Tỷ lệ bỏ qua gộp trên mọi lượt chạy của chế độ. Năng lượng là công suất toàn bo (cộng cả hai bo ở chế độ hai nút) chia cho tốc độ dự đoán, đo ở 2,0 / 2,0 / 3,9 / 9,5 / 18,4 luồng/giây. Recall tấn công thực đo, gộp mọi lượt chạy của chế độ, là 100% khi không có cổng lọc (242 luồng tấn công) và 70,0% ở chế độ A (2.990 luồng tấn công, 15 lượt chạy), khớp với recall 68,1% của cổng lọc ở Bảng 4.15. *Ở tốc độ gửi 3 luồng/giây, một nút có cổng lọc và không cổng lọc đều cho 2,4–2,5 dự đoán/giây. Cấu hình có cổng lọc chỉ bị xếp xuống mức 2 luồng/giây vì một trong hai lần lặp mất 10,6 giây để xử lý hết phần tồn. †Cận dưới: ở mức 20 luồng/giây, một lần lặp theo kịp (17,1 luồng/giây), lần còn lại mất 12,3 giây để xử lý hết phần tồn. ‡Cổng lọc và bộ phân loại là hai tiến trình riêng trên cùng một bo. Chế độ này được đo bổ sung ngày 12/10/2026 với cùng bo mạch, mô hình và tệp phát lại, năng lượng đo ở 17,8 luồng/giây.

nào (2,4–2,5 dự đoán/giây ở tốc độ gửi 3 luồng/giây, như khi không có cổng lọc), vì mỗi lô vẫn phải chờ một lần gọi PySpark. Khi cổng lọc và bộ phân loại chạy thành hai tiến trình riêng trên cùng một bo, hệ thống đã duy trì được 17,8 luồng/giây với 0,32 J mỗi dự đoán (lần đo bổ sung ngày 12/10/2026, cùng bo mạch, mô hình và tệp phát lại). Bo mạch thứ hai chỉ thêm khoảng một phần tư thông lượng, trong khi năng lượng mỗi dự đoán tăng gần gấp đôi. *Mở rộng theo chiều ngang* (không cổng lọc) vẫn bị PySpark giới hạn ở 3,3 luồng/giây. *Spark cluster* duy trì được ít nhất 8,7 luồng/giây. Ở mức 20 luồng/giây, một trong hai lần lặp theo kịp (17,1 luồng/giây) còn lần kia thì không, nên thông lượng của nó nằm trong khoảng 9–17 luồng/giây: vẫn thấp hơn pipeline split, lại phụ thuộc thêm nút chủ trên máy điều phối. Vì vậy pipeline split là cấu hình khuyến nghị.

- **Độ trễ**: ở tốc độ duy trì, pipeline split trả dự đoán trung vị sau khoảng 1 giây, chủ yếu là các luồng được cổng lọc kết luận benign. Các luồng được chuyển sang bộ phân loại phải chờ một lô PySpark, nên p95 trên mọi luồng là 7,1 giây. Mọi chế độ đều có đuôi độ trễ vài giây vì cùng một lý do: một lô PySpark trên bo mạch mất vài giây bất kể kích thước lô (Bảng 4.18). Hệ thống do đó phù hợp với cảnh báo gần thời gian thực (cỡ giây) chứ chưa phải chặn lưu lượng theo từng gói.

- **Năng lượng**: công suất được lấy mẫu bằng `tegrastats` (30 giây không tải, sau đó cửa sổ tải). Ở 18,4 luồng/giây trên hai bo, pipeline split tiêu tốn 0,60 J mỗi dự đoán, so với 3,04 J của một nút ở 2 luồng/giây, tức giảm khoảng 5 lần. Phần công suất hoạt động (đã trừ công suất nghỉ) chỉ tăng 1,4–1,6 W khi có tải ở mọi chế độ, trừ chế độ horizontal (2,6 W, cả hai bo cùng chạy Spark), nên năng lượng mỗi dự đoán chủ yếu phản ánh việc dàn trải công suất bo mạch trên nhiều dự đoán hơn.

- **Chất lượng phát hiện**: không có cổng lọc, bộ phân loại bắt được mọi luồng tấn công trong các lượt đo. Với cổng lọc ở quantile 0,995, recall tấn công thực đo là 70,0% (gộp 15 lượt chạy), đúng như điểm vận hành của cổng lọc. Như vậy thông lượng gấp 9 lần đi kèm việc bỏ sót khoảng 30% luồng tấn công ở ngay tầng cổng lọc, và đây là một lựa chọn ngưỡng có thể điều chỉnh theo Bảng 4.15. Một lượt phát lại trọn 10.000 luồng (mỗi luồng đúng một lần, 20 luồng/giây) cho kết quả đầu cuối: precision 100% (không luồng benign nào bị báo tấn công), recall 69,5% và F1 0,820 (1.055 trên 1.518 luồng tấn công). Cả 463 luồng bị sót đều là luồng cổng lọc đã bỏ qua. Bộ phân loại không phân loại sai luồng nào được chuyển sang.

Bảng 4.22. Chế độ pipeline split, với mô hình huấn luyện trên CICIDS2017, khi phát lại trọn 10.000 luồng của CICIDS2017 và của CSE-CIC-IDS2018

| **Phát lại 10.000 luồng, mỗi luồng một lần** | **CICIDS2017** | **CSE-CIC-IDS2018** |
|---|---|---|
| Luồng cổng lọc chuyển tiếp (luồng benign bị chuyển tiếp) | 11,0% (0,6%) | 30,4% (28,4%) |
| Thông lượng duy trì (luồng/giây) | 22,4 | ≈8* |
| Luồng tấn công / phát hiện được | 1.518 / 1.055 | 1.276 / 10 |
| Bỏ sót: do cổng lọc / do bộ phân loại | 463 / 0 | 712 / 554 |
| Recall / F1 đầu cuối (precision 100%) | 69,5% / 0,820 | 0,8% / 0,016 |
| Bộ phân loại chạy riêng, recall ngoại tuyến | 99,8% | 1,8% |

*Lượt phát lại 1.250 giây ở 8 luồng/giây xử lý hết phần tồn sau 1,9 giây. Ở 12 luồng/giây, cả hai lần lặp đều xả hàng đợi quá hạn (18,5 và 28,9 giây). Hàng cuối chấm cùng rừng cây ngoại tuyến qua bản xuất ONNX (dự đoán trùng khớp với PySpark).

**Phát lại CSE-CIC-IDS2018 trên cụm biên.** Để kiểm tra hệ thống đã triển khai trên một mạng khác, 10.000 luồng lấy từ tập kiểm tra đã khử trùng lặp của CSE-CIC-IDS2018 (12,8% tấn công, cột ánh xạ về tên của CICIDS2017 như ở Mục 4.2.9) được phát lại qua đúng chế độ pipeline split, giữ nguyên cổng lọc và bộ phân loại huấn luyện trên CICIDS2017 (Bảng 4.22). Cổng lọc, vốn hiệu chỉnh trên lưu lượng benign của CICIDS2017, chuyển tiếp 30% luồng của bộ 2018 thay vì 11%, nên bộ phân loại nhận tải gấp ba và thông lượng duy trì giảm từ 22,4 xuống khoảng 8 luồng/giây. Chất lượng phát hiện giảm mạnh: recall chỉ còn 0,8% (10 trên 1.276 luồng tấn công), và chính bộ phân loại, không riêng cổng lọc, bỏ sót gần hết tấn công của bộ 2018 (chạy riêng không có cổng lọc, recall chỉ 1,8%). Kết quả này khớp với đánh giá khác bộ ngoại tuyến ở Mục 4.2.9: khi đưa sang mạng mới, cả thông lượng lẫn chất lượng phát hiện đều phụ thuộc lưu lượng, nên cần hiệu chỉnh lại cổng lọc trên lưu lượng benign tại chỗ và huấn luyện lại bộ phân loại.

**So với số liệu đo trước.** Một lượt đo trước đó của cùng hệ thống ghi nhận 62,5 luồng/giây, p95 13 ms và tỷ lệ bỏ qua 95,8% cho pipeline split. Phép đo đó chỉ đếm các dự đoán ghi được trong cửa sổ tải và chỉ tính độ trễ suy luận của từng nút, nên bỏ sót các luồng còn nằm trong hàng đợi của bộ phân loại PySpark. Mô hình triển khai khi đó cũng dùng danh sách đặc trưng cũ. Các số liệu ở mục này thay thế toàn bộ số liệu đó.

## 4.4 Thảo luận và so sánh tổng hợp

Kết quả thực nghiệm cho thấy lựa chọn đặc trưng dựa trên SHAP là hướng tiếp cận phù hợp cho hệ thống IDS trên bộ dữ liệu CICIDS2017: dưới cấu hình đánh giá loại trừ rò rỉ dữ liệu, SHAP Top-30 giữ được hiệu năng gần như nguyên vẹn trong khi cắt khoảng 60% số đặc trưng. Ngược lại, học tổ hợp không mang lại lợi thế trên chính cấu hình này: Hybrid Bagging đạt F1 0,9966 và Soft Voting đạt 0,9967, đều thấp hơn XGB đơn lẻ (0,9970). Học

tổ hợp chỉ vượt mô hình đơn ở những cấu hình mà mô hình đơn yếu hơn (RF Top-30: 0,9938 so với 0,9922; PCA $k$=40: 0,9940 so với 0,9939), nghĩa là nó bù cho bộ đặc trưng kém chứ không tạo thêm hiệu năng phân loại. Chi phí đi kèm lại lớn: trên SHAP Top-30, Hybrid Bagging cần 4909 giây huấn luyện (so với 1226 giây của XGB), 46,6 giây dự đoán (so với 7,1 giây) và mô hình nặng 7,15 MB, gấp khoảng ba lần XGB (2,37 MB). Vì vậy cấu hình triển khai lên thiết bị biên là một *mô hình đơn* trên SHAP Top-30, không phải học tổ hợp.

Khi triển khai trên cụm hai Jetson Orin Nano Super, Random Forest với SHAP Top-30 đặc trưng (F1 0,9959 trên tập kiểm tra) sau cổng lọc autoencoder duy trì được 22,4 luồng/giây ở chế độ pipeline split, với độ trễ đầu cuối trung vị khoảng 1 giây, p95 7,1 giây và 0,60 J mỗi dự đoán. Cái giá là recall tấn công 69,5% (F1 0,820, precision 100%) khi phát lại trọn 10.000 luồng, và mọi luồng bị sót đều do cổng lọc bỏ qua. Trên lưu lượng CSE-CIC-IDS2018, cùng pipeline chỉ phát hiện dưới 1% luồng tấn công và thông lượng giảm còn khoảng 8 luồng/giây, nên mỗi mạng mới cần hiệu chỉnh cổng lọc và huấn luyện lại bộ phân loại. Triển khai IDS học máy trên thiết bị biên vì vậy khả thi cho cảnh báo gần thời gian thực, và nút thắt nằm ở bộ phân loại PySpark: cùng mô hình chạy qua ONNX Runtime nhanh hơn khoảng 7.200 lần, nên đây là hướng cải thiện trực tiếp nhất.

# CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Trong chương này, luận văn sẽ trình bày những kết quả mà luận văn đã đạt được so với mục tiêu đặt ra ban đầu. Bên cạnh đó, luận văn cũng trình bày những hạn chế và đưa ra hướng phát triển trong tương lai của nghiên cứu.

## 5.1 Những đóng góp chính của luận văn

Ba câu hỏi nghiên cứu nêu ở Mục 1.3.4 được trả lời như sau:

- **RQ1 (giảm chiều có giữ được hiệu năng không)**: Có, với SHAP Top-30. Cấu hình này dùng 30 trên 77 đặc trưng và đạt F1 0,9970, so với 0,9971 của cấu hình đầy đủ (cùng dùng XGB). Kiểm định tương đương xác nhận chênh lệch nhỏ hơn $2{,}30 \times 10^{-4}$ điểm F1. RF Importance (0,9922) và PCA (0,9939) mất hiệu năng nhiều hơn.
- **RQ2 (học tổ hợp có lợi thế không)**: Không. Trên cấu hình tốt nhất, Hybrid Bagging đạt F1 0,9966 so với 0,9970 của XGB đơn lẻ, trong khi chi phí huấn luyện và dung lượng mô hình cao hơn nhiều lần.
- **RQ3 (triển khai trên cụm biên có khả thi không)**: Khả thi cho cảnh báo gần thời gian thực. Chế độ tách pipeline trên hai Jetson duy trì được 22,4 luồng/giây với độ trễ đầu cuối p95 7,1 giây và 0,60 J mỗi dự đoán. Cái giá là recall tấn công 69,5% khi phát lại trọn 10.000 luồng, do cổng lọc bỏ qua một phần luồng tấn công, và nút thắt là bộ phân loại PySpark.

Các đóng góp chính của luận văn với đề tài *"Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT"* gồm:

- **Xây dựng nền tảng xử lý dữ liệu lớn**: Đề xuất và triển khai thành công hệ thống IDS trên nền tảng Apache Spark. Giải pháp này tận dụng khả năng tính toán song song để xử lý các luồng dữ liệu mạng từ môi trường IoT, hướng tới khả năng mở rộng trong việc giám sát bảo mật. Khả năng xử lý phân tán được kiểm chứng ở mức minh chứng khái niệm (proof-of-concept) trên cụm thiết bị biên.
- **Đánh giá học tổ hợp dựa trên bằng chứng thay vì kỳ vọng**: Xây dựng mô hình Hybrid Bagging Ensemble kết hợp Top-3 thuật toán có F1 cao nhất trên tập kiểm định (xác định ở chương thực nghiệm, không cố định trước), rồi so sánh trực tiếp với

các mô hình đơn lẻ kèm kiểm định ý nghĩa thống kê. Dưới quy trình đánh giá loại trừ rò rỉ dữ liệu, học tổ hợp *không* mang lại lợi thế trên cấu hình tốt nhất: Hybrid Bagging đạt F1 0,9966 so với 0,9970 của XGB đơn lẻ, trong khi chi phí huấn luyện và dung lượng mô hình cao hơn nhiều lần. Đây là một kết quả âm tính nhưng có giá trị: trên bài toán nhị phân đã gần bão hòa, việc ghép mô hình không bù lại được chi phí, và mô hình đơn mới là lựa chọn phù hợp cho thiết bị biên.

- **Đánh giá trung thực, loại trừ rò rỉ dữ liệu**: Nhận diện và xử lý các nguồn rò rỉ thường gặp trên CICIDS2017 (đặc trưng `destination_port` rò rỉ nhãn và trùng lặp luồng ở mức đặc trưng), đồng thời định lượng mức thổi phồng hiệu năng do rò rỉ, nhờ đó kết quả phản ánh đúng hiệu năng thực của mô hình.

- **Tối ưu hóa và giải thích mô hình**: Ứng dụng kỹ thuật SHAP để lựa chọn các đặc trưng quan trọng nhất, giúp giảm số lượng đặc trưng (khoảng 60%) mà vẫn giữ nguyên hiệu năng phân loại: dưới cấu hình loại trừ rò rỉ dữ liệu, Random Forest với SHAP Top-30 đạt F1 0,9955, tương đương cấu hình dùng toàn bộ đặc trưng (0,9949). Đồng thời, SHAP cung cấp giải thích chi tiết cho từng quyết định của mô hình, điều cần thiết khi vận hành trong trung tâm giám sát an ninh.

- **Đo đạc triển khai phân tán trên thiết bị biên**: Triển khai pipeline suy luận trên cụm hai Jetson Orin Nano Super và đo thông lượng, độ trễ đầu cuối và năng lượng theo cách có tính đến hàng đợi. Tách cổng lọc autoencoder và bộ phân loại ra hai bo mạch nâng thông lượng lên khoảng 9 lần so với một nút (22,4 so với 2,5 luồng/giây). Khi phát lại trọn 10.000 luồng, pipeline không báo nhầm luồng benign nào (precision 100%) và phát hiện 69,5% luồng tấn công (F1 0,820). Mọi luồng bị sót đều do cổng lọc bỏ qua. Cùng một rừng ngẫu nhiên chạy qua ONNX Runtime nhanh hơn PySpark khoảng 7.200 lần, cho thấy nút thắt nằm ở engine suy luận chứ không ở mô hình.

- **Đánh giá khác bộ dữ liệu**: Huấn luyện trên CICIDS2017 rồi kiểm tra trên CSE-CIC-IDS2018 (và ngược lại) cho thấy hiệu năng cùng bộ gần hoàn hảo không chuyển sang bộ khác: AUC-PR giảm từ 0,9997 xuống 0,4849 và từ 0,9679 xuống 0,1926. Hai kỹ thuật thích ứng miền không giám sát không khắc phục được, trong khi huấn luyện lại với 1% dữ liệu có nhãn của miền đích đưa F1 lên 0,92 và 0,99. Phát lại CSE-CIC-IDS2018 qua chính pipeline trên cụm biên cho cùng kết luận: recall chỉ 0,8% và thông lượng giảm còn khoảng 8 luồng/giây vì cổng lọc chuyển tiếp nhiều gấp ba.

## 5.2 Hạn chế của đề tài

Mặc dù đã đạt được các kết quả nêu trên, luận văn vẫn còn một số hạn chế cần được khắc phục trong các giai đoạn tiếp theo:

- **Phạm vi phân loại**: Hệ thống chính được triển khai dựa trên bài toán phân loại nhị phân (Tấn công so với Bình thường). Luận văn đã bổ sung đánh giá đa lớp theo từng loại tấn công (Mục 4.2.10) để phân tích điểm yếu ở các lớp hiếm, nhưng việc xây dựng một bộ phân loại đa lớp hoàn chỉnh cho triển khai thời gian thực vẫn là hướng phát triển tiếp theo.

- **Phạm vi bộ dữ liệu**: Đánh giá khác bộ dữ liệu (Mục 4.2.9) mới thực hiện trên một cặp, CICIDS2017 và CSE-CIC-IDS2018, với mẫu 20% của bộ sau. Ngoài ra, F1 khác bộ là một đại lượng *không tất định* giữa các lượt chạy: xác suất dự đoán của cụm luồng tấn công miền đích nằm sát ngưỡng quyết định 0,5, nên cùng một cấu hình và cùng một tệp dữ liệu cho hai nhóm kết quả rời nhau, F1 trong khoảng 0,029–0,085 hoặc 0,185–0,301. Vì vậy kết quả khác bộ được báo cáo qua 18 lượt chạy và lấy AUC-PR làm đại lượng so sánh chính. Mọi so sánh khác bộ về sau, kể cả với công trình khác, cũng cần lặp lại nhiều lượt mới có ý nghĩa. Hướng khắc phục căn bản là hiệu chỉnh lại ngưỡng trên miền đích thay vì giữ ngưỡng 0,5 học từ miền nguồn.

- **Quy mô dữ liệu và phần cứng biên**: Khía cạnh xử lý phân tán mới được kiểm chứng ở mức minh chứng khái niệm (proof-of-concept) trên cụm hai thiết bị Jetson (8 GB) với khối lượng dữ liệu cỡ vài triệu luồng của CICIDS2017, chứ chưa phải ở quy mô dữ liệu lớn thực thụ (terabyte trở lên) hay trên cụm tính toán nhiều nút mạnh. Việc đánh giá khả năng mở rộng ở quy mô lớn hơn vẫn là hướng tiếp theo.

- **Đánh đổi recall tại cổng lọc và độ trễ của PySpark**: Ở ngưỡng đã chọn (quantile 0,995), cổng lọc autoencoder chỉ chuyển tiếp khoảng 68–73% luồng tấn công sang bộ phân loại (tùy chế độ triển khai), nên recall của cả hệ thống ở chế độ tách pipeline bị chặn ở mức đó. Bộ phân loại PySpark trên bo mạch lại mất vài giây cho mỗi lô, khiến độ trễ đầu cuối p95 ở mọi chế độ nằm trong khoảng 5,2–11,3 giây. Hệ thống hiện phù hợp với cảnh báo gần thời gian thực hơn là chặn lưu lượng tức thời.

- **Chưa đánh giá trước lưu lượng đối kháng**: Hệ thống chưa được kiểm thử trước các tấn công đối kháng/né tránh (adversarial/evasion) và giả định phân bố tấn công xấp

xỉ CICIDS2017. Do đó, độ bền vững trước kẻ tấn công cố ý đánh lừa mô hình vẫn là câu hỏi mở.

## 5.3 Hướng phát triển trong tương lai

Dựa trên những kết quả đã đạt được và các hạn chế đã nhận diện, luận văn đề xuất một số hướng phát triển trong tương lai, mỗi hướng tương ứng với một hạn chế đã nêu:

- **Cải thiện độ nhạy cho các lớp tấn công hiếm**: Các lớp hiếm suy giảm theo hai kiểu (Mục 4.2.10): nhầm loại trong cùng họ (XSS bị nhận thành Brute Force) và lọt hẳn thành Benign (SQL Injection, 54% Bot). Bốn hướng khắc phục tương ứng:
  - ◇ *Xử lý mất cân bằng có chủ đích* ở pha huấn luyện đa lớp: undersampling lớp Benign, oversampling/SMOTE [72] cho lớp hiếm (áp dụng sau khi chia tập để tránh rò rỉ), hoặc gộp ba biến thể Web Attack thành một lớp để đổi độ mịn định danh lấy recall.
  - ◇ *Điều chỉnh thuật toán*: trọng số lớp cho bài toán đa lớp, focal loss [73] với XGBoost, và hạ ngưỡng quyết định theo lớp. Thêm false positive ở lớp hiếm là chấp nhận được, vì với IDS báo nhầm rẻ hơn bỏ sót.
  - ◇ *Kiến trúc phân tầng*, tận dụng chính thiết kế hai tầng có cổng lọc của đề tài: tầng nhị phân trên biên giữ recall tổng, còn bộ định danh loại chỉ huấn luyện trên các luồng tấn công nên không phải cạnh tranh với 380 nghìn mẫu Benign. Bộ này có thể huấn luyện lại ngoại tuyến với cân bằng lớp mà không đụng hệ thống đang chạy.
  - ◇ *Đặc trưng chuỗi thời gian theo host* (chu kỳ kết nối lặp, entropy kích thước gói qua cửa sổ trượt). Hành vi beaconing C&C khiến 54% Bot lọt lưới không bộc lộ qua thống kê từng luồng độc lập.
- **Kiểm chứng trên bộ dữ liệu IoT chuyên biệt**: Đánh giá quy trình trên các bộ dữ liệu chứa lưu lượng IoT thực thụ (Bot-IoT, TON_IoT, CIC-IoT-2023) và mở rộng thí nghiệm liên bộ dữ liệu lên toàn bộ CSE-CIC-IDS2018, nhằm thu hẹp khoảng cách giữa phạm vi "IoT" của đề tài và dữ liệu CICIDS2017 hiện dùng.
- **Tối ưu suy luận biên bằng ONNX/TensorRT**: Tích hợp phiên suy luận ONNX vào pipeline streaming và tăng tốc bằng TensorRT trên GPU Jetson để giảm chi phí JVM/Spark ở pha suy luận (trong khi vẫn giữ Spark cho huấn luyện phân tán): kết

quả đo ở Bảng 4.18 cho thấy cùng một rừng ngẫu nhiên (dự đoán trùng nhau 100%) chạy qua ONNX Runtime đạt thông lượng gấp ≈7.200 lần bản PySpark đã triển khai. Vì mọi chế độ triển khai hiện bị bộ phân loại PySpark giới hạn thông lượng và đuôi độ trễ vài giây, thay engine suy luận là cách trực tiếp nhất để nâng thông lượng và rút độ trễ đầu cuối xuống mức mili giây.

- **Độ bền vững trước tấn công đối kháng**: Đánh giá và gia cố hệ thống trước lưu lượng né tránh (adversarial/evasion), đặc biệt là hai bề mặt tấn công mới do cổng lọc bất thường tạo ra: giả dạng lưu lượng bình thường để vượt cổng lọc và cố ý tạo sai số tái tạo cao để dồn tải lên nút phân loại.

# PHỤ LỤC A. CÔNG TRÌNH KHOA HỌC ĐÃ CÔNG BỐ

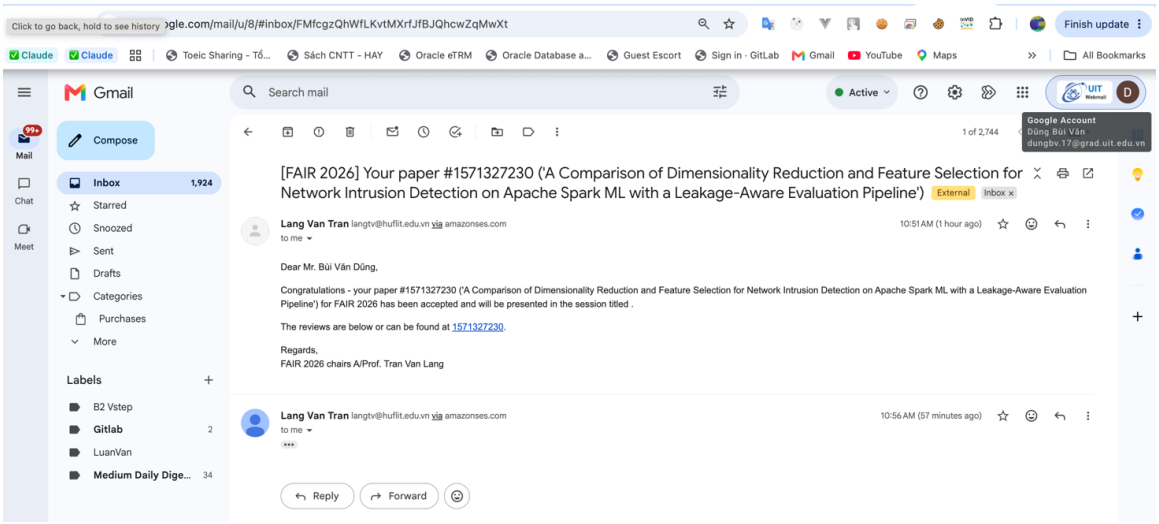
Phần này liệt kê hai bài báo khoa học liên quan đến đề tài luận văn (viết chung với ThS. Nguyễn Vũ Thái, TS. Đỗ Trí Nhựt và TS. Nguyễn Văn Dũ). Bài thứ nhất đã được trình bày tại Hội nghị FAIR 2026 (Mục A.1). Bài thứ hai đã được chấp nhận đăng tại SOICT 2026 (Mục A.2). Bản PDF đầy đủ của cả hai bài được đính kèm ở cuối luận văn, sau phần Tài liệu tham khảo.

1. Thai Nguyen Vu, **Bui Van Dung**, Tri Nhut Do, Van Du Nguyen, "A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline," National Conference on Fundamental and Applied Information Technology Research (FAIR), 2026.

2. Thai Nguyen Vu, **Bui Van Dung**, Tri Nhut Do, Van Du Nguyen, "A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark," International Symposium on Information and Communication Technology (SOICT 2026), Communications in Computer and Information Science (CCIS), Springer (**chỉ mục Scopus**). Đã được chấp nhận đăng và trình bày báo cáo (oral), TP. Hồ Chí Minh, 04–05/12/2026.

## A.1 Bài báo tại Hội nghị FAIR 2026

Bài báo thứ nhất trình bày phần đánh giá ngoại tuyến của luận văn (Chương 4). Hình A.1 là thư thông báo chấp nhận đăng bài của Ban tổ chức Hội nghị FAIR 2026, và Hình A.2 là giấy chứng nhận bài báo đã được trình bày tại hội nghị (TP. Hồ Chí Minh, ngày 08/10/2026). Biên nhận chuyển giao bản quyền bài báo cho IEEE (IEEE Copyright and Consent Form) được đính kèm ngay sau hai hình này.




Hình A.1. Thư thông báo chấp nhận đăng bài (paper #1571327230) của Ban tổ chức Hội nghị FAIR 2026




Hình A.2. Giấy chứng nhận bài báo được trình bày tại Hội nghị Khoa học Quốc gia lần thứ XIX FAIR 2026

# IEEE COPYRIGHT AND CONSENT FORM

To ensure uniformity of treatment among all contributors, other forms may not be substituted for this form, nor may any wording of the form be changed. This form is intended for original material submitted to the IEEE and must accompany any such material in order to be published by the IEEE. Please read the form carefully and keep a copy for your files.

**A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline**

**Mr. Thai Nguyen Vu, Mr. Bùi Văn Dũng, Dr. Tri Nhut Do and Dr. Van Du Nguyen**

**2026 19th Conference on Fundamental and Applied IT Research (FAIR)**

## COPYRIGHT TRANSFER

The undersigned hereby assigns to The Institute of Electrical and Electronics Engineers, Incorporated (the "IEEE") all rights under copyright that may exist in and to: (a) the Work, including any revised or expanded derivative works submitted to the IEEE by the undersigned based on the Work; and (b) any associated written or multimedia components or other enhancements accompanying the Work.

## GENERAL TERMS

1. The undersigned represents that he/she has the power and authority to make and execute this form.

2. The undersigned agrees to indemnify and hold harmless the IEEE from any damage or expense that may arise in the event of a breach of any of the warranties set forth above.

3. The undersigned agrees that publication with IEEE is subject to the policies and procedures of the IEEE PSPB Operations Manual.

4. In the event the above work is not accepted and published by the IEEE or is withdrawn by the author(s) before acceptance by the IEEE, the foregoing copyright transfer shall be null and void. In this case, IEEE will retain a copy of the manuscript for internal administrative/record-keeping purposes.

5. For jointly authored Works, all joint authors should sign, or one of the authors should sign as authorized agent for the others.

6. The author hereby warrants that the Work and Presentation (collectively, the "Materials") are original and that he/she is the author of the Materials. To the extent the Materials incorporate text passages, figures, data or other material from the works of others, the author has obtained any necessary permissions. Where necessary, the author has obtained all third party permissions and consents to grant the license above and has provided copies of such permissions and consents to IEEE

**You have indicated that you DO NOT wish to have video/audio recordings made of your conference presentation under terms and conditions set forth in "Consent and Release."**

BY TYPING IN YOUR FULL NAME BELOW AND CLICKING THE SUBMIT BUTTON, YOU CERTIFY THAT SUCH ACTION CONSTITUTES YOUR ELECTRONIC SIGNATURE TO THIS FORM IN ACCORDANCE WITH UNITED STATES LAW, WHICH AUTHORIZES ELECTRONIC SIGNATURE BY AUTHENTICATED REQUEST FROM A USER OVER THE INTERNET AS A VALID SUBSTITUTE FOR A WRITTEN SIGNATURE.

| Thai Nguyen Vu | 29-09-2026 |
|---|---|
| **Signature** | **Date (dd-mm-yyyy)** |



# Information for Authors

## AUTHOR RESPONSIBILITIES

The IEEE distributes its technical publications throughout the world and wants to ensure that the material submitted to its publications is properly available to the readership of those publications. Authors must ensure that their Work meets the requirements as stated in section 8.2.1 of the IEEE PSPB Operations Manual, including provisions covering originality, authorship, author responsibilities and author misconduct. More information on IEEE's publishing policies may be found at http://www.ieee.org/publications_standards/publications/rights/authorrightsresponsibilities.html Authors are advised especially of IEEE PSPB Operations Manual section 8.2.1. B12: "It is the responsibility of the authors, not the IEEE, to determine whether disclosure of their material requires the prior consent of other parties and, if so, to obtain it." Authors are also advised of IEEE PSPB Operations Manual section 8.1.1B: "Statements and opinions given in work published by the IEEE are the expression of the authors."

## RETAINED RIGHTS/TERMS AND CONDITIONS

- Authors/employers retain all proprietary rights in any process, procedure, or article of manufacture described in the Work.
- Authors/employers may reproduce or authorize others to reproduce the Work, material extracted verbatim from the Work, or derivative works for the author's personal use or for company use, provided that the source and the IEEE copyright notice are indicated, the copies are not used in any way that implies IEEE endorsement of a product or service of any employer, and the copies themselves are not offered for sale.
- Although authors are permitted to re-use all or portions of the Work in other works, this does not include granting third-party requests for reprinting, republishing, or other types of re-use.The IEEE Intellectual Property Rights office must handle all such third-party requests.
- Authors whose work was performed under a grant from a government funding agency are free to fulfill any deposit mandates from that funding agency.

## AUTHOR ONLINE USE

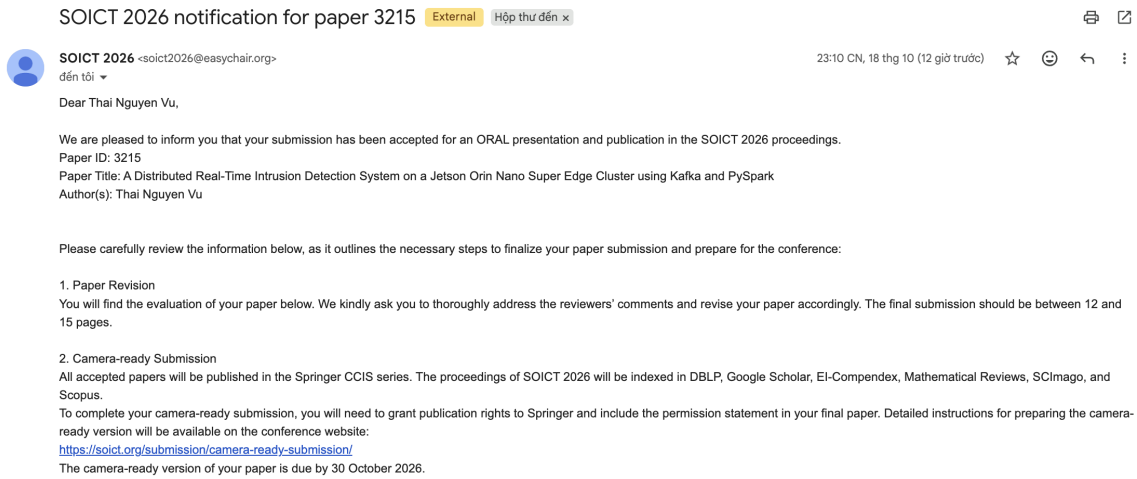
- **Personal Servers**. Authors and/or their employers shall have the right to post the accepted version of IEEE-copyrighted articles on their own personal servers or the servers of their institutions or employers without permission from IEEE, provided that the posted version includes a prominently displayed IEEE copyright notice and, when published, a full citation to the original IEEE publication, including a link to the article abstract in IEEE Xplore. Authors shall not post the final, published versions of their papers.

- **Classroom or Internal Training Use.** An author is expressly permitted to post any portion of the accepted version of his/her own IEEE-copyrighted articles on the author's personal web site or the servers of the author's institution or company in connection with the author's teaching, training, or work responsibilities, provided that the appropriate copyright, credit, and reuse notices appear prominently with the posted material. Examples of permitted uses are lecture materials, course packs, e-reserves, conference presentations, or in-house training courses.
- **Electronic Preprints.** Before submitting an article to an IEEE publication, authors frequently post their manuscripts to their own web site, their employer's site, or to another server that invites constructive comment from colleagues. Upon submission of an article to IEEE, an author is required to transfer copyright in the article to IEEE, and the author must update any previously posted version of the article with a prominently displayed IEEE copyright notice. Upon publication of an article by the IEEE, the author must replace any previously posted electronic versions of the article with either (1) the full citation to the IEEE work with a Digital Object Identifier (DOI) or link to the article abstract in IEEE Xplore, or (2) the accepted version only (not the IEEE-published version), including the IEEE copyright notice and full citation, with a link to the final, published article in IEEE Xplore.

**Questions about the submission of the form or manuscript must be sent to the publication's editor.**
**Please direct all questions about IEEE copyright policy to:**
**IEEE Intellectual Property Rights Office, copyrights@ieee.org, +1-732-562-3966**

## A.2 Bài báo tại Hội nghị SOICT 2026

Bài báo thứ hai trình bày phần triển khai trên cụm thiết bị biên của luận văn (Chương 4). Hình A.3 là thư thông báo chấp nhận đăng bài của Ban chương trình SOICT 2026 (mã bài 3215, ngày 18/10/2026). Bài báo được chấp nhận trình bày báo cáo (oral) và đăng trong kỷ yếu thuộc bộ Communications in Computer and Information Science (CCIS) của Springer. Theo thư chấp nhận, kỷ yếu SOICT 2026 được chỉ mục trong DBLP, Google Scholar, EI-Compendex, Mathematical Reviews, SCImago và Scopus. Bản đính kèm ở cuối luận văn là bản hoàn chỉnh (camera-ready) đã sửa theo góp ý của các phản biện.



SOICT 2026 notification for paper 3215 External Hộp thư đến ×

**SOICT 2026** <soict2026@easychair.org> 23:10 CN, 18 thg 10 (12 giờ trước)
đến tôi

Dear Thai Nguyen Vu,

We are pleased to inform you that your submission has been accepted for an ORAL presentation and publication in the SOICT 2026 proceedings.
Paper ID: 3215
Paper Title: A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark
Author(s): Thai Nguyen Vu

Please carefully review the information below, as it outlines the necessary steps to finalize your paper submission and prepare for the conference:

1. Paper Revision
You will find the evaluation of your paper below. We kindly ask you to thoroughly address the reviewers' comments and revise your paper accordingly. The final submission should be between 12 and 15 pages.

2. Camera-ready Submission
All accepted papers will be published in the Springer CCIS series. The proceedings of SOICT 2026 will be indexed in DBLP, Google Scholar, EI-Compendex, Mathematical Reviews, SCImago, and Scopus.
To complete your camera-ready submission, you will need to grant publication rights to Springer and include the permission statement in your final paper. Detailed instructions for preparing the camera-ready version will be available on the conference website:
https://soict.org/submission/camera-ready-submission/
The camera-ready version of your paper is due by 30 October 2026.

Hình A.3. Thư thông báo chấp nhận đăng bài (paper 3215) của Ban chương trình SOICT 2026

# TÀI LIỆU THAM KHẢO


[1] Ansam Khraisat, Iqbal Gondal, Peter Vamplew, and Joarder Kamruzzaman. "Survey of intrusion detection systems: techniques, datasets and challenges". In: *Cybersecurity* 2 (2019), p. 20. DOI: 10.1186/s42400-019-0038-7.

[2] Mohamed Amine Ferrag, Leandros Maglaras, Sotiris Moschoyiannis, and Helge Janicke. "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study". In: *Journal of Information Security and Applications* 50 (2020), p. 102419. DOI: 10.1016/j.jisa.2019.102419.

[3] R. Vinayakumar, Mamoun Alazab, K. P. Soman, Prabaharan Poornachandran, Ameer Al-Nemrat, and Sitalakshmi Venkatraman. "Deep learning approach for intelligent intrusion detection system". In: *IEEE Access* 7 (2019), pp. 41525–41550. DOI: 10.1109/ACCESS.2019.2895334.

[4] Amardeep Singh and Julian Jang-Jaccard. *Autoencoder-based Unsupervised Intrusion Detection using Multi-Scale Convolutional Recurrent Networks*. 2022. arXiv: 2204.03779 [cs.CR].

[5] Pengfei Sun, Pengju Liu, Qi Li, Chenxi Liu, Xiangling Lu, Ruochen Hao, and Jinpeng Chen. "DL-IDS: Extracting features using CNN-LSTM hybrid network for intrusion detection system". In: *Security and Communication Networks* 2020 (2020), p. 8890306. DOI: 10.1155/2020/8890306.

[6] N. Dash, S. Chakravarty, A. K. Rath, et al. "An optimized LSTM-based deep learning model for anomaly network intrusion detection". In: *Scientific Reports* 15 (2025), p. 1554. DOI: 10.1038/s41598-025-85248-z.

[7] Sukhvinder Singh Bamber, Aditya Vardhan Reddy Katkuri, Shubham Sharma, and Mohit Angurala. "A hybrid CNN-LSTM approach for intelligent cyber intrusion detection system". In: *Computers & Security* 148 (2025), p. 104146. DOI: 10.1016/j.cose.2024.104146.

[8] Tasnimul Hasan, Abrar Hossain, Mufakir Qamar Ansari, and Talha Hussain Syed. "Enhanced Intrusion Detection in IIoT Networks: A Lightweight Approach with Autoencoder-Based Feature Learning". In: *Proceedings of the 10th International Conference on Internet of Things, Big Data and Security (IoTBDS)*. SciTePress, 2025, pp. 207–214. DOI: 10.5220/0013203700003944.

[9] P. R. Chithra Rani and K. Baalaji. “Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms”. In: *Discover Applied Sciences* 7 (2025), p. 928. DOI: 10.1007/s42452-025-06871-z.

[10] Kushboo Nasir, Sahar K. Badri, Daniyal M. Alghazzawi, Mohammed Yahya Alghamdi, Mona Alkhozae, Abeer Almakky, Rania M. Alhazmi, and Muhammad Zubair Asghar. “HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning”. In: *Scientific Reports* 16 (2026), p. 2313. DOI: 10.1038/s41598-025-32183-8.

[11] Jackson Diaz-Gorrin, Candido Caballero-Gil, and Ljiljana Brankovic. “Enhancing Network Security with Generative AI on Jetson Orin Nano”. In: *Applied Sciences* 16.3 (2026), p. 1442. DOI: 10.3390/app16031442.

[12] Zhenxiong Zhang, Lei Zhang, Jialong Xu, Zhengze Chen, and Peng Wang. “An Edge-Deployable Lightweight Intrusion Detection System for Industrial Control”. In: *Electronics* 15.3 (2026), p. 644. DOI: 10.3390/electronics15030644.

[13] Laila Nassef, Mohammed Ibrahim Alghamdi, Slim Ben Chaabane, Qaisar Abbas, Wedad M. Alawad, Omar H. Albalawi, Osamah I. Alqaisi, and Bahjat Fakieh. “Lightweight and Energy-Aware Intrusion Detection for Industrial IoT Using TinyML and Edge AI”. In: *Scientific Reports* 16.1 (2026), p. 18524. DOI: 10.1038/s41598-026-50690-0.

[14] Kotyada Mohan Kiran Kumar, M. Vivek Srikar Reddy, Karthik Ullas, and M. Supriya. “Distributed Intrusion Detection System using Kafka and Spark Streaming”. In: *2025 International Conference on Visual Analytics and Data Visualization (ICVADV)*. IEEE, 2025, pp. 302–309. DOI: 10.1109/ICVADV63329.2025.10961691.

[15] Ilhan Firat Kilincer, Fatih Ertam, and Abdulkadir Sengur. “Machine learning methods for cyber security intrusion detection: Datasets and comparative study”. In: *Computer Networks* 188 (2021), p. 107840. DOI: 10.1016/j.comnet.2021.107840.

[16] M. Al Lail, A. Garcia, and S. Olivo. “Machine learning for network intrusion detection—A comparative study”. In: *Future Internet* 15.7 (2023), p. 243. DOI: 10.3390/fi15070243.

[17] Yung-Chung Wang, Yi-Chun Houng, Han-Xuan Chen, and Shu-Ming Tseng. "Network Anomaly Intrusion Detection Based on Deep Learning Approach". In: *Sensors* 23.4 (2023), p. 2171. DOI: 10.3390/s23042171.

[18] L. Yang, A. Moubayed, I. Hamieh, and A. Shami. "Tree-Based Intelligent Intrusion Detection System in Internet of Vehicles". In: *2019 IEEE Global Communications Conference (GLOBECOM)*. 2019, pp. 1–6. DOI: 10.1109/GLOBECOM38437.2019.9013892.

[19] L. Yang, A. Moubayed, and A. Shami. "MTH-IDS: A Multi-Tiered Hybrid Intrusion Detection System for Internet of Vehicles". In: *IEEE Internet of Things Journal* 9.1 (2022), pp. 616–632. DOI: 10.1109/JIOT.2021.3084796.

[20] L. Yang, A. Shami, G. Stevens, and S. DeRusett. "LCCDE: A Decision-Based Ensemble Framework for Intrusion Detection in The Internet of Vehicles". In: *2022 IEEE Global Communications Conference (GLOBECOM)*. 2022, pp. 3545–3550. DOI: 10.1109/GLOBECOM48099.2022.10001280.

[21] Jose Luis Hernandez-Ramos, Georgios Karopoulos, Efstratios Chatzoglou, Vasileios Kouliaridis, Enrique Marmol, Aurora Gonzalez-Vidal, and Georgios Kambourakis. "Intrusion Detection Based on Federated Learning: A Systematic Review". In: *ACM Computing Surveys* 57.12 (2025), pp. 1–65. DOI: 10.1145/3731596.

[22] R. Baidar, S. Maric, and R. Abbas. *Hybrid deep learning–federated learning powered intrusion detection system for IoT/5G advanced edge computing network*. https://arxiv.org/abs/2509.15555. arXiv:2509.15555. 2025.

[23] B. Williams and L. Qian. "Semi-supervised learning for intrusion detection in large computer networks". In: *Applied Sciences* 15.11 (2025), p. 5930. DOI: 10.3390/app15115930.

[24] N. Albanbay, Y. Tursynbek, K. Graffi, R. Uskenbayeva, Z. Kalpeyeva, Z. Abilkaiyr, and Y. Ayapov. "Federated learning-based intrusion detection in IoT networks: Performance evaluation and data scaling study". In: *Journal of Sensor and Actuator Networks* 14.4 (2025), p. 78. DOI: 10.3390/jsan14040078.

[25] Thien Duc Nguyen, Samuel Marchal, Markus Miettinen, Hossein Fereidooni, N. Asokan, and Ahmad-Reza Sadeghi. "DÏoT: A Federated Self-learning Anomaly Detection System for IoT". In: *2019 IEEE 39th International Conference on Distributed*

*Computing Systems (ICDCS)*. 2019, pp. 756–767. DOI: 10.1109/ICDCS.2019.00080.

[26] Lynda Boukela, Gongxuan Zhang, Méziane Yacoub, and Samia Bouzefrane. "A Near-Autonomous and Incremental Intrusion Detection System Through Active Learning of Known and Unknown Attacks". In: *2021 International Conference on Security, Pattern Analysis, and Cybernetics (SPAC)*. 2021, pp. 374–379. DOI: 10.1109/SPAC53836.2021.9539947.

[27] L. Atzori, A. Iera, and G. Morabito. "The Internet of Things: A survey". In: *Computer Networks* 54.15 (2010), pp. 2787–2805. DOI: 10.1016/j.comnet.2010.05.010.

[28] Cisco. *Cisco Annual Internet Report (2018–2023) White Paper*. https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html. 2020.

[29] R. Khan, S. U. Khan, R. Zaheer, and S. Khan. "Future Internet: The Internet of Things architecture, possible applications and key challenges". In: *Proceedings of the 10th International Conference on Frontiers of Information Technology (FIT)*. 2012, pp. 257–260. DOI: 10.1109/FIT.2012.53.

[30] H. HaddadPajouh, A. Dehghantanha, R. M. Parizi, M. Aledhari, and H. Karimipour. "A survey on Internet of Things security: Requirements, challenges, and solutions". In: *Internet of Things* 14 (2021), p. 100129. DOI: 10.1016/j.iot.2019.100129.

[31] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung. "Intrusion detection system: A comprehensive review". In: *Journal of Network and Computer Applications* 36.1 (2013), pp. 16–24. DOI: 10.1016/j.jnca.2012.09.004.

[32] Varun Chandola, Arindam Banerjee, and Vipin Kumar. "Anomaly detection: A survey". In: *ACM Computing Surveys* 41.3 (2009), pp. 1–58. DOI: 10.1145/1541880.1541882.

[33] Lukas Ruff, Jacob R. Kauffmann, Robert A. Vandermeulen, Grégoire Montavon, Wojciech Samek, Marius Kloft, Thomas G. Dietterich, and Klaus-Robert Müller. "A unifying review of deep and shallow anomaly detection". In: *Proceedings of the IEEE* 109.5 (2021), pp. 756–795. DOI: 10.1109/JPROC.2021.3052449.

[34] Manuel Lopez-Martin, Belen Carro, Antonio Sanchez-Esguevillas, and Jaime Lloret. “Conditional Variational Autoencoder for Prediction and Feature Recovery Applied to Intrusion Detection in IoT”. In: *Sensors* 17.9 (2017), p. 1967. DOI: 10.3390/s17091967.

[35] Khaled Alrawashdeh and Carla Purdy. “Toward an online anomaly intrusion detection system based on deep learning”. In: *Proc. 15th IEEE Int. Conf. on Machine Learning and Applications (ICMLA)*. 2016, pp. 195–200. DOI: 10.1109/ICMLA.2016.0040.

[36] L. Husamaldin and N. Saeed. “Big Data Analytics Correlation Taxonomy”. In: *Information* 11.1 (2020), p. 17. DOI: 10.3390/info11010017.

[37] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. “Apache Spark: A Unified Engine for Big Data Processing”. In: *Communications of the ACM* 59.11 (2016), pp. 56–65. DOI: 10.1145/2934664.

[38] H. Karau, A. Konwinski, P. Wendell, and M. Zaharia. *Learning Spark: Lightning-fast big data analysis*. O’Reilly Media, 2015.

[39] Xiangrui Meng, Joseph Bradley, Burak Yavuz, Evan Sparks, Shivaram Venkataraman, Davies Liu, Jeremy Freeman, DB Tsai, Manish Amde, Sean Owen, Doris Xin, Reynold Xin, Michael J. Franklin, Reza Zadeh, Matei Zaharia, and Ameet Talwalkar. “MLlib: Machine Learning in Apache Spark”. In: *Journal of Machine Learning Research* 17.34 (2016), pp. 1–7.

[40] A. L. Buczak and E. Guven. “A survey of data mining and machine learning methods for cyber security intrusion detection”. In: *IEEE Communications Surveys & Tutorials* 18.2 (2016), pp. 1153–1176. DOI: 10.1109/COMST.2015.2494502.

[41] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull. “Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset”. In: *Future Generation Computer Systems* 100 (2019), pp. 779–796. DOI: 10.1016/j.future.2019.05.041.

[42] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.

[43] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32.

[44] D. R. Cox. “The regression analysis of binary sequences”. In: *Journal of the Royal Statistical Society. Series B (Methodological)* 20.2 (1958), pp. 215–232.

[45] Jerome H. Friedman. “Greedy function approximation: A gradient boosting machine”. In: *Annals of Statistics* 29.5 (2001), pp. 1189–1232.

[46] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. “LightGBM: A highly efficient gradient boosting decision tree”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 3149–3157.

[47] Andrew McCallum and Kamal Nigam. “A comparison of event models for naive Bayes text classification”. In: *AAAI Workshop on Learning for Text Categorization*. 1998.

[48] Corinna Cortes and Vladimir Vapnik. “Support-vector networks”. In: *Machine learning* 20.3 (1995), pp. 273–297.

[49] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *Nature* 323 (1986), pp. 533–536.

[50] Tianqi Chen and Carlos Guestrin. “XGBoost: A scalable tree boosting system”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 785–794.

[51] Zhi-Hua Zhou. *Ensemble Methods: Foundations and Algorithms*. Chapman and Hall/CRC, 2012.

[52] Patrik Goldschmidt and Daniela Chudá. “Network intrusion datasets: A survey, limitations, and recommendations”. In: *Computers & Security* 156 (2025), p. 104510. DOI: 10.1016/j.cose.2025.104510.

[53] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. “Toward generating a new intrusion detection dataset and intrusion traffic characterization”. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 2018, pp. 108–116. DOI: 10.5220/0006639801080116.

[54] Canadian Institute for Cybersecurity and Communications Security Establishment. *CSE-CIC-IDS2018 dataset: A collaborative project between the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC)*. https://www.unb.ca/cic/datasets/ids-2018.html. 2018.

[55] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. "Dos and Don'ts of Machine Learning in Computer Security". In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, pp. 3971–3988.

[56] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Éric Totel. "Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes". In: *Risks and Security of Internet and Systems (CRiSIS 2022)*. Ed. by Slim Kallel, Mohamed Jmaiel, Mohammad Zulkernine, Ahmed Hadj Kacem, Frédéric Cuppens, and Nora Cuppens. Vol. 13857. Lecture Notes in Computer Science. Cham: Springer, 2023, pp. 18–33. DOI: 10.1007/978-3-031-31108-6_2.

[57] Isabelle Guyon and André Elisseeff. "An introduction to variable and feature selection". In: *Journal of Machine Learning Research* 3 (2003), pp. 1157–1182.

[58] Ian T. Jolliffe. *Principal Component Analysis*. 2nd ed. Springer Series in Statistics. Springer, 2002. DOI: 10.1007/b98835.

[59] Scott M. Lundberg and Su-In Lee. "A Unified Approach to Interpreting Model Predictions". In: *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*. Curran Associates, 2017, pp. 4768–4777.

[60] Gints Engelen, Vera Rimmer, and Wouter Joosen. "Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study". In: *2021 IEEE Security and Privacy Workshops (SPW)*. 2021, pp. 7–12. DOI: 10.1109/SPW53761.2021.00009.

[61] Claude Nadeau and Yoshua Bengio. "Inference for the Generalization Error". In: *Machine Learning* 52.3 (2003), pp. 239–281.

[62] Donald J. Schuirmann. "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability". In: *Journal of Pharmacokinetics and Biopharmaceutics* 15.6 (1987), pp. 657–680. DOI: 10.1007/BF01068419.

[63] Laurens D'hooge, Tim Wauters, Bruno Volckaert, and Filip De Turck. "Inter-dataset generalization strength of supervised machine learning methods for intrusion detection". In: *Journal of Information Security and Applications* 54 (2020), p. 102564. DOI: 10.1016/j.jisa.2020.102564.

[64] Marco Cantone, Claudio Marrocco, and Alessandro Bria. “Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study”. In: *IEEE Access* 12 (2024), pp. 144489–144508. DOI: 10.1109/ACCESS.2024.3472907.

[65] Mohanad Sarhan, Siamak Layeghy, and Marius Portmann. “Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection”. In: *Big Data Research* 30 (2022), p. 100359. DOI: 10.1016/j.bdr.2022.100359.

[66] Baochen Sun, Jiashi Feng, and Kate Saenko. “Return of Frustratingly Easy Domain Adaptation”. In: *Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI)*. 2016. DOI: 10.1609/aaai.v30i1.10306.

[67] NVIDIA Corporation. *Jetson Orin Nano Super Developer Kit*. https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/. 2024.

[68] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. “Edge computing: Vision and challenges”. In: *IEEE Internet of Things Journal* 3.5 (2016), pp. 637–646. DOI: 10.1109/JIOT.2016.2579198.

[69] Salam Hamdan, Moussa Ayyash, and Sufyan Almajali. “Edge-Computing Architectures for Internet of Things Applications: A Survey”. In: *Sensors* 20.22 (2020), p. 6441. DOI: 10.3390/s20226441.

[70] Alessandro Sforzin, Félix Gómez Mármol, Mauro Conti, and Jens-Matthias Bohli. “RPiDS: Raspberry Pi IDS — A Fruitful Intrusion Detection System for IoT”. In: *Proc. Int. IEEE Conf. on Ubiquitous Intelligence & Computing (UIC/ATC/ScalCom)*. 2016, pp. 440–448. DOI: 10.1109/UIC-ATC-ScalCom-CBDCom-IoP-SmartWorld.2016.0080.

[71] Jay Kreps, Neha Narkhede, and Jun Rao. “Kafka: a Distributed Messaging System for Log Processing”. In: *Proceedings of the NetDB Workshop*. 2011.

[72] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. “SMOTE: Synthetic Minority Over-sampling Technique”. In: *Journal of Artificial Intelligence Research* 16 (2002), pp. 321–357.

[73] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. “Focal Loss for Dense Object Detection”. In: *2017 IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2999–3007. DOI: 10.1109/ICCV.2017.324.

# A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline


Thai Nguyen Vu
*Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam*
thaivn_ph@utc.edu.vn

Bui Van Dung
*University of Information Technology VNU-HCM, Ho Chi Minh City, Vietnam*
dungbv.17@grad.uit.edu.vn

Tri Nhut Do
*University of Information Technology VNU-HCM, Ho Chi Minh City, Vietnam*
trinhutdo@uit.edu.vn

Van Du Nguyen
*Faculty of Information Technology Nong Lam University, Ho Chi Minh City, Vietnam*
nvdu@hcmuaf.edu.vn



***Abstract*—Machine-learning-based network intrusion detection systems (IDS) typically process dozens to hundreds of network-flow features, which is computationally expensive and hard to deploy at the edge. This paper presents a systematic comparative study of four dimensionality-reduction strategies on Apache Spark ML: a baseline using all features, selection by Random Forest Importance, selection by SHAP, and PCA. Each strategy is evaluated across eight classification algorithms together with two ensemble methods on the CICIDS2017 dataset. Unlike most prior work, our emphasis falls on the rigor of the evaluation pipeline: (i) we remove the label-leaking feature `destination_port` and deduplicate flows at the feature level before splitting the data; (ii) we fix the comparison configuration *a priori* to avoid selection bias; (iii) we select models on a validation set held out from the training set, together with an in-distribution stability check and a cross-dataset generalization evaluation (between CICIDS2017 and CSE-CIC-IDS2018); (iv) we assess statistical significance using a paired permutation test over multiple resampled training/test splits, together with Nadeau–Bengio corrected confidence intervals and effect sizes; and (v) we add a per-attack-type multiclass evaluation. The experiments find that feature selection cuts the feature count by about 60% while SHAP Top-30 stays statistically equivalent to the full feature set within $\pm 2.3 \times 10^{-4}$ F1. Cross-dataset F1 collapses and, in one direction, is bimodal across identical runs, so we report its two regimes and compare on AUC-PR. The SHAP Top-30 model is exported to the Jetson Orin Nano Super edge device in a companion systems paper.**



***Index Terms*—Intrusion detection, Apache Spark, Feature selection, SHAP, Data leakage**


## I. Introduction

Cyberattacks are growing in scale and sophistication, demanding IDS that are both accurate and scalable on large data [1], [2]. Machine learning has become the dominant approach for anomaly-based IDS, but modern network-flow datasets such as CICIDS2017 [3] provide nearly 80 features per flow, which increases training, inference, and memory cost, and that cost is hardest to absorb on an edge device. Dimensionality reduction addresses that cost, but the available strategies, embedded, unsupervised, or explanation-based, measure "importance" in non-equivalent ways, and they are rarely compared under the same distributed pipeline.

A second issue, discussed less often but no less serious, is whether the evaluation pipeline can be trusted. Many studies report near-perfect binary F1 on CICIDS2017, which is usually a sign of data leakage rather than of model capability. *Data leakage* is information unavailable at deployment time entering training or evaluation, inflating scores so they no longer reflect generalization. Labeling errors and duplicate flows in CICIDS2017 are documented and substantially affect evaluation [4]. There are two common sources of leakage. First, `destination_port` encodes the attacked service, since CICIDS2017 aims its attacks at fixed ports, giving the model a port-to-label shortcut. Second, feature-level duplicate flows are not removed by full-duplicate removal, so a random split places the same sample into both the training and test sets. This paper handles both before comparing methods.

**Main contributions.** (1) A uniform comparison of

RF Importance, SHAP, and PCA over the same eight Spark ML algorithms and two ensembles; (2) Integration of SHAP-based explanation [5] with a direct contrast against RF Importance [6]; (3) A leakage-aware and statistically tested evaluation pipeline: removing the leaking feature, deterministic feature-level deduplication (label-conflicting groups are removed entirely), model selection on a validation set held out from the training set, and a paired permutation test (exact enumeration) over multiple resamplings with Nadeau–Bengio correction and effect size; (4) A per-attack-type multiclass evaluation, exposing the rare-class weaknesses that binary F1 hides; (5) An edge-aware model-selection recommendation: jointly considering F1, the number of features, and model size for the Jetson Orin Nano Super.

## II. Related Work

IDS surveys [1], [2], [7] show that both traditional machine learning and deep learning are widely applied on standard benchmarks such as CICIDS2017 [3]. Many works center on one model family, for instance a deep belief network [8] or deep neural networks benchmarked against classical classifiers [9], without a uniform comparison of dimensionality-reduction strategies on the same distributed pipeline. Apache Spark ML [10], [11] supports large-scale processing, but few studies combine feature selection, model explanation, and statistical testing in a single unified experimental framework. Recent edge IDS studies target resource-constrained platforms, for instance ensemble stacking deployed on an NVIDIA Jetson Nano GPU [12] and a metaheuristic-optimized, SHAP-explained detector evaluated under edge-like resource budgets [13].

**Feature selection.** Variable-selection theory [14] distinguishes three groups: filter, wrapper, and embedded. Random Forest Importance [6] (embedded) and PCA [15] (unsupervised) are two popular directions for IDS but measure different quantities. SHAP [5] provides consistent local explanations and has been used for IDS [16], but is rarely contrasted directly with RF Importance on the same Spark training set.

**Concept drift, anomaly, and evaluation.** Concept drift [17] affects IDS stability over time. Anomaly-detection surveys [18], [19] systematize unsupervised detectors that are trained on normal data alone, and such detectors, autoencoders in particular, are used both as a pre-filter ahead of a supervised classifier and to reduce feature dimensionality for IDS on edge devices [20], [21]. However, data leakage ("data snooping") and statistical-significance testing (identified as among the most common pitfalls of ML in security [22]) are rarely handled explicitly on CICIDS2017, and we handle both here. Table I summarizes the comparison.

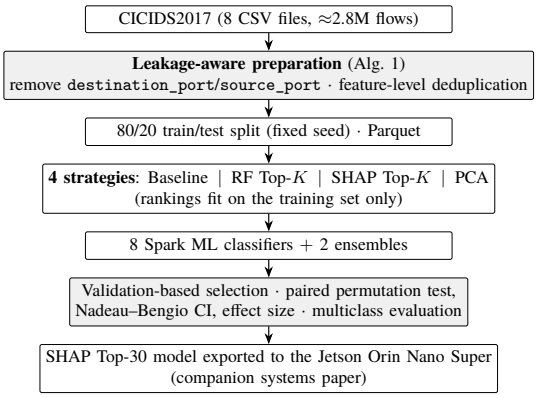



Figure 1. Leakage-aware evaluation pipeline: from raw CICIDS2017 flows to the statistically validated, edge-ready model.

## III. Method

### A. Overview of the Spark ML pipeline

The offline pipeline consists of sequential steps: data cleaning, deduplication, `VectorAssembler`, `StandardScaler`, an (optional) dimensionality-reduction transform, and then the classifier. Binary labels: Benign (0) and Attack (1). The `fit` of StandardScaler/PCA and the feature ranking use the training set alone, and the test set is the final holdout.

### B. Leakage-aware data preparation

We merge the eight CICIDS2017 CSV files, normalize column names, replace infinite/missing values, and then apply two leakage-removal steps *before* splitting the data:

- **Removing the label-leaking feature.** `destination_port` (and `source_port` if present) is removed because it directly encodes the target service of the attack. Keeping it would let the model memorize the port-to-label mapping instead of learning flow behavior. An ablation (keep vs. remove the port) quantifies the impact.
- **Feature-level deduplication (deterministic).** Beyond removing fully duplicate records, we remove flows that are identical across all feature columns before splitting, preventing the same sample from appearing in both the training and test sets. For groups of flows that are feature-identical but have conflicting labels (Benign and Attack, a known error in CICIDS2017 [4]), the entire group is removed rather than arbitrarily keeping one row depending

Table I
CAPABILITY COMPARISON WITH RELATED IDS STUDIES ON CICIDS2017 AND SPARK/EDGE ("—": NOT ADDRESSED; "PARTIAL": DESTINATION_PORT KEPT).

| Reference | Dataset | Feature reduction / explainability | Leakage-aware | Statistical testing | Cross-dataset | Multi-class |
|---|---|---|---|---|---|---|
| Khraisat et al. [2] | Multiple (survey) | General | — | — | — | — |
| Vinayakumar et al. [9] | Multiple (incl. CICIDS2017) | None | — | — | — | Yes |
| Ferrag et al. [7] | Multiple | None | — | — | — | Yes |
| Alrawashdeh & Purdy [8] | KDD Cup 99 | RBM (unsupervised) | — | — | — | Yes |
| Sarhan et al. [16] | Multiple (NetFlow, CICFlowMeter) | SHAP | — | — | — | — |
| Chithra Rani et al. [12] | Multiple (incl. CICIDS2017) | PCA | — | — | — | Yes |
| Nasir et al. [13] | Multiple | GWO + SHAP | — | — | — | Yes |
| Hasan et al. [21] | Edge-IIoTset | Autoencoder | — | — | — | Yes |
| Cantone et al. [23] | CICIDS2017/2018, LycoS | mRMR | Partial | — | Yes | — |
| **The proposed method** | **CICIDS2017, CSE-CIC-IDS2018** | **RF/SHAP/PCA** | **Yes** | **Yes** | **Yes** | **Yes** |

**Algorithm 1** Leakage-aware preparation of CICIDS2017

**Input:** CSV files $\{F_1, \ldots, F_N\}$; split ratio $\alpha$=0.8
**Output:** $D_{\text{train}}$, $D_{\text{test}}$ (Parquet)
$D \leftarrow \bigcup_i F_i$ ▷ normalize names; $\pm\infty \rightarrow$ NaN
$D \leftarrow \textsc{DropDuplicates}(D)$; $D \leftarrow \textsc{DropNaN}(D)$
$y_i \leftarrow 0$ if $\textsc{Upper}(\text{label}_i) = \text{BENIGN}$ else 1
$\mathcal{F} \leftarrow$ numeric columns $\setminus$ {`destination_port`, `source_port`, `protocol`, identifiers}
**for all** groups $g$ of rows identical on all of $\mathcal{F}$ **do**
  **if** labels within $g$ conflict (Benign and Attack) **then**
    drop $g$ entirely ▷ contradictory ground truth; logged
  **else** keep one deterministic representative of $g$
split $D \rightarrow D_{\text{train}}/D_{\text{test}}$ ($\alpha$, fixed seed); write Parquet

on data partitioning. The numbers of removed groups and rows are reported (270 groups, 44,260 rows), so the deduplication result is deterministic and reproducible.

Algorithm 1 summarizes the procedure, and Figure 1 places it in the full evaluation pipeline. After processing, the data are split into **80% training / 20% test** (with a fixed seed), stored in Parquet format, and reused throughout the experiments. The number of remaining numeric features is 77, and the number of samples after deduplication is 1,785,349/447,275 (2,232,624 total after feature-level dedup).

### *C. Classifiers and the four dimensionality-reduction strategies*

Eight algorithms: Decision Tree, Logistic Regression, SVM (LinearSVC), Naive Bayes, Random Forest, GBT, XGBoost, MLP, together with two ensembles (weighted Hybrid Bagging Top-3 and Soft Voting). LightGBM is excluded because its native library (via SynapseML) supports only x86_64 and cannot run on the Jetson Orin Nano Super edge device (ARM64), the deployment target of this study. XGBoost and GBT represent the gradient-boosting family. The four strategies are **Baseline** (all features), **RF Top-$K$** and **SHAP Top-$K$** ($K \in \{20, 30, 40\}$), and **PCA** ($k \in \{20, 30, 40\}$). The RF and SHAP rankings use the training set alone. SHAP is computed on a class-stratified sample so that both benign and attack traffic are represented. PCA is fixed at $k$=40, more than the Top-30 of the selection methods, because PCA extracts rather than selects: each principal component is a linear combination carrying only part of the variance, so more components are needed to retain an equivalent amount of variance. Of $\{20, 30, 40\}$, $k$=40 retains the most variance, so it is the fairest point to represent PCA.

### *D. Hyperparameters and class-imbalance handling*

The hyperparameters (Table II) are fixed a priori and shared across all dimensionality-reduction methods, so that the observed differences reflect the feature-selection method rather than per-model tuning. Hyperparameter tuning is separated into a distinct stage (grid search with cross-validation) and applied only to the selected configuration. The stochastic models (RF, GBT, XGBoost, MLP) fix the `seed` for reproducibility. Class imbalance is handled the same way throughout. Every model that supports per-sample weighting, namely Decision Tree, Logistic Regression, SVM, Naive Bayes, Random Forest, and GBT, is assigned class weights according to the benign/attack ratio, equivalent to XGBoost's `scale_pos_weight` mechanism. MLP alone does not support sample weights in Spark ML and is therefore trained without weights, as noted in the limitations.

### *E. Two-stage comparison design*

To avoid selection bias from trying many configurations on the same test set, we separate the *exploration stage* (sweeping $K \in \{20, 30, 40\}$) from the *confirmation stage* (four configurations fixed a priori: Baseline, RF

Table II
MAIN HYPERPARAMETERS OF THE EIGHT CLASSIFIERS (FIXED A PRIORI).

| Model | Main hyperparameters |
|---|---|
| Decision Tree | maxDepth=15, minInstancesPerNode=5, impurity=entropy |
| Logistic Regression | maxIter=200, regParam=0.001, L2, threshold=0.5 |
| SVM (LinearSVC) | maxIter=200, regParam=0.001 |
| Naive Bayes | gaussian, smoothing=1.0 |
| Random Forest | numTrees=200, maxDepth=15, featureSubset=sqrt |
| GBT | maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8 |
| XGBoost | n_estimators=300, max_depth=8, lr=0.05, subsample=0.8, colsample=0.8 |
| MLP | layers=[$d$,64,32,2], maxIter=150, stepSize=0.01 |
| Class balancing: per-sample class weights (6 models) / positive-class weight scaling (XGBoost), MLP unweighted. | |

Top-30, SHAP Top-30, PCA $k$=40), on which the comparative conclusions rest. No configuration is chosen for its test F1. Within each configuration, the representative model and the tested method pair are chosen by F1 on a validation set (20% held out from the training set, fixed seed), so the test set never participates in selection.

### F. Rigorous evaluation pipeline

**Metrics.** Accuracy, Precision, Recall, F1, AUC-ROC and AUC-PR, plus training/prediction time and model size. Because the data are imbalanced, AUC-PR and attack-class recall are prioritized over accuracy.

**In-distribution stability check.** Each method's representative model is re-evaluated on a test subset, while distribution shift itself is assessed by the cross-dataset experiment (Section IV).

**Statistical-significance testing.** The two top methods are trained on $N$ independent resamplings of the training/test split (default $N$=6), and in each round both use the same split so that the comparison is paired. As a result, the variance reflects the effect of data sampling rather than only of model initialization. The 95% confidence interval is computed using the $t$ distribution ($t_{0.975,N-1}$, not the value 1.96). Because the resamplings overlap in training data, we report alongside it the Nadeau–Bengio [24] corrected confidence interval, with the variance multiplied by $(1/N + n_{\text{test}}/n_{\text{train}})$, and take the corrected interval as the basis for conclusions. Significance is tested by a two-sided paired permutation test on the per-round F1 difference, enumerating all $2^N$ sign patterns exactly rather than sampling them. Its floor is $p = 2/2^N$, so $N \geq 6$ is needed to reach below $0.05$ at all. We report the paired Cohen's $d$ alongside $p$, and test equivalence directly with a TOST, since a non-significant $p$ is by itself weak evidence for it.

**Multiclass evaluation.** Because binary classification on CICIDS2017 is almost perfectly separable, we add a per-attack-type multiclass evaluation, comprising per-class precision/recall/F1, macro-F1, and a confusion matrix. It exposes the rare classes (Web Attack, Bot, Infiltration) that binary F1 hides.

## IV. EXPERIMENTS AND RESULTS

### A. Setup

We use CICIDS2017 with an 80/20 training/test split after leakage removal (Section III-B), Apache Spark 3.4.1 and Java 17, on a cluster of 2× Jetson Orin Nano Super (8 GB) and a coordinator host. The host acts as Spark *master* only and runs no executor, so every training time reported here reflects compute on the two homogeneous Jetson workers rather than the stronger host CPU. Every script logs the run configuration (seed, environment variables, library versions) to ensure reproducibility. The full source code, configuration, and feature lists are publicly available at https://github.com/vuthainguyen1602/Thesis_IDS (branch `develop`). The pipeline can be reproduced on a single machine and does not require an edge-device cluster.

### B. Port-feature ablation

Table III quantifies the impact of `destination_port`, using a Random Forest with the Table II hyperparameters, fixed a priori (class-weighted, binary task). After feature-level deduplication, the port's isolated effect is small. Binary F1 is 0.9964 with the port and 0.9954 without it (recall 0.9988 vs. 0.9961, AUC-PR 0.9999 vs. 0.9997), a gap of ≈0.001: the in-domain binary task is near saturation, and feature-level duplicates, the other leakage source, have already been removed. The direction, slightly higher F1 and recall with the port, is nonetheless consistent with leakage. All subsequent results use the port-removed configuration.

That ≈0.001 gap belongs to the tree family, not to the dataset: across all eight classifiers (Table III) the inflation

spans $+0.0007$ (XGBoost) to $+0.0415$ (SVM), because the linear models lean on the port as a service identifier where the tree ensembles recover the same boundary from behavioral statistics. An ablation on one model does not bound the leak for another model class.

Table III
F1 INFLATION FROM DESTINATION_PORT PER CLASSIFIER (BINARY TASK, BOTH ARMS TRAINED ON THE SAME SPLIT).

| Model | F1 (no port) | F1 (port) | ΔF1 |
|---|---|---|---|
| SVM (LinearSVC) | 0.7859 | 0.8274 | +0.0415 |
| Logistic Regression | 0.7986 | 0.8381 | +0.0395 |
| Naive Bayes | 0.2985 | 0.3093 | +0.0108 |
| MLP | 0.9660 | 0.9711 | +0.0052 |
| GBT | 0.9924 | 0.9941 | +0.0017 |
| Decision Tree | 0.9953 | 0.9966 | +0.0014 |
| Random Forest | 0.9954 | 0.9964 | +0.0010 |
| XGBoost | 0.9972 | 0.9979 | +0.0007 |

### C. Comparison of the four dimensionality-reduction methods

Table IV summarizes the best model per method, and Figure 2 visualizes F1 for each method $\times$ algorithm pair. The comparison design aims to verify whether feature selection (RF/SHAP Top-30) retains near-baseline performance while cutting ≈60% of the features. PCA $k$=40 reduces dimensionality in an unsupervised manner at the cost of explainability.

### D. Statistical testing (paired resampling)

Table V reports the mean F1 over $N$=6 resamplings for the two top methods with both 95% intervals, $t$ and Nadeau–Bengio (N–B), and the paired permutation-test $p$-value. The N–B intervals still overlap. The two methods differ significantly only at the floor of the test's resolution ($p \approx 0.031$, Cohen's $d \approx 1.94$), yet the absolute F1 gap is about 0.0001. A significance test cannot establish equivalence, so we add two one-sided tests (TOST [25]) against a 0.001 F1 margin fixed in advance, on the Nadeau–Bengio error: $p_{\text{TOST}} = 3.9 \times 10^{-6}$, and the tightest margin these six splits support at $\alpha = 0.05$ is $\pm 2.3 \times 10^{-4}$ F1 — equivalence in the positive sense, not mere indistinguishability.

### E. Robustness and multiclass evaluation

In the in-distribution stability check (a subset of the test set), the ranking of the methods is preserved (XGBoost leads in all four methods). The multiclass evaluation (Figure 4, Table VI) shows a wide gap between weighted-F1 (≈0.998) and macro-F1 (≈0.806): the rare classes carry the loss. Reading the confusion matrix by where the errors land separates two failure modes. XSS is mostly mislabeled but still detected: 81/111 samples are predicted as Web Brute Force, so at the binary (deployment) level 76% of XSS flows are still flagged as attacks. The damaging mode is leakage into Benign: all 6 SQL Injection samples, 54% of Bot (158/290), and 27% of Web Brute Force are predicted benign. Three mechanisms plausibly drive this: extreme imbalance (6–311 samples vs. 380k benign), short HTTP attack flows that resemble browsing once the port-80 shortcut is gone, and Bot C&C beaconing that resembles background traffic.

### F. Cross-dataset generalization

To go beyond *in-domain* evaluation, we train on one dataset and test on the other (cross-dataset evaluation, run in both directions) on the leakage-free feature set shared by the two datasets. Because CSE-CIC-IDS2018 [26] is very large (∼16M flows) we use a label-stratified sample (fraction 0.2, fixed seed): ∼3.14M flows, ∼2.39M after feature-level deduplication, against ∼2.23M for CICIDS2017.

The contrast is the sharpest result in the paper: in-domain F1 is high and stable in both directions (0.9953 and 0.9246, CV <0.1%), yet cross-dataset performance collapses, most clearly in the threshold-free AUC-PR, 0.9997 to 0.4849 (the mean of all 18 runs) and 0.9679 to 0.1926. Cross-dataset F1 sits in two bands for 2017→2018 and around 0.0544 in the other direction, at very low recall (0.073 and 0.030) but still-usable precision (0.772 and 0.272): the model misses the other testbed's attacks rather than alarming indiscriminately. For 2017→2018 that F1 is not even single-valued: the 18 runs split into two disjoint regimes with nothing in between. A mean is therefore the wrong summary: 0.1276 falls in the empty gap and its 95% interval holds 1 of the 18 runs. Of five seeds run twice, three stayed in their regime and two switched, so the distributed fit settles it, not the seed. AUC-PR shows no detectable difference between them (0.4444–0.5163 against 0.4532–0.5397, Welch $|t|$=1.3), so the ranking does not change, only where the 0.5 threshold falls against a cluster of near-identical target flows that crosses it as a block. A single-run cross-dataset F1 only reports which regime the run landed in, so we report the distribution and compare on AUC-PR. The causes are concrete: a different CICFlowMeter version (changed feature scales), a different network configuration, and non-overlapping attack tooling (HOIC/LOIC-HTTP exist

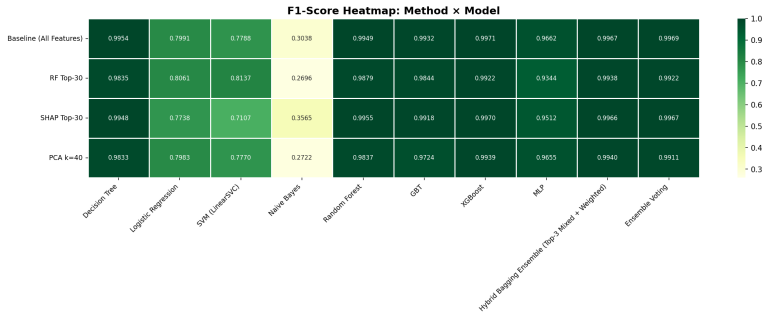



Figure 2. F1 heatmap by method and algorithm (binary, port removed).

Table IV
SUMMARY OF THE BEST MODEL PER METHOD (VALUES FROM THE RELEASED EXPERIMENT OUTPUTS).

| Method | Model | F1 | AUC-PR | Train (s) | Size (MB) |
|---|---|---|---|---|---|
| Baseline | XGBoost | 0.9971 | 0.9999 | 4615.5 | 2.32 |
| RF Top-30 | XGBoost | 0.9922 | 0.9998 | 1295.1 | 2.52 |
| SHAP Top-30 | XGBoost | 0.9970 | 0.9999 | 1226.1 | 0.003[†] |
| PCA $k$=40 | XGBoost | 0.9939 | 0.9998 | 1444.8 | 3.82 |

[†]Sizes are obtained by serializing each fitted model *on the training cluster*. The distributed model-saving step writes the tree-ensemble data partitions on whichever executor holds them, so entries in the KB range (here 0.003 MB) capture only the driver-local metadata and *under-report* the true size. Since the size of a tree ensemble is governed by the number of trees and nodes rather than by the number of input features, the SHAP Top-30 XGBoost model is in reality comparable to its RF Top-30 counterpart (≈2.5 MB). For the same reason, a reduced feature set can even yield a slightly *larger* model than the baseline (2.52 vs. 2.32 MB): with fewer split candidates the trees need more nodes to reach comparable purity. The single-node export of the deployed pipeline measures model sizes reliably.

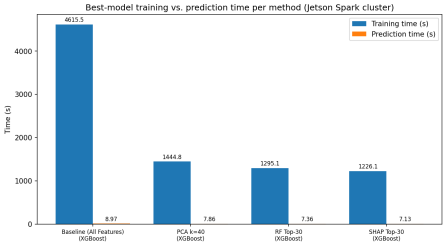



Figure 3. Training and inference time of each method's representative model on the training Spark cluster (Section IV).

Table V
MULTI-RESAMPLING STABILITY AND PAIRED TESTING (VALUES FROM THE RELEASED EXPERIMENT OUTPUTS).

| XGBoost | F1 ± $t$-CI95 | N–B CI95 | $p$ |
|---|---|---|---|
| Baseline | 0.99734 ± 0.00007 | ± 0.00012 | 0.031 |
| SHAP Top-30 | 0.99720 ± 0.00009 | ± 0.00015 | |

only in 2018). Is it merely a scale mismatch? Two classic unsupervised adaptations (refitting the scaler on the target, and CORAL [27], both without target labels or retraining) recover no F1 (0.000 either direction, in three runs), so the shift sits in the conditional distribution, not the first two moments. CORAL does lift AUC-PR

Table VI
PER-CLASS PRECISION/RECALL/F1 (RANDOM FOREST, ALL FEATURES).

| Class | Support | Precision | Recall | F1 |
|---|---|---|---|---|
| Benign | 380,119 | 0.9981 | 0.9997 | 0.9989 |
| DoS Hulk | 34,446 | 0.9990 | 0.9904 | 0.9947 |
| PortScan | 383 | 1.0000 | 0.9530 | 0.9759 |
| Web Attack – Brute Force | 311 | 0.7346 | 0.7299 | 0.7323 |
| Bot | 290 | 1.0000 | 0.4552 | 0.6256 |
| Web Attack – XSS | 111 | 1.0000 | 0.0270 | 0.0526 |
| Infiltration | 12 | 1.0000 | 0.6667 | 0.8000 |
| Web Attack – SQL Injection | 6 | 0.0000 | 0.0000 | 0.0000 |
| 7 remaining classes, all F1 $\geq 0.95$ (see released outputs) | | | | |
| macro-F1 $= 0.806$ | | weighted-F1 $= 0.998$ | | |

Table VII
CROSS-DATASET GENERALIZATION (2017 = CICIDS2017, 2018 = CSE-CIC-IDS2018): 18 RUNS OF IDENTICAL CODE AND DATA (13 FOREST SEEDS, FIVE RUN TWICE). THE 2017→2018 F1 IS BIMODAL, SO THAT DIRECTION IS SPLIT INTO ITS TWO REGIMES, AND THE OTHER ROWS ARE MEANS.

| Train → Test | F1 | AUC-PR |
|---|---|---|
| 2017 → 2017 (in-domain) | 0.9953 | 0.9997 |
| 2017 → 2018 (cross, 11 runs) | 0.029–0.085 | 0.4778 |
| 2017 → 2018 (cross, 7 runs) | 0.185–0.301 | 0.4960 |
| 2018 → 2018 (in-domain) | 0.9246 | 0.9679 |
| 2018 → 2017 (cross) | 0.0544 ± 0.0019 | 0.1926 |

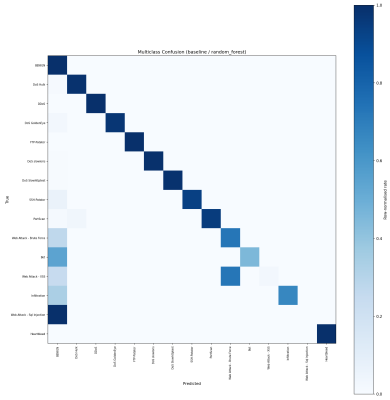


Figure 4. Multiclass confusion matrix (row-normalized).

for 2018→2017 (0.179 to 0.250) with F1 still zero: the ranking improves, the source-fitted 0.5 threshold does not survive, and adaptation here has to recalibrate the threshold as well as the features. Labeling a little of the new network works immediately: retraining on 1% of the target's training split (19k flows) reaches F1 0.92 and 0.99, and pooling them with the 1.79M source flows is worse (0.80, 0.96) than dropping the source data. This collapse is consistent with prior cross-dataset IDS studies [23], [28] and reinforces the central claim of this paper: near-perfect in-domain scores do not certify deployability under distribution shift, so cross-dataset evaluation must be mandatory in IDS validation, and domain adaptation is a prerequisite for practical transfer.

## V. Discussion

**Method comparison.** Baseline and SHAP Top-30 are equivalent within $\pm 2.3 \times 10^{-4}$ F1 (Table V), so secondary criteria decide the deployment choice: explainability, size, stability, feature count. RF Top-30 trails SHAP Top-30 by ≈0.005 F1 at the same feature count, and SHAP Top-30 also yields consistent local explanations for SOC operations.

**Bridge to the edge device.** Cutting ≈60% of the features reduces vector-assembly cost and model footprint on the Jetson Orin Nano Super, so the selection criterion has to weigh feature count, model size and training/prediction time (Figure 3) alongside F1. Latency and throughput on the Jetson appear in a companion systems paper.

**Limitations of the binary evaluation.** Removing the port and deduplicating brings binary F1 down to a realistic level, but binary classification remains a relatively easy task. Only under multiclass evaluation do the weaknesses on rare attack classes emerge, and comparing the reduction methods on it is left to future work.

## VI. Threats to Validity

**Internal validity.** The RF/SHAP rankings are derived from the original training set and kept fixed across resamplings, which follows the "a priori configuration" design but lets the ranking see flows that a later resampling puts in its test split. Re-ranking SHAP inside a resampling is expensive, so we did it for two of the six: F1 moved by at most $7.4 \times 10^{-5}$, in opposite directions, and 25–26 of the 30 features were reselected. The fixed ranking is not inflating F1. The Top-3 component models of the ensemble are selected on that same held-out validation set, at the cost of training the base models on less data. Class balancing applies to all models except MLP (Spark ML offers no sample weights for it), so the MLP result carries that caveat.

**External validity.** CICIDS2017 (2017) may not reflect current traffic, and benchmark NIDS datasets carry documented quality limitations [29], so verification on the full CSE-CIC-IDS2018 and newer datasets (CIC-IoT) is needed. The default random split shuffles time, although a time-based split is also supported. The cross-dataset conclusion (Section IV-F) rests on a stratified sample of CSE-CIC-IDS2018, not the whole set. The model assumes an attack distribution approximating CICIDS2017 and has not been tested against deliberately evasive traffic, and feature-level deduplication itself shifts the benign/attack ratio.

**Statistical validity.** Cross-dataset F1 is not deterministic across runs and is bimodal (Section IV-F), so it is reported over $n{=}18$ runs and each adaptation figure is compared within its own run. The small sample size ($N{=}6$) gives the paired permutation test low power: because the permutation test is two-sided, the smallest attainable $p$ is only $2/2^N$, so $N{=}6$ just barely reaches the $0.05$ threshold and can hardly detect small effects. The equivalence claim does not rest on that $p$-value, however: the TOST of Section III-F tests it directly, against the same Nadeau–Bengio error that accounts for the overlap between resamplings. Increasing $N$ by nested cross-validation is the next step.

## VII. Conclusion

This paper compares four dimensionality-reduction strategies for IDS on Spark ML under a leakage-aware and statistically tested evaluation pipeline. The contribution lies as much in how to evaluate reliably as in which method wins: removing the leaking feature,

deterministic feature-level deduplication, model selection on a validation set, exact-enumeration paired testing with Nadeau–Bengio correction, multiclass evaluation, and cross-dataset generalization between CICIDS2017 and CSE-CIC-IDS2018 (Section IV-F). Future work covers an ablation of the deduplication step (the second leakage source), the full CSE-CIC-IDS2018 and newer datasets (RoEduNet, CIC-IoT), transfer of the SHAP Top-30 set between datasets, and real-time edge deployment.

## Conflict of Interest

The authors declare no conflict of interest.

## References


[1] M. Ring, S. Wunderlich, D. Grüdl, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[2] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.

[3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.

[4] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and É. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Risks and Security of Internet and Systems (CRiSIS 2022), Revised Selected Papers*, pp. 18–33, Springer, 2023.

[5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *Journal of Information Security and Applications*, vol. 50, p. 102419, 2020.

[8] K. Alrawashdeh and C. Purdy, "Toward an online anomaly intrusion detection system based on deep learning," in *Proc. 15th IEEE Int. Conf. on Machine Learning and Applications (ICMLA)*, pp. 195–200, 2016.

[9] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.

[10] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud'10)*, 2010.

[11] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar, "MLlib: Machine learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

[12] P. R. Chithra Rani and K. Baalaji, "Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms," *Discover Applied Sciences*, vol. 7, p. 928, 2025.

[13] K. Nasir, S. K. Badri, D. M. Alghazzawi, M. Y. Alghamdi, M. Alkhozae, A. Almakky, R. M. Alhazmi, and M. Z. Asghar, "HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning," *Scientific Reports*, vol. 16, p. 2313, 2026.

[14] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[15] I. T. Jolliffe, *Principal Component Analysis*. Springer Series in Statistics, Springer, 2nd ed., 2002.

[16] M. Sarhan, S. Layeghy, and M. Portmann, "Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection," *Big Data Research*, vol. 30, p. 100359, 2022.

[17] J. Gama, I. Zliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[18] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[19] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, G. Montavon, W. Samek, M. Kloft, T. G. Dietterich, and K.-R. Müller, "A unifying review of deep and shallow anomaly detection," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 756–795, 2021.

[20] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Conditional variational autoencoder for prediction and feature recovery applied to intrusion detection in IoT," *Sensors*, vol. 17, no. 9, p. 1967, 2017.

[21] T. Hasan, A. Hossain, M. Q. Ansari, and T. H. Syed, "Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning," 2025. arXiv:2501.15266.

[22] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium (USENIX Security 22)*, pp. 3971–3988, 2022.

[23] M. Cantone, C. Marrocco, and A. Bria, "Machine learning in network intrusion detection: A cross-dataset generalization study," *IEEE Access*, vol. 12, pp. 144489–144508, 2024.

[24] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.

[25] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, pp. 657–680, 1987.

[26] Canadian Institute for Cybersecurity and Communications Security Establishment, "CSE-CIC-IDS2018 dataset: A collaborative project between the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC)." https://www.unb.ca/cic/datasets/ids-2018.html, 2018.

[27] B. Sun, J. Feng, and K. Saenko, "Return of frustratingly easy domain adaptation," in *Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI)*, 2016.

[28] L. D'hooge, T. Wauters, B. Volckaert, and F. De Turck, "Inter-dataset generalization strength of supervised machine learning methods for intrusion detection," *Journal of Information Security and Applications*, vol. 54, p. 102564, 2020.

[29] P. Goldschmidt and D. Chudá, "Network intrusion datasets: A survey, limitations, and recommendations," *Computers & Security*, vol. 156, p. 104510, 2025.

# A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Thai Nguyen Vu[1(✉)[0009−0001−9714−1031]], Bui Van Dung[2,3], Tri Nhut Do[2,3], and Van Du Nguyen[4]

[1] Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam
thaivn_ph@utc.edu.vn
[2] Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam
[3] University of Information Technology, Ho Chi Minh City, Vietnam
dungbv.17@grad.uit.edu.vn, trinhutdo@uit.edu.vn
[4] Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam
nvdu@hcmuaf.edu.vn

**Abstract.** Deploying machine learning (ML) based intrusion detection on resource-constrained edge devices requires balancing accuracy, latency, and operational cost. We present a split-deployment IDS architecture across two NVIDIA Jetson Orin Nano Super devices connected through Apache Kafka. The system implements a two-stage pipeline: (1) a lightweight autoencoder anomaly gate on Jetson #1 filters benign traffic, and (2) a PySpark `PipelineModel` classifier on Jetson #2 processes only suspicious flows. Central infrastructure (Kafka, PostgreSQL, InfluxDB, Grafana) runs on a host PC. We evaluate three distributed modes (pipeline split, horizontal Kafka consumer scaling, and a Spark standalone cluster) while streaming CICIDS2017 network flows. Following every flow from send to verdict, queueing included, pipeline split sustains 22 flows/s against 2.5 for a single node running the whole pipeline ($\approx 9\times$), at an end-to-end p95 of about 7 s and 0.60 J per verdict against 3.04 J. Most of the gain comes from decoupling the stages rather than from the second board: the gate run inside the classifier's process gains nothing, while the two stages as separate processes on one board sustain 17.8 flows/s at 0.32 J per verdict. The price is recall. Over all 10,000 replayed flows the deployed pipeline raises no false alarm (precision 100%) but detects 69.5% of attack flows (F1 0.82), and every miss is a flow the gate skipped. On replayed CSE-CIC-IDS2018 traffic the pipeline detects under 1% of attacks and, because the gate forwards three times as many flows, cuts capacity to 8 flows/s.

**Keywords:** Intrusion detection · Edge computing · Jetson Orin Nano Super · Kafka · PySpark · Stream processing.

## 1 Introduction

Network intrusion detection systems (IDS) increasingly must operate closer to data sources to reduce bandwidth and response time [11, 28]. Recent work confirms that

accurate detectors now fit on single-board hardware: ensemble and metaheuristic-optimized detectors have been demonstrated on IoT-edge platforms [6, 21], lightweight models have been designed for deployment on industrial control devices [31], and NVIDIA Jetson-class boards have been used both for autoencoder-based IIoT detection [12] and, on the Jetson Orin Nano specifically, for generative-AI-assisted traffic classification [8]. Such devices nonetheless offer limited CPU, memory, and thermal headroom, so a monolithic detector that classifies every flow saturates the board quickly.

Three gaps persist in this literature. First, the recent edge-IDS studies above evaluate a *single* device: streaming middleware such as Kafka has been paired with Spark for distributed detection [15], but the resulting pipelines are not measured on a multi-node cluster of constrained edge boards with explicit role separation, so it remains unclear how much a second board actually buys and in which distribution mode. Second, throughput, latency percentiles, and per-inference energy are rarely reported together for one deployed system: even recent work framed as energy-aware leaves the energy footprint and on-device latency unevaluated [22]. Third, offline detection scores are increasingly under scrutiny: methodological pitfalls in security ML [2] and documented labeling and duplication errors in CICIDS2017 [17] mean that a model selected under a leaky protocol may be optimistic by several F1 points before it is ever deployed.

We address all three at once. A practical edge IDS must decide *which* model to deploy (small, accurate, explainable, and selected under a leakage-aware protocol) and *how* to distribute inference across nodes under streaming load. We therefore split detection into a lightweight autoencoder gate and a heavier PySpark classifier, place the two stages in different ways on Jetson Orin Nano Super boards, and measure each placement end to end. The building blocks (an autoencoder, Kafka, Spark MLlib) are established. We do not propose a new detector. The contribution is empirical: which placement pays off on real edge hardware, why, and what the gate costs in detection.

**Contributions.** (1) A placement study of a two-stage IDS (autoencoder gate, Spark classifier) on a two-board Jetson cluster: single node with and without the gate, pipeline split on one and on two boards, horizontal Kafka scaling, and a Spark standalone cluster, which shows that decoupling the stages, not the filter alone or the second board, delivers most of the gain; (2) an end-to-end evaluation that follows every flow from send to verdict, reporting sustained capacity, latency percentiles with queueing included, energy per verdict, and the precision, recall and F1 of the deployed pipeline after gating, metrics that offline studies typically omit; (3) a same-model engine baseline (PySpark, NumPy, ONNX Runtime) that locates the cost of serving with Spark on the board; (4) open, reproducible scripts integrated with a leakage-aware Spark ML training pipeline, so the deployed model is the one validated under a rigorous offline protocol. All code and configurations are publicly available[5].

## 2 Related Work

**Edge and on-device intrusion detection.** Edge computing pushes analytics closer to data sources to reduce latency and bandwidth [11, 28], and successive work has shrunk

[5] `https://github.com/vuthainguyen1602/Thesis_IDS` (branch `develop`).

IDS models until they fit single-board hardware. Early systems such as RPiDS [26] showed that a complete detection stack could run on a Raspberry Pi at all. Recent studies instead optimize the detector itself, via deep ensemble stacking on IoT-edge platforms [6], metaheuristic hyper-parameter search combined with explainability [21], and compact models tailored to industrial control traffic [31]. NVIDIA Jetson boards are a recurring target: autoencoder-based feature learning reaches sub-millisecond inference on a Jetson Nano [12], and generative models have been run on the Jetson Orin Nano to classify traffic and synthesize training data at once [8]. These studies share a single-device assumption: the detector is benchmarked in isolation on one board, so how detection work should be placed across several boards, and what a second board actually buys, remain open.

**Streaming and distributed detection pipelines.** Kafka [16] decouples producers from consumers and has become the de-facto ingest layer for streaming analytics, while Spark [30] and its MLlib library [20] supply the distributed training and scoring primitives. Combining the two for intrusion detection is an established pattern. Kiran Kumar et al. [15], for instance, route Kafka streams into Spark classifiers for distributed detection and report accuracy and execution time, but neither the hardware nor its resource use. On an 8 GB ARM board the JVM footprint is itself a design constraint, and per-node CPU, memory, and energy traces are rarely published. More broadly, ML-based IDS surveys and benchmark studies [9,14,29] catalog detection accuracy across dozens of models but seldom report the end-to-end throughput or latency percentiles of a running deployment.

**Two-stage and split inference.** Anomaly-detection surveys [5, 24] systematize the family of detectors that model normal behavior alone. This property is what makes a cascaded design attractive, in which a cheap unsupervised stage filters the bulk of normal data and a costlier supervised stage adjudicates the remainder. Unsupervised deep detectors, deep belief networks [1] and autoencoders [12, 18], are the usual choice for the cheap stage, since they can be calibrated on benign traffic alone. Distributing detection across devices has so far been explored mainly along the federated-learning axis, where nodes collaborate during training while inference stays local [13, 23]. Our split is orthogonal and concerns inference: the two stages are physically separated across boards and joined by a Kafka topic, so gate and classifier scale independently, and no prior work applies a PySpark MLlib classifier in this role on the NVIDIA Jetson Orin Nano Super. Placement matters for both cost and latency: recent energy-aware edge IDS work names slow per-sample inference on small boards as the barrier to strict real-time use [22], and skipping the heavy stage is the most direct way to cut that cost.

**Evaluation protocol.** A parallel line of work questions how faithfully offline scores transfer to deployment. Arp et al. [2] catalog recurring pitfalls in security ML, and Lanvin et al. [17] document labeling and duplication errors in CICIDS2017 [27] that measurably inflate reported performance. Sarhan et al. [25] similarly show that the choice of standard feature set shapes both generalizability and explainability. Our offline selection responds by choosing a leakage-free SHAP Top-30 model [19], and the present paper deploys exactly that model, so the operational numbers reported here do not belong to a configuration that would fail a stricter offline audit. Concept drift [10] lies outside the present scope and motivates the retraining direction we outline in the conclusion.

Table 1 positions our system against representative related work along the five axes above: multi-node role separation, streaming ingest, joint energy and latency reporting, leakage-aware model selection, and open code.

**Table 1.** Capability comparison with prior edge IDS deployments

| Reference | Platform | Multi-node roles | Streaming ingest | Energy & p95 latency | Leakage-aware model | Open code |
|---|---|---|---|---|---|---|
| Sforzin et al. [26] | Raspberry Pi | No | No | — | — | — |
| López-Martín et al. [18] | Commodity PC (CPU) | No | No | — | — | — |
| Nguyen et al. [23] | IoT gateways | Federated | No | — | — | — |
| Hasan et al. [12] | 1× Jetson Nano | No | No | — | — | — |
| Chithra Rani and Baalaji [6] | 1× Jetson Nano | No | No | — | — | — |
| Nasir et al. [21] | Edge-like (simulated) | No | No | — | — | — |
| **The proposed system** | **2× Jetson Orin Nano S.** | **Yes** | **Yes (Kafka)** | **Yes** | **Yes** | **Yes** |

"—": not reported or not applicable.

## 3 System Architecture

### 3.1 Split Deployment Overview

Fig. 1 shows the split-deployment architecture. A host PC runs Kafka, PostgreSQL, InfluxDB, and Grafana. Two Jetson Orin Nano Super nodes consume messages from the network-flow topic. In pipeline-split mode, Jetson #1 runs the anomaly gate and forwards only suspicious flows to a dedicated suspicious-flow topic. Jetson #2 then applies the PySpark classifier and writes the classification results back to PostgreSQL. Both nodes stream performance metrics to InfluxDB/Grafana and raise alerts via Email/Slack (dashed arrows in Fig. 1).

### 3.2 Pipeline Roles

- **Anomaly gate:** a lightweight autoencoder + scaler over all 77 leak-free flow features. The MSE threshold is calibrated offline as a quantile (0.995) of a held-out benign validation split (10% of benign training flows, never the test set), so the operating point is committed before evaluation.
- **Classifier:** a Random Forest PySpark `PipelineModel` over the leakage-free SHAP Top-30 feature set (Sect. 5.2).
- **Combined role:** gate and classifier in one process, used for horizontal scaling via Kafka consumer groups.

Each tier scores flows in batches of 20, or sooner once the oldest waiting flow has been held for 1 s, so the classifier, which receives only a small share of traffic, never holds a partial batch indefinitely.

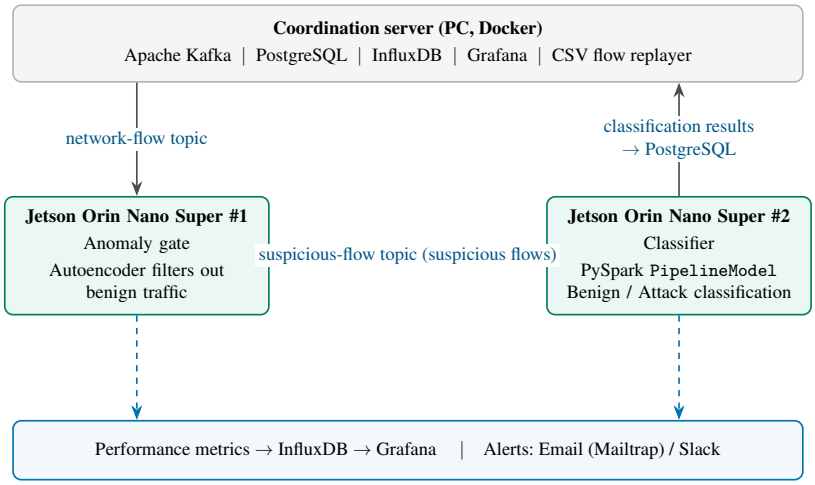



**Fig. 1.** Split-deployment architecture (PC + 2× Jetson Orin Nano Super)

### 3.3 Monitoring and Alerting

Each node tags its metrics with a unique node identifier in InfluxDB and PostgreSQL. Alerts are emitted from a designated primary node only, avoiding duplicate notifications. Fig. 2 shows the Grafana dashboard during the 500 s run that replays all 10,000 CICIDS2017 test flows through the split deployment (Sect. 7): prediction throughput, detected-attack count, inference latency, and per-node CPU/RAM utilization are all tracked in real time, alongside the most recent attack alerts persisted to PostgreSQL. Its CPU panel shows the load asymmetry: at peak load the gate node idles at 1.3% CPU while the PySpark classifier node reaches 94.5%, so the classifier dominates the compute budget.

## 4 Implementation

Training and model export run on a Spark standalone cluster (host PC as master, two Jetson Orin Nano Super devices as workers) using Apache Spark [30] and its MLlib library [20]. The cluster topology is shown in Fig. 3. The host only coordinates (Spark master) and does not run an executor, so all reported training compute reflects the two homogeneous Jetson workers rather than the stronger host CPU. The edge stack is implemented in Python with configurable per-node roles. Every offline run logs its configuration (random seed, leakage/dedup switches, library versions) so each exported model is traceable to the exact pipeline that produced it.

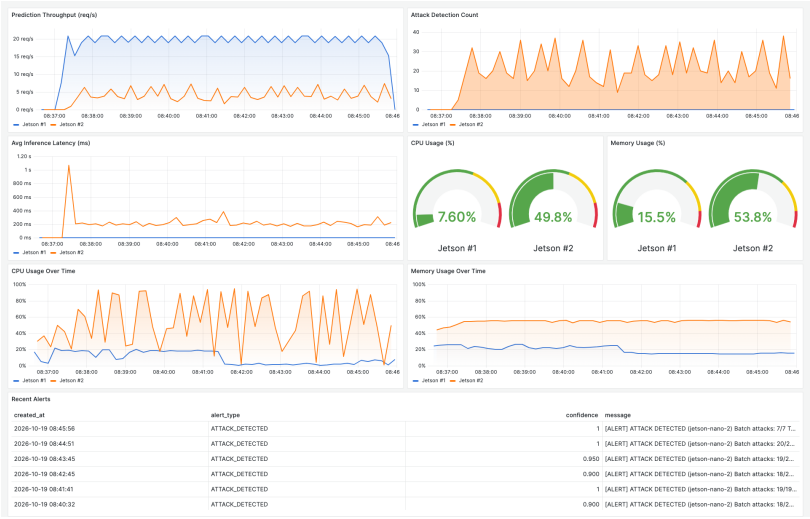



**Fig. 2.** Real-time Grafana monitoring of the edge IDS during CICIDS2017 replay

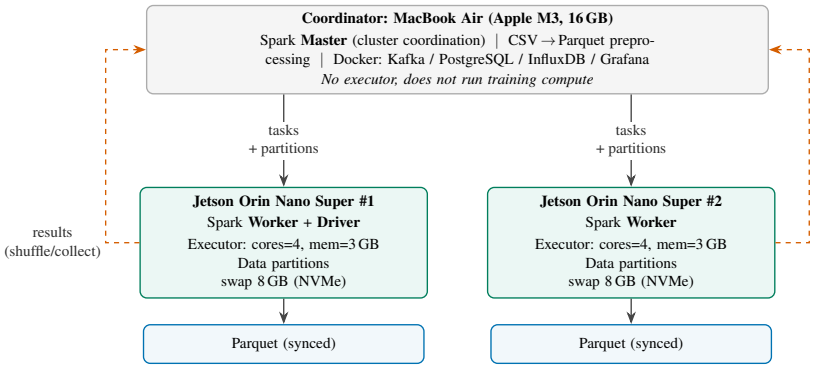



**Fig. 3.** Apache Spark standalone training cluster: the host as *master* (no executor), two Jetson Orin Nano Super devices as *worker*/*executor* nodes

## 5 From Spark Training to Edge Deployment

This paper does not compare Spark against Jetson Orin Nano Super as competing platforms. The two components address complementary phases of the same train–deploy pipeline (Fig. 4). Spark answers which model and how many features achieve acceptable accuracy on the full CICIDS2017 dataset [27]. The Jetson Orin Nano Super answers whether that model can then run in real time under memory, thermal, and bandwidth constraints at the network edge [28].

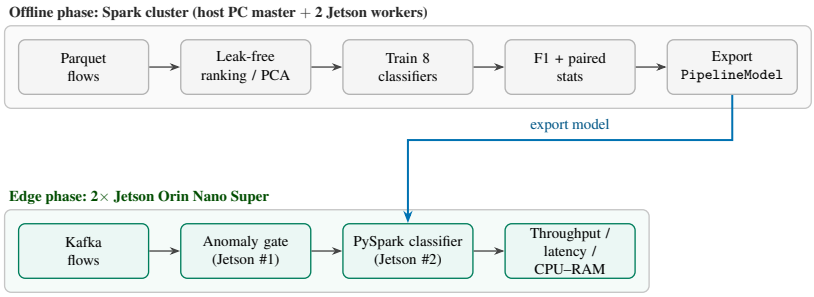



**Fig. 4.** Two-phase train–deploy pipeline linking Spark ML experiments to Jetson edge inference

### 5.1 Offline Phase (Spark ML), Leakage-Aware

In the offline selection stage, we evaluate four feature configurations (baseline with all features, RF Importance Top-30, SHAP Top-30, and PCA with $k$=40) under a fixed 80/20 train–test split, using eight Spark ML classifiers and statistical validation (repeated paired resampling and permutation tests). LightGBM is excluded because its SynapseML native libraries are x86_64-only and cannot run on the ARM64 Jetson Orin Nano Super deployment target. The offline protocol is leakage-aware [2]: it removes information that would be unavailable at deployment time but that otherwise inflates test scores. Motivated by documented labeling and duplication errors in CICIDS2017 [17], the label-leaking `destination_port` feature is removed and flows that are identical on all feature columns are de-duplicated before splitting. This yields realistic F1 values (Table 2) rather than the near-perfect scores obtained when these leaks are left in place. Offline metrics constrain which exported model is worth deploying. They are not directly comparable to edge throughput or latency.

**Table 2.** Offline choice of the feature set: test F1 of XGBoost and of Random Forest (RF), the deployed classifier

| Feature set | Features | XGBoost F1 | RF F1 | XGBoost size (MB) | Edge rationale |
|---|---|---|---|---|---|
| All features (baseline) | 77 | **0.9971** | 0.9949 | 2.32 | Reference |
| **SHAP Top-30** | 30 | 0.9970 | **0.9955** | 2.37[†] | Explainable, deployed |
| RF-importance Top-30 | 30 | 0.9922 | 0.9879 | 2.52 | Embedded ranking |
| PCA $k$=40 | 40 | 0.9939 | 0.9837 | 3.82 | Unsupervised |

The SHAP ranking comes from TreeExplainer on the 77-feature XGBoost model. [†]Measured on a single-node retrain of the same configuration (test F1 0.9971 against 0.9970 in the original run): the distributed save on the training cluster kept only driver-side metadata for this model.

### 5.2 Edge Phase (Jetson Orin Nano Super)

The exported `PipelineModel` (VectorAssembler, StandardScaler, classifier) is loaded on Jetson #2 with only the selected SHAP features, reducing assembly and inference cost relative to the 77-feature baseline. The deployed input vector contains the 30 SHAP-selected features. The ranking is computed after `destination_port` has been removed as a label-leaking channel, consistent with the leakage-aware offline protocol. A lightweight autoencoder gate on Jetson #1 filters benign flows before they reach Spark inference, further lowering load on the classifier node.

We deploy the SHAP Top-30 feature set on Jetson #2 with a Random Forest classifier. SHAP Top-30 is the Random Forest's best feature set in Table 2: with 30 of 77 features it scores F1 0.9955 offline (0.9959 for the exported model), less than 0.002 below the best XGBoost score. Random Forest is also a native Spark ML estimator, so the exported `PipelineModel` loads on the ARM board without the native libraries that Spark's XGBoost integration requires. Sect. 7 then evaluates this choice under streaming load. The full four-method comparison and drift analysis are beyond the scope of this paper.

## 6 Experimental Setup

The testbed comprises two NVIDIA Jetson Orin Nano Super Developer Kits and a host PC running Docker Compose, connected over a Wi-Fi LAN. Each Jetson board pairs a 6-core Arm Cortex-A78AE CPU with an NVIDIA Ampere GPU featuring 1024 CUDA and 32 Tensor cores, delivering roughly 67 TOPS, and is equipped with 8 GB of LPDDR5 memory and a 256 GB NVMe SSD. On the software side, streaming inference runs on PySpark 3.4.1 over Apache Kafka 3.5 (Confluent Platform 7.5.0), while InfluxDB and Grafana collect and visualize runtime metrics. The workload replays 10,000 flows sampled from the CICIDS2017 test split (15.2% attacks, never seen in training). We compare a single node running the full pipeline, with and without the gate, against the three distributed modes A (pipeline split), B (horizontal Kafka scaling, full role on both boards) and C (Spark standalone cluster, host as master). A further placement, A1, runs the pipeline split on one board, with gate and classifier as two separate processes on Jetson #2. A1 was measured in a later session with the same boards, models, replay file and network.

Every verdict stores the flow's producer timestamp and replay row, so each flow sent during a load window is followed to its verdict wherever it is issued. We report *sustained capacity*, the highest offered rate at which every flow sent in a 45 s window receives its verdict within 10 s of the window's end (about one PySpark batch); send-to-verdict latency p50/p95 over those flows, queueing included (the offset between the host and board clocks is measured before and after every window and subtracted); the gate-skip ratio and the confusion counts of the final verdicts computed from the flows themselves; and energy per verdict from `tegrastats` at the sustained rate. Each rate is run twice after a warm-up, with a fresh Kafka consumer group per run so no run inherits another's backlog. Because every window replays the sample from its first row, the capacity windows cover only its first few thousand flows. For the detection metrics,

one additional 500 s run of mode A at 20 flows/s, below its sustained rate, replays all 10,000 flows once.

## 7 Results

A host-side script starts the pipeline roles on the Jetsons over SSH, drives the load and collects the per-flow records. A sweep raises the offered rate per mode until flows stop finishing within the 10 s bound. Table 3 and Fig. 5 summarize the sustained operating point of each mode.

**Table 3.** Sustained operating point per deployment mode on the 2-node Jetson cluster: the highest rate at which every flow got its verdict within 10 s of the load window, with latency measured from send to verdict

| Mode | Boards | Sustained (flows/s) | e2e p50 / p95 (s) | Gate skip (%) | J / verdict |
|---|---|---|---|---|---|
| Single node, gate off | 1 | 2.5 | 5.9 / 11.3 | – | 3.04 |
| Single node + gate | 1 | 2.0* | 3.8 / 7.8 | 89.0 | 3.04 |
| B: Horizontal, gate off | 2 | 3.3 | 4.1 / 8.2 | – | 3.20 |
| C: Spark cluster | 2 | $\geq$8.7† | 0.7 / 5.2 | 88.6 | 1.15 |
| A1: Split, one board | 1 | 17.8‡ | 0.6 / 5.3 | 89.0 | 0.32 |
| A: Pipeline split | 2 | 22.4 | 1.0 / 7.1 | 88.8 | 0.60 |

Energy is total-board power over verdict rate, measured at 2.0, 2.0, 3.9, 9.5, 17.8 and 18.4 flows/s respectively. Both boards are summed in two-node modes. Gate skip is pooled over all runs of a mode. Detection metrics are in Table 5. ‡Delivered at an offered 20 flows/s. At 30 flows/s both repeats drained late (15.2 and 16.1 s). *At 3 flows/s the single node with and without the gate both deliver 2.4–2.5 verdicts/s, and the gated one falls to the 2 flows/s step only because one of its two repeats drained in 10.6 s. †Lower bound: at 20 flows/s one repeat kept up at 17.1 flows/s and the other drained in 12.3 s.

**Capacity.** Pipeline split sustains 22.4 flows/s, about 9$\times$ the single node that runs the whole pipeline (2.5 flows/s). The gate alone does not explain the gain: with the gate added to the single node, where gate and classifier run one after the other in the same process, it shows no measurable gain (2.4–2.5 verdicts/s at 3 flows/s, the same as without the gate), because each batch still waits for a PySpark call that costs seconds regardless of how few flows reach it. Pipeline split decouples the two stages, so the gate keeps consuming while the classifier works through its share. Decoupling, not the second board, accounts for most of the gain: the same split as two processes on one board (A1) sustains 17.8 flows/s, 7$\times$ the single node, and the second board adds about a quarter on top. Horizontal scaling of the full role (gate off) stays Spark-bound at 3.3 flows/s. The Spark standalone cluster sustains at least 8.7 flows/s. At 20 flows/s one of two repeats kept up (17.1 flows/s) and the other did not, so its capacity lies between about 9 and 17 flows/s, below pipeline split and with an extra host dependency.

**Latency.** At its sustained rate, pipeline split returns a median verdict in about 1 s, dominated by gate-only verdicts. Flows forwarded to the classifier wait for a PySpark

batch, and the p95 over all flows is 7.1 s. Every mode shows a multi-second tail for the same reason: one PySpark batch on the board takes seconds regardless of its size.

**Energy efficiency.** Power is sampled with `tegrastats` (30 s idle baseline, then the load window). Serving 18.4 flows/s on two boards, pipeline split spends 0.60 J per verdict against 3.04 J for a single node at its 2 flows/s, a 5× reduction. On one board, A1 needs only 0.32 J per verdict at 17.8 flows/s, since the second board's idle draw disappears. The active (idle-subtracted) power rises by 1.4–1.6 W under load in every mode except horizontal scaling, where both boards run Spark (2.6 W).

**Inference-engine baseline (cost of the unified stack).** We deploy a Spark `PipelineModel` on the edge for consistency with the distributed training phase: the same stack trains on the cluster and serves on the device. To quantify the price of that unification, we export the deployed forest to NumPy and ONNX Runtime and serve all three through the edge's own inference layer on the classifier node, batch size 10 (Table 4). The three engines return identical predictions on every replayed flow. ONNX Runtime serves the same forest at about 10,100 flows/s, about 7,200× the PySpark path, at $<0.01\%$ of its active energy per inference. The gap motivates exporting the `PipelineModel` to ONNX/TensorRT for serving while retaining Spark for training. Wiring the ONNX session into the streaming pipeline is left as future work. A batch-size sweep on the same board shows the PySpark cost is a fixed per-call overhead (7.1–7.4 s from batch 10 to 1,000), not the trees. Larger batches narrow the gap to about 27× at 10,000 flows, but at 20 flows/s collecting such a batch takes over 8 minutes.

**Table 4.** Single-node inference cost of the deployed forest (same trees, identical predictions) across engines, batch size 10

| Engine | Throughput (flows/s) | Batch p95 (ms) | Active energy/inf. (mJ) |
|---|---|---|---|
| PySpark (deployed) | 1.4 | 7515 | 1506 |
| NumPy | 54.8 | 194 | 11.3 |
| ONNX Runtime | 10,092 | 1.2 | 0.087 |

**Gate trade-off.** The anomaly gate is tunable rather than a single fixed point: sweeping its threshold trades attack recall against the fraction of flows offloaded from Spark (Fig. 6). At the pre-committed quantile of 0.995 it forwards 68.1% of attack flows at a 0.49% false-positive rate while skipping 89.4% of all flows offline. Flows the gate skips are final, so the gate bounds the system's recall. Scoring all 77 features matters: restricted to the classifier's 30 SHAP features, the same gate forwards only 44.8% of attack flows at the same quantile, because a subset chosen to separate the classes is not the one that exposes anomalies.

**End-to-end detection after gating.** Table 5 scores the final verdicts of the deployed pipeline against the replay labels, with gate-only verdicts counted as benign. In the run that replays all 10,000 flows once (mode A at 20 flows/s, sustained for 500 s with a 5.9 s drain and an end-to-end p95 of 5.1 s), the pipeline detects 1,055 of 1,518 attack flows (recall 69.5%) and raises no false alarm on 8,483 benign flows, so F1 is 0.82, against

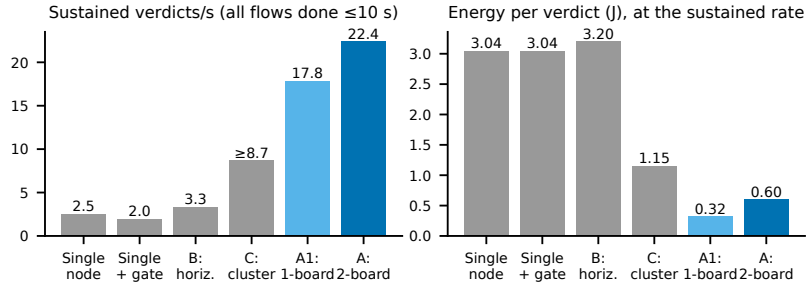



**Fig. 5.** Sustained verdict throughput (left) and energy per verdict at that rate (right) per deployment mode. The Spark-cluster bar is a lower bound (Table 3).

0.996 offline for the classifier alone. All 463 missed attacks were skipped by the gate. The classifier made no error on the flows it received. Every placement with the gate lands at the same point (recall 68–73%, no false positive), and without the gate the deployed pipeline makes no error on the windows replayed.

**Table 5.** End-to-end detection of the deployed pipeline: final verdicts against replay labels, gate-only verdicts counted as benign

| Mode | Verdicts | Attack | FN gate / clf | FP | Recall (%) | F1 |
|---|---|---|---|---|---|---|
| **A: full replay (10,000 flows)** | **10,001** | **1,518** | **463 / 0** | **0** | **69.5** | **0.820** |
| A: capacity windows | 19,543 | 2,990 | 898 / 0 | 0 | 70.0 | 0.823 |
| A1: split, one board | 5,858 | 882 | 278 / 0 | 0 | 68.5 | 0.813 |
| C: Spark cluster | 2,499 | 381 | 115 / 0 | 0 | 69.8 | 0.822 |
| Single node + gate | 1,480 | 200 | 55 / 0 | 0 | 72.5 | 0.841 |
| Gate off (single, B) | 1,855 | 242 | – / 0 | 0 | 100.0 | 1.000 |

Precision is 100% in every row (no benign flow received an attack verdict). Capacity-window rows pool every repeat of a mode. Each window replays the sample from its first row, so these rows overlap and cover 3,601 (A), 1,351 (A1), 829 (C) and 219 (single node + gate) distinct flows. The full-replay row covers all 10,000 once (the sender's loop repeats one row).

## 8 Discussion

Two findings qualify the design. First, decoupling the stages matters more than the filter or the hardware: the gate in the classifier's process gains nothing, while a second board adds only a quarter more capacity at twice the energy per verdict, so one board running the split is the more efficient choice and a second board buys headroom. Second, the throughput is paid for in recall, and Table 5 shows the gate's skips are the pipeline's only errors, so an operator should trade recall for capacity knowingly along the curve of Fig. 6.

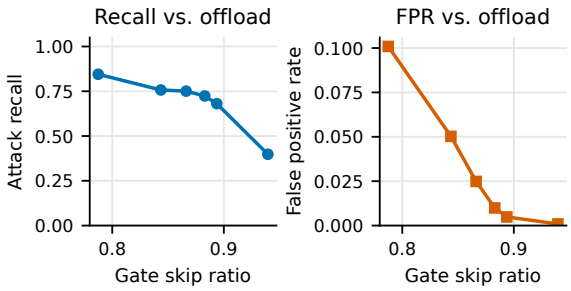



**Fig. 6.** Anomaly-gate operating curve: attack recall (left) and false-positive rate (right) vs. gate-skip ratio

**Dependence on the host.** The boards run all detection, but Kafka, PostgreSQL, InfluxDB and Grafana run on the host PC, so the deployment is not autonomous: flows enter through the host's broker and verdicts are stored there. If the host fails, detection stops, even though both boards remain healthy. With the PySpark classifier the host is not the bottleneck: the boards saturate first, and a round trip from host to board averages about 5 ms (3.6–9.5 ms per measurement) on the Wi-Fi LAN. The coupling still bounds scaling: every added board is one more consumer of the same broker and one more writer to the same database, one insert per verdict. A field deployment would therefore move ingest to the edge: a broker on, or next to, the board that receives the flow export, with verdicts buffered locally and forwarded to a central store in batches, so that detection keeps running when the uplink or the host is lost. Kafka's replication would let such a broker be spread over the boards themselves. These unmeasured variants trade the JVM broker's memory on an 8 GB board against independence from the host.

**Beyond CICIDS2017.** To test the deployed system on another network, we replayed 10,000 flows sampled from the deduplicated, leak-free test split of CSE-CIC-IDS2018 [3] (12.8% attacks, columns mapped to the CICIDS2017 names) through the same pipeline split, keeping the models trained on CICIDS2017 (Table 6). The gate, calibrated on CICIDS2017 benign traffic, forwards 30% of the 2018 flows instead of 11%, so the classifier receives three times the load and capacity falls from 22.4 to about 8 flows/s. Detection collapses to a recall of 0.8%, and the classifier itself, not only the gate, misses almost every 2018 attack. An offline study with the same model family (class-weighted Random Forest, same leakage-aware protocol, shared features, 18 runs) agrees with earlier inter-dataset studies [4, 7]: across datasets AUC-PR falls to 0.48 (2017→2018) and 0.19 (2018→2017), while training on 1% of the target dataset's labelled flows restores F1 to 0.92 and 0.99. A new network therefore needs the gate recalibrated on local benign traffic and the classifier retrained.

**Limitations.** Benchmarks are obtained on a two-node lab cluster over Wi-Fi. Wired networking and larger clusters may shift latency tails. Capacity is bounded by the PySpark classifier, whose per-batch cost of several seconds sets both the tail latency and the ceiling. The ∼67-TOPS GPU remains unexploited by the CPU-only pipeline. The gate bounds recall, and its threshold is calibrated on benign training data only. Energy figures subtract a single pre-run idle baseline, so within-run drift in background power is

**Table 6.** The deployed pipeline split (models trained on CICIDS2017) on replayed CICIDS2017 and CSE-CIC-IDS2018 flows

| Replayed flows (10,000 each, once) | CICIDS2017 | CSE-CIC-IDS2018 |
|---|---|---|
| Flows forwarded by the gate (benign forwarded) | 11.0% (0.6%) | 30.4% (28.4%) |
| Sustained capacity (flows/s) | 22.4 | ≈8* |
| Attack flows / detected | 1,518 / 1,055 | 1,276 / 10 |
| Missed: skipped by gate / by classifier | 463 / 0 | 712 / 554 |
| End-to-end recall / F1 (precision 100%) | 69.5% / 0.820 | 0.8% / 0.016 |
| Classifier alone, offline recall | 99.8% | 1.8% |

*A 1,250 s run at 8 flows/s drained in 1.9 s. At 12 flows/s both repeats drained late (18.5 and 28.9 s). The classifier-alone row scores the same forest offline through its ONNX export (identical predictions).

not captured. Live traffic and deliberately evasive (adversarial) traffic remain untested, and the host-side services were not moved to the edge.

## 9 Conclusion

We presented a reproducible distributed edge IDS for Jetson Orin Nano Super clusters that pairs Kafka streaming with PySpark inference and deploys a model validated under a leakage-aware offline protocol. Measured from send to verdict with queueing included, placing a lightweight autoencoder gate and the Spark classifier on separate boards sustains 22.4 flows/s against 2.5 for a single node running the whole pipeline (≈9×), at an end-to-end p95 of about 7 s and 0.60 J per verdict against 3.04 J. The gain requires decoupling the stages: the same gate run in the classifier's process brings no measurable gain, whereas the two stages as separate processes on one board already sustain 17.8 flows/s at 0.32 J per verdict. It is paid for in recall. Over all 10,000 replayed flows the pipeline raises no false alarm but detects 69.5% of attacks (F1 0.82), and every miss is a flow the gate skipped. Horizontal scaling and the Spark standalone cluster stay below pipeline split. On CSE-CIC-IDS2018 replay the gate forwards three times as many flows and the pipeline detects under 1% of attacks, so a new network needs a recalibrated gate and a retrained classifier. Serving the same forest, ONNX Runtime is about 7,200× faster than the deployed PySpark path at its batch size of 10, with identical predictions, a concrete target for the next iteration of the serving layer. Future work includes TensorRT optimization, live-traffic validation, and federated retraining under drift [13].

**Disclosure of Interests.** The authors declare that they have no competing interests.

## References

1. Alrawashdeh, K., Purdy, C.: Toward an online anomaly intrusion detection system based on deep learning. In: Proc. 15th IEEE Int. Conf. on Machine Learning and Applications (ICMLA). pp. 195–200 (2016). https://doi.org/10.1109/ICMLA.2016.0040

2. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 3971–3988 (2022)
3. Canadian Institute for Cybersecurity and Communications Security Establishment: CSE-CIC-IDS2018 dataset: A collaborative project between the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC). `https://www.unb.ca/cic/datasets/ids-2018.html` (2018), last accessed 2026/10/19
4. Cantone, M., Marrocco, C., Bria, A.: Machine learning in network intrusion detection: A cross-dataset generalization study. IEEE Access **12**, 144489–144508 (2024). https://doi.org/10.1109/ACCESS.2024.3472907
5. Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection: A survey. ACM Computing Surveys **41**(3), 1–58 (2009). https://doi.org/10.1145/1541880.1541882
6. Chithra Rani, P.R., Baalaji, K.: Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms. Discover Applied Sciences **7**, 928 (2025). https://doi.org/10.1007/s42452-025-06871-z
7. D'hooge, L., Wauters, T., Volckaert, B., De Turck, F.: Inter-dataset generalization strength of supervised machine learning methods for intrusion detection. Journal of Information Security and Applications **54**, 102564 (2020). https://doi.org/10.1016/j.jisa.2020.102564
8. Diaz-Gorrin, J., Caballero-Gil, C., Brankovic, L.: Enhancing network security with generative AI on Jetson Orin Nano. Applied Sciences **16**(3), 1442 (2026). https://doi.org/10.3390/app16031442
9. Ferrag, M.A., Maglaras, L., Moschoyiannis, S., Janicke, H.: Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. Journal of Information Security and Applications **50**, 102419 (2020). https://doi.org/10.1016/j.jisa.2019.102419
10. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. ACM Computing Surveys **46**(4), 44:1–44:37 (2014). https://doi.org/10.1145/2523813
11. Hamdan, S., Ayyash, M., Almajali, S.: Edge-computing architectures for Internet of Things applications: A survey. Sensors **20**(22), 6441 (2020). https://doi.org/10.3390/s20226441
12. Hasan, T., Hossain, A., Ansari, M.Q., Syed, T.H.: Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning. In: Proceedings of the 10th International Conference on Internet of Things, Big Data and Security (IoTBDS). pp. 207–214. SciTePress (2025). https://doi.org/10.5220/0013203700003944
13. Hernandez-Ramos, J.L., Karopoulos, G., Chatzoglou, E., Kouliaridis, V., Marmol, E., Gonzalez-Vidal, A., Kambourakis, G.: Intrusion detection based on federated learning: A systematic review. ACM Computing Surveys **57**(12), 1–65 (2025). https://doi.org/10.1145/3731596
14. Khraisat, A., Gondal, I., Vamplew, P., Kamruzzaman, J.: Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity **2**, 20 (2019). https://doi.org/10.1186/s42400-019-0038-7
15. Kiran Kumar, K.M., Srikar Reddy, M.V., Ullas, K., Supriya, M.: Distributed intrusion detection system using Kafka and Spark streaming. In: 2025 International Conference on Visual Analytics and Data Visualization (ICVADV). pp. 302–309. IEEE (2025). https://doi.org/10.1109/ICVADV63329.2025.10961691
16. Kreps, J., Narkhede, N., Rao, J.: Kafka: a distributed messaging system for log processing. In: Proceedings of the NetDB Workshop (2011)
17. Lanvin, M., Gimenez, P.F., Han, Y., Majorczyk, F., Mé, L., Totel, É.: Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes. In: Kallel, S., Jmaiel, M., Zulkernine, M., Hadj Kacem, A., Cuppens, F., Cuppens, N. (eds.) Risks and Security of Internet and Systems (CRiSIS 2022). Lecture Notes in Computer

Science, vol. 13857, pp. 18–33. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-31108-6_2
18. Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A., Lloret, J.: Conditional variational autoencoder for prediction and feature recovery applied to intrusion detection in IoT. Sensors **17**(9), 1967 (2017). https://doi.org/10.3390/s17091967
19. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17). pp. 4768–4777. Curran Associates (2017)
20. Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai, D., Amde, M., Owen, S., Xin, D., Xin, R., Franklin, M.J., Zadeh, R., Zaharia, M., Talwalkar, A.: MLlib: Machine learning in Apache Spark. Journal of Machine Learning Research **17**(34), 1–7 (2016)
21. Nasir, K., Badri, S.K., Alghazzawi, D.M., Alghamdi, M.Y., Alkhozae, M., Almakky, A., Alhazmi, R.M., Asghar, M.Z.: HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning. Scientific Reports **16**, 2313 (2026). https://doi.org/10.1038/s41598-025-32183-8
22. Nassef, L., Alghamdi, M.I., Chaabane, S.B., Abbas, Q., Alawad, W.M., Albalawi, O.H., Alqaisi, O.I., Fakieh, B.: Lightweight and energy-aware intrusion detection for industrial IoT using TinyML and edge AI. Scientific Reports **16**(1), 18524 (2026). https://doi.org/10.1038/s41598-026-50690-0
23. Nguyen, T.D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., Sadeghi, A.R.: DÏoT: A federated self-learning anomaly detection system for IoT. In: Proc. IEEE 39th Int. Conf. on Distributed Computing Systems (ICDCS). pp. 756–767 (2019). https://doi.org/10.1109/ICDCS.2019.00080
24. Ruff, L., Kauffmann, J.R., Vandermeulen, R.A., Montavon, G., Samek, W., Kloft, M., Dietterich, T.G., Müller, K.R.: A unifying review of deep and shallow anomaly detection. Proceedings of the IEEE **109**(5), 756–795 (2021). https://doi.org/10.1109/JPROC.2021.3052449
25. Sarhan, M., Layeghy, S., Portmann, M.: Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection. Big Data Research **30**, 100359 (2022). https://doi.org/10.1016/j.bdr.2022.100359
26. Sforzin, A., Gómez Mármol, F., Conti, M., Bohli, J.M.: RPiDS: Raspberry Pi IDS — a fruitful intrusion detection system for IoT. In: Proc. Int. IEEE Conf. on Ubiquitous Intelligence & Computing (UIC/ATC/ScalCom). pp. 440–448 (2016). https://doi.org/10.1109/UIC-ATC-ScalCom-CBDCom-IoP-SmartWorld.2016.0080
27. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP). pp. 108–116. SciTePress (2018). https://doi.org/10.5220/0006639801080116
28. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. IEEE Internet of Things Journal **3**(5), 637–646 (2016). https://doi.org/10.1109/JIOT.2016.2579198
29. Vinayakumar, R., Alazab, M., Soman, K.P., Poornachandran, P., Al-Nemrat, A., Venkatraman, S.: Deep learning approach for intelligent intrusion detection system. IEEE Access **7**, 41525–41550 (2019). https://doi.org/10.1109/ACCESS.2019.2895334
30. Zaharia, M., Chowdhury, M., Franklin, M.J., Shenker, S., Stoica, I.: Spark: Cluster computing with working sets. In: Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud'10) (2010)
31. Zhang, Z., Zhang, L., Xu, J., Chen, Z., Wang, P.: An edge-deployable lightweight intrusion detection system for industrial control. Electronics **15**(3), 644 (2026). https://doi.org/10.3390/electronics15030644